

Gattini su Instagram

A.Mas

AA 2024/2025

Contents

1	Robots and sensors	7
	MARRTino and Orazio	7
1.1	Sensors for ego-motion	7
1.2	Measuring the environment	7
1.3	Mobile base robots	7
1.3.1	MARRtino	8
1.3.2	Orazio	8
2	Introduction to probabilities	9
2.1	Random variables and basic concepts	9
2.2	Events and probability	9
2.3	Axioms	9
2.4	Continuous domains	9
2.4.1	Cumulative Density Function (CDF)	9
2.4.2	Probability Density Function (PDF)	9
2.4.3	Normal distribution	10
2.5	Robot environment interaction	10
2.6	State	10
2.7	Probabilistic generative laws	11
2.8	Belief distributions	11
2.9	Conditional probability	12
2.10	Joint distributions	12
2.11	Chain rule	12
2.12	Marginalization	12
2.13	Independence	13
2.14	Conditional independence	13
2.15	Bayes rule	13
2.15.1	Localizing Orazio	14
2.16	Probability identities	14
2.17	Functions of random variables: statistics	14
2.18	Entropy	14
3	Modeling dependencies Bayesian networks	15
3.1	Considerations about dimensions	15
3.2	Queries	15
3.3	Conditional independence	15
3.4	Making order	15
3.5	Example of inference	16
3.6	Considerations	16
3.7	Semantics	16
3.8	Bayes nets with discrete and continuous nodes	17
4	Dynamic Bayesian networks (Filtering)	17
4.1	Dynamic Bayesian Networks (DBN)	17
4.2	Inference in a DBN	18
4.3	Filtering: Bayes recursion	18
4.4	Filtering: alternative formulation	19
4.4.1	To sum up	19

5	Gaussian Distribution	20
5.1	Circles	20
5.2	Gaussian	20
5.2.1	Partitioned Gaussian densities	21
5.2.2	Marginalization	21
5.2.3	Conditioning	21
5.2.4	Chain rule	22
5.2.5	Affine transformations	22
5.2.6	Taylor expansion	22
6	Kalman filters	23
6.1	Odometric localization	23
6.1.1	Euler integration	23
6.1.2	2nd order Runge-Kutta integration	23
6.1.3	Exact integration	24
6.2	System model	24
6.2.1	Linear discrete-time system (no noise)	24
6.2.2	Linear discrete-time system with Gaussian noise	24
6.2.3	Nonlinear discrete-time system with noise	24
6.3	Linear systems and Gaussian noise	25
6.4	Kalman Filter	25
6.4.1	Kalman Filter without noise	25
6.4.2	Kalman Filter with process noise	26
6.4.3	Full Kalman filters	27
6.5	Stochastic approach	27
6.6	To sum up	28
6.6.1	Mathematical derivation of the KF	28
6.7	Non-Linear systems and Gaussian noise	29
6.8	Extended Kalman filter	29
6.8.1	Mathematical derivation of the EKF	29
6.8.2	Practical considerations	30
6.9	Summary	30
7	EKF SLAM applications	31
7.1	SLAM definition	31
7.1.1	Online SLAM problem	31
7.1.2	Full SLAM problem	31
7.1.3	Continuous and discrete components	31
7.1.4	Challenges	31
7.2	Application	31
7.2.1	Scenario	32
7.2.2	EKF	32
7.2.3	Domains: state space	32
7.2.4	Domains: control and measurements	32
7.2.5	Transition function	32
7.2.6	Measurement function	33
7.2.7	Control noise	33
7.2.8	Measurement noise	34
7.2.9	Jacobian 1: state	34
7.2.10	Jacobian 2: controls	34
7.2.11	Jacobian 3: measurements	35
7.2.12	Data association	35
7.2.13	Populating the map	35
7.2.14	Updating the map	36
7.3	Limitations of EKF SLAM	36
7.4	g2o wrapper	37

8	Data association	38
8.1	Finding the best association	38
8.1.1	Evaluation an association	38
8.2	Joint prediction	38
8.2.1	Independence loss	39
8.2.2	Using the joint prediction	39
8.3	Gaussian case	39
8.4	(En)forcing independence	40
8.4.1	Independence and assignment	41
8.5	Nearest neighbor	41
8.6	Greedy association	42
8.6.1	Pruning heuristic	42
8.7	To sum up	42
9	Unscented transform	43
9.1	Gaussian	43
9.1.1	Unscented Gaussian	43
9.2	Sigma points from parameters	43
9.3	Operations	44
9.3.1	Unscented marginalization	44
9.3.2	Unscented functions	44
9.3.3	Unscented chain rule	44
9.3.4	Unscented conditioning	45
9.4	System with Gaussian noise	45
9.5	Unscented Filtering	45
9.5.1	Predict	46
9.5.2	Update: chain rule	46
9.5.3	Update: conditioning	47
9.5.4	Wrapup (UKF)	47
9.6	Summary	47
10	Differentiation	48
10.1	Approaching the problem	48
10.2	Numerical differentiation	48
10.3	Chain rule	48
10.4	Parsing tree example	48
10.4.1	Parsing tree for derivatives	49
10.5	Atoms and unary functions	49
10.6	Binary operators	49
11	Particle distributions and particle filters	50
11.1	Introduction	50
11.2	Sampling from a distribution	50
11.2.1	Generating samples (assuming $p(x)$ in closed form)	50
11.2.2	Discrete case	50
11.2.3	Uniform sampling	51
11.2.4	Importance sampling	51
11.3	Resampling	51
11.3.1	Low variance sampling	51
11.4	Particle densities	52
11.4.1	Why particles are cool	52
11.5	Operations	53
11.5.1	Transformations	53
11.5.2	Marginalization	53
11.5.3	Chain Rule	53
11.5.4	Conditioning	53
11.6	Dynamic systems and the particle filter	54
11.6.1	Prediction step	54
11.6.2	Update step	55
11.6.3	Particle filter algorithm	55
11.7	Practical considerations and properties	55

12 Finding neighbours	56
12.1 Neighbour search	56
12.2 Distance metrics	56
12.2.1 Brute force approach	56
12.2.2 Distance map	56
12.2.3 KD-Trees	57
12.2.4 Other tools	57
12.3 Applications	58
13 Least squares introduction ICP optimization	59
13.1 Maximum Likelihood estimation	59
13.2 Gaussian assumption	59
13.3 Gauss-Newton method	59
13.3.1 Linearization	60
13.3.2 Minimizing the quadratic approximation	60
13.4 Least squares algorithm (one iteration)	60
13.4.1 Methodology	61
13.5 Gauss-Newton in SLAM	61
13.6 ICP optimization in 2D	61
13.6.1 State and measurements	62
13.6.2 Jacobian	62
13.6.3 Variants and practical remarks	62
13.6.4 Solving the linear system	63
13.6.5 Testing: good and bad initial guess	63
14 Least squares and uncertainty	64
14.1 Consequences of Taylor	64
14.2 Chain rule	64
14.3 Conditioning	64
14.4 Conclusions	65
15 3D Point Registration using ICP	66
15.1 Example ICP optimization in 3D	66
15.1.1 State and measurements	66
15.1.2 On rotation matrices	66
15.1.3 Jacobian	67
15.1.4 Sum up	67
15.2 ICP linear relaxation	67
16 Robust estimators	69
16.1 Gauss-Newton optimization	69
16.1.1 L2 norm	69
16.1.2 L1 norm	69
16.2 Outliers	69
16.2.1 M estimators	69
16.3 Robustifiers	70
16.3.1 Robust GN algorithm (one iteration)	70
17 Least squares on manifolds	72
17.1 Differences on a manifold	72
17.2 Least squares and Gauss in the manifold	72
17.2.1 Gauss in non-Euclidean spaces	72
17.2.2 Least square	73
17.2.3 Methodology	73
17.3 Example: ICP optimization in 2D	73
17.4 Uncertainty of the solution	75
17.4.1 Linearization	75
17.4.2 Unscented transform	75
17.4.3 Measurement uncertainty	75
17.5 Manifold least squares (Omega on chart)	75
17.6 Conclusion	76

18 ICP Optimization on a manifold	77
18.1 ICP 3D algorithm (one iteration)	77
18.2 Methodology	77
18.3 Testing	78
18.4 Outliers	78
18.4.1 Behavior with outliers	78
18.5 Conclusions	78
19 Registration on a manifold	80
19.1 Similarity registration in 3D	80
19.2 Similarity iterative cloud point (SICP)	80
19.2.1 State and boxplus	80
19.2.2 Operators	81
19.2.3 Measurements	81
19.2.4 Error and Jacobian	81
19.3 Exercise	82
20 Mini recap of epipolar geometry on unit sphere	83
20.1 Lines	83
20.1.1 Lines, closest point	83
20.1.2 Line intersection	83
20.1.3 Line intersection (camera-specific)	83
20.2 Essential matrix	84
20.3 Fundamental matrix	84
20.4 Essential matrix vs vs fundamental matrix vs homography	85
20.5 Extracting R and t from E	85
20.6 Estimating E (8 point)	86
20.6.1 Considerations	86
20.6.2 The normalized 8-point algorithm (not in the course slides)	86
20.7 How to get T ?	87
20.8 Concusions	87
21 RANSAC	89
21.1 3D point registration	89
21.2 RANSAC: random sample consensus	89
21.3 RANSAC: What do we need	90
21.4 Correspondences	90
21.5 Determining a solution	90
21.6 Pruning correspondences	90
21.7 Selecting inliers	90
21.8 How many rounds?	90
21.9 Pros and cons	91
22 Multi-Point Registration	92
22.1 State	92
22.1.1 State (implementation)	92
22.2 Boxplus	93
22.3 Measurements and predictions	93
22.4 Jacobians	93
22.5 H Matrix and B vector	94
22.6 Degrees of Freedom	94
22.6.1 Eliminating variables	94
22.6.2 Adding a prior	95
22.7 Structure	95
22.8 Conclusions	95

23 Multi-pose registration and graph-SLAM	96
23.1 Graph-based SLAM in a nutshell	96
23.2 What does the graph look like?	96
23.3 Graph Optimization	96
23.4 The Edge Information Matrices	97
23.5 Pose graph	97
23.5.1 State	97
23.5.2 Measurements and predictions	97
23.5.3 Boxplus	98
23.5.4 Jacobians	98
23.6 Information Matrix	98
23.7 H matrix and b vector	99
23.8 Chordal Distance	99
23.8.1 Prediction and error	100
23.8.2 Jacobian	100
23.9 Conclusions	101
24 Factor graphs	102
24.1 Large problems	102
24.2 Large and sparse problems	102
24.3 Graphical representation	102
24.4 Structure	103
24.5 Structure and efficiency	103
24.6 Solving Sparse Systems	103
24.6.1 Permutations and Cholesky	104
24.6.2 Comments	104
24.7 Total least squares	104
24.7.1 Pose-Landmark Constraint	104
24.7.2 Pose-proj constraint	105
24.7.3 Pose-pose constraint	105
24.8 Conclusions	105

1 Robots and sensors

MARRTino and Orazio

1.1 Sensors for ego-motion

Some sensors for mobile robots are

- wheels encoders (mounted on the wheels),
- IMU[IMuAcvsGyro], so a combination of
 - accelerometers: used to measure the acceleration, mechanically, it's a spring; the longer the spring stretches, the bigger the acceleration is. We can implement it with a micro mechanic device,
 - gyroscope: used to measure the rotational changing or maintaining orientation, it's based on the principle of preserving angular momentum. Typically, a gyro is composed of a rotor suspended inside three rings (gimbals)

The estimate of ego-motion is obtained by integrating the sensor measurements of these devices. This results in an accumulated drift due to the noise affecting the measurement. In absence of an external reference, there is no way to recover from these errors.

1.2 Measuring the environment

There are two main type of sensors to measure the environment:

- active:
 - ultrasound/sonars: they have an extensive field of view (FOV);
 - laser range finder: wide FOV, highly accurate (since it works both with the phase of the sinusoidal), secure for collision detection;
 - structured-light cameras;
 - infrared;
- passive:
 - RGB cameras:
 - * monocular cameras: those measure the intensity of the light projected onto a (planar) ccd¹ through a system of lenses and/or mirrors. It provides a lot of information, however it projects a 3D scene on a 2D plane, so we loose depth,
 - * stereo cameras: a combination of two monocular cameras that allow triangulation known the geometry of the environment and the camera parameters; this means that, if the correspondence of points in different images is known, we can reconstruct the 3D scene, having depth. However, there is an error in depth that depends in the distance. RGB stereo camera are sensible to lack of texture;
 - * RGB-D cameras (kinect): with this type of camera we are able to sense color and depth even with little to no texture. Using an active light source it's possible to retrieve depth either via stereo triangulation² or via time of flight³. Typically ok when used indoors since environment conditions should allow to sense the emitted light;
 - tactiles.

1.3 Mobile base robots

A mobile robot is composed of a mobile platform that can move in the environment carrying a certain load and has sensors and actuators. At a low level, the inputs are the desired velocity of the joints and the output is the state of the joints; at high level, the controlled input is the linear or angular velocity of the wheel and the output is the relative position of the base with respect to an initial configuration, obtained via odometry (by integration of the joint's states).

¹Charge Coupled Device

²emitter and source are in different positions

³emitter and source are in the same position

1.3.1 MARRtino

A simple, cheap but complete mobile robot. It's entirely open source and integrated in ROS through a simple node that publishes/subscribes standard topics.

1.3.2 Orazio

A simplified redesign of MARRtino, uses Arduino and it's easier to assemble. It's entirely open source and integrated in ROS through a simple node that publishes/subscribes standard topics.



((a)) MARRtino
[RobSens]



((b)) Orazio
[RobSens]

2 Introduction to probabilities

Probabilistic robotics models quantities such as sensor measurements, controls, and robot/environment states as random variables.

Probabilistic inference involves calculating probabilistic laws for variables derived from others and observed data.

The goal is often to estimate or make decisions in complex environments.

2.1 Random variables and basic concepts

A **random variable** (denoted X) can take on multiple values according to specific probabilistic laws: a specific value is denoted by x , the space of values a random variable can take can be discrete or continuous.

For a discrete random variable X , $p(X = x)$ denotes the probability that X has value x . To simplify notation, this is often abbreviated as $p(x)$.

2.2 Events and probability

An **event** is a specific outcome or a set of outcomes for a random variable, such as "the signal strength S falls in the interval s_i ".

Formally, a probability is a function mapping each measurable subset of the event space Ω to the interval $[0, 1]$

$$P : \Omega \rightarrow [0, 1] \quad (2.1)$$

Probabilities are always non-negative ($p(X = x) \geq 0$).

2.3 Axioms

Valid probability assignments must fulfill the following axioms:

1. the probability of an event is greater than or equal to 0: $P(E) \geq 0$
2. the probability of the union of a set of disjoint events is the sum of the probabilities of the individual events: $P(\bigcup_i E_i) = \sum_i P(E_i)$
3. the probability of the outcome being in the set of all possible outcomes (Ω or Σ) is 1: $P(\Sigma) = 1$ or $P(\Omega) = 1$

For discrete distributions, the probabilities of all possible values sum to one: $\sum_x p(X = x) = 1$.

2.4 Continuous domains

In the case of **continuous spaces**, random variables can take on a continuum of values. For continuous signals, considering infinitesimally small intervals makes recovering probability through statistics difficult. The probability of a continuous random variable assuming exactly a specific value is generally 0.

2.4.1 Cumulative Density Function (CDF)

For continuous domains, the **cumulative density function** (CDF) is defined as the probability that the random variable X is lower than a certain value x

$$F(x) = P(X < x) \quad (2.2)$$

The CDF is a monotonically increasing function.

2.4.2 Probability Density Function (PDF)

For continuous distributions, we consider the **probability density function** (PDF), denoted $p(x)$. It is defined as the derivative of the CDF:

$$p(x) = \frac{\partial P(x)}{\partial x} \quad (2.3)$$

To compute the probability for a generic event $E \subseteq \Omega$, we integrate the PDF over the event space:

$$P(E) = \int_{x \in E} p(x) dx \quad (2.4)$$

An integration corresponds to summing up all the infinitesimal disjoint events that cover E . Just as discrete probabilities sum to 1, a PDF always integrates to 1: $\int p(x)dx = 1$.

However, unlike a discrete probability, the value of a PDF is not upper-bounded by 1. The terms probability, probability density, and probability density function are often used interchangeably in the context of continuous random variables.

2.4.3 Normal distribution

The **normal distribution** is a common continuous distribution that plays a major role in probabilistic robotics. Unless explicitly stated, continuous random variables are assumed to possess PDFs. The PDF of a one-dimensional normal distribution with mean μ and variance σ^2 is given by the following Gaussian function:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2} \frac{(x - \mu)^2}{\sigma^2}\right) \quad (2.5)$$

This is often abbreviated as $\mathcal{N}(x; \mu, \sigma^2)$. For multi-dimensional vectors $\mathbf{x}$, normal distributions are called multivariate and their density functions have the form:

$$p(\mathbf{x}) = \det(2\pi\Sigma)^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^T \Sigma^{-1}(\mathbf{x} - \mu)\right) \quad (2.6)$$

Where:

- μ is the mean vector,
- Σ is a positive semidefinite and symmetric matrix called the **covariance matrix**.

The argument in the exponent is quadratic in $\mathbf{x}$, with parameters μ and Σ . Equation ?? (multivariate) is a generalization of Equation ?? (scalar); they are equivalent if $\mathbf{x}$ is a scalar and $\Sigma = \sigma^2$.

2.5 Robot environment interaction

A robot interacts with its environment, which is a dynamical system with internal state. This interaction involves two fundamental types:

- the robot influences the state of its environment through its **actuators** via control actions. These actions change the state of the world. The effect is often unpredictable, affecting both the environment state and the robot's internal belief about that state. Even without explicit action, state usually changes, so a "do-nothing" action is assumed for consistency;
- the robot gathers information about the state through its sensors via environment **sensor measurements**. Perception is the process of using sensors (e.g., camera, range scan, tactile sensors) to obtain information about the environment's state. Sensor measurements are typically noisy and may arrive with delay. They are also called **observations** or percepts.

The robot can keep a record of these interactions, collectively called the **data**:

- **environment measurement data**, denoted z_t , provides information about the state at time t . Small timing effects are usually ignored, assuming the measurement corresponds to a specific point in time. Sequences of measurements are denoted $z_{t_1:t_2} = z_{t_1}, z_{t_1+1}, \dots, z_{t_2}$ for $t_1 \leq t_2$.
- **control data**, denoted u_t , carries information about the change of state in the environment in the time interval $(t-1; t]$. Examples include robot velocity or odometers, which measure the effect of a control action. Odometers, despite being sensors, are treated as control data because they convey state change information. Sequences are denoted $u_{t_1:t_2} = u_{t_1}, u_{t_1+1}, \dots, u_{t_2}$ for $t_1 \leq t_2$.

The distinction is crucial: perception increases knowledge, while motion/control tends to induce a loss of knowledge due to noise and stochasticity. Perception and control occur concurrently.

2.6 State

The **state** of the environment is the collection of all aspects of the robot and its environment that can impact the future. State at time t is denoted x_t .

- Dynamic state changes over time (e.g., location of people).

- Static state tends to remain constant (e.g., location of walls).

State also includes variables about the robot itself like:

- **robot pose**: for planar robots, this is often 3 variables (2D location and heading);
- **kinematic state**: like joint angles in manipulation;
- **dynamic state**: velocity of robot or joints;
- **landmarks**: location and features of distinct and stationary surrounding objects;
- location and velocities of other moving objects/people;
- sensor status or battery level.

A state x_t is **complete** if it is the best predictor of the future: past states, measurements, or controls offer no additional predictive information.

This relates to the concept of Markov chains.

Completeness is mostly a theoretical concept, and practical implementations use an incomplete state by selecting a subset of variables.

State can be continuous (e.g., pose), discrete (e.g., sensor broken status), or hybrid (containing both). Time is treated as discrete ($t = 0, 1, 2, \dots$).

2.7 Probabilistic generative laws

The evolution of state and measurements is governed by probabilistic laws. The state x_t is generated stochastically from x_{t-1} . In general, this could depend on all past states, measurements, and controls:

$$p(x_t | x_{0:t-1}, z_{1:t-1}, u_{1:t}) \quad (2.7)$$

However, if the state x is complete, x_{t-1} is a sufficient summary of previous events, and the state evolution depends only on the previous state and the current control u_t :

$$p(x_t | x_{0:t-1}, z_{1:t-1}, u_{1:t}) = p(x_t | x_{t-1}, u_t) \quad (2.8)$$

This property is an example of **conditional independence**. The probability $p(x_t | x_{t-1}, u_t)$ is called the **state transition probability**. It specifies how the environment state evolves over time based on robot controls u_t .

Robot environments are stochastic, hence this is a probability distribution. A time-invariant form is $p(x' | u, x)$.

Measurements are generated based on the state. If x_t is complete, the measurement z_t depends only on the current state x_t :

$$p(z_t | x_{0:t}, z_{1:t-1}, u_{1:t}) = p(z_t | x_t) \quad (2.9)$$

Knowledge of other variables is irrelevant if x_t is complete.

The probability $p(z_t | x_t)$ is called the measurement probability. It specifies the probabilistic law for generating measurements z from state x . It is often appropriate to think of measurements as noisy projections of the state. A time-invariant form is $p(z | x)$. The state transition probability $p(x_t | x_{t-1}, u_t)$ and the measurement probability $p(z_t | x_t)$ describe the dynamical stochastic system.

2.8 Belief distributions

A **belief**⁴ reflects the robot's internal knowledge about the environment state. Since the true state often cannot be measured directly (e.g., robot pose), the robot must infer it from data. Probabilistic robotics represents beliefs using conditional probability distributions: these are posterior probabilities over state variables conditioned on available data. The belief over state x_t is denoted $bel(x_t)$, which is an abbreviation for the posterior probability distribution over x_t at time t , conditioned on all past measurements $z_{1:t}$ and controls $u_{1:t}$:

$$bel(x_t) = p(x_t | z_{1:t}, u_{1:t}) \quad (2.10)$$

This belief is calculated after incorporating the measurement z_t . A posterior calculated just after executing control u_t but before incorporating z_t is denoted $\overline{bel}(x_t)$:

$$\overline{bel}(x_t) = p(x_t | z_{1:t-1}, u_{1:t}) \quad (2.11)$$

This is often referred to as the **prediction** in probabilistic filtering. It predicts the state at time t based on the previous belief and the current control. Calculating $bel(x_t)$ from $\overline{bel}(x_t)$ by incorporating z_t is called correction or the measurement update.

⁴Also known as state of knowledge or information state

2.9 Conditional probability

Conditional probability allows us to determine the probability of an event given that another event has occurred. Given two random variables X and Y , the probability that X takes value x given that Y takes value y is denoted:

$$p(x|y) = p(X = x|Y = y) \quad (2.12)$$

If $p(y) > 0$, the conditional probability is defined as:

$$p(x|y) = \frac{p(x, y)}{p(y)} \quad (2.13)$$

In the Orazio scenario, $P(S = s_i|L_j)$ denotes a distribution over signal strengths given the knowledge of the location L_j . If X and Y are independent, then

$$p(x|y) = p(x)p(y)/p(y) = p(x) \quad (2.14)$$

Knowing Y tells nothing about X if they are independent. Conditional probabilities can also be conditioned on additional variables, for example $p(x|y, z)$. Properties of conditional probability $P(A|B)$ given $P(B) > 0$:

- if A and B are incompatible ($A \cap B = \emptyset$), then $P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{P(\emptyset)}{P(B)} = 0$;
- if $B \subset A$ (B is a subset of A), then $A \cap B = B$, so $P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{P(B)}{P(B)} = 1$;
- if $A \subset B$ (A is a subset of B), then $A \cap B = A$, so $P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{P(A)}{P(B)}$.

2.10 Joint distributions

The joint distribution of two random variables X and Y describes the probability of the event that X takes value x and Y takes value y . It is given by $p(x, y) = p(X = x \text{ and } Y = y)$. The domain of the joint distribution $P(S, L)$ is all possible pairs of signal strengths and locations. It can be visualized as a grid where columns are locations and rows are signal strengths.

2.11 Chain rule

The definition of conditional probability $p(x|y) = \frac{p(x, y)}{p(y)}$ can be rearranged to express the joint probability in terms of conditional and marginal probabilities:

$$p(x, y) = p(x|y)p(y) \quad (2.15)$$

Similarly, $p(x, y) = p(y|x)p(x)$. Thus, the **chain rule** states

$$p(x, y) = p(x|y)p(y) = p(y|x)p(x) \quad (2.16)$$

The joint distribution can be obtained by multiplying each element of the conditional distribution by the probability of the conditioning variable.

2.12 Marginalization

From a joint distribution $p(x, y)$, we can obtain the distribution of a single variable, called the marginal distribution, by summing or integrating over the other variable(s). This process is called marginalization. For discrete variables, suppressing variable Y from $p(x, y)$ gives the marginal distribution $p(x)$ by summing over all possible values of y :

$$p(x) = \sum_y p(x, y) = \sum_y p(x|y)p(y) \quad \text{discrete case} \quad (2.17)$$

This process corresponds to summing a row or column in a joint distribution table. For continuous variables, marginalization is done by integrating:

$$p(x) = \int p(x, y)dy = \int p(x|y)p(y)dy \quad \text{continuous case} \quad (2.18)$$

These formulas are also known as the **Theorem of Total Probability**. This theorem is used, for example, to compute the denominator $p(y)$ in Bayes rule.

2.13 Independence

Two events (or random variables) A and B are independent if the event B does not modify the probability of event A . The independence is reciprocal

$$P(A|B) = P(A) \implies P(B|A) = P(B) \quad (2.19)$$

If X and Y are independent, their joint distribution is the product of their individual distributions:

$$p(x, y) = p(x)p(y) \quad (2.20)$$

This follows directly from the definition of conditional probability. To test if two variables are independent from a joint table, one can

1. compute the marginals,
2. compute a hypothetical joint by multiplying the marginals (assuming independence),
3. compare this to the original joint table.

If they are the same, the variables are independent.

2.14 Conditional independence

Two variables x and y are conditionally independent given a variable z if y carries no information about x when the value of z is known. The relationship for conditional independence is:

$$p(x, y|z) = p(x|z)p(y|z) \quad (2.21)$$

Equivalent forms are $p(x|z) = p(x|z, y)$ and $p(y|z) = p(y|z, x)$. In the Orazio scenario with two WiFi signals, signal strengths S_A and S_B are implicitly correlated because both depend on the robot's location L . If Orazio's location L is unknown, knowing S_A gives information about L , which in turn gives information about S_B . However, if the location L is known, the signals S_A and S_B are independent given L :

$$P(S_A, S_B|L) = P(S_A|L)P(S_B|L) \quad (2.22)$$

Conditional independence does not imply absolute independence:

$$p(x, y|z) = p(x|z)p(y|z) \not\Rightarrow p(x, y) = p(x)p(y) \quad (2.23)$$

Conversely, absolute independence does not imply conditional independence:

$$p(x, y) = p(x)p(y) \not\Rightarrow p(x, y|z) = p(x|z)p(y|z) \quad (2.24)$$

2.15 Bayes rule

Bayes rule is fundamental in probabilistic inference, relating a conditional probability $p(x|y)$ to its "inverse" $p(y|x)$. It requires $p(y) > 0$:

$$p(x|y) = \frac{p(y|x)p(x)}{p(y)} \quad (2.25)$$

Using the theorem of total probability (marginalization) for the denominator $p(y)$ gives the alternative forms:

$$\begin{aligned} p(x|y) &= \frac{p(y|x)p(x)}{\sum_{x'} p(y|x')p(x')} && \text{Discrete} \\ p(x|y) &= \frac{p(y|x)p(x)}{\int p(y|x')p(x')dx'} && \text{Continuous} \end{aligned} \quad (2.26)$$

In this context:

- $p(x)$ is the prior probability distribution over X , summarizing knowledge about X before incorporating data,
- y is the data (e.g., a sensor measurement),
- $p(x|y)$ is the posterior probability distribution over X after incorporating data y .

. Bayes rule provides a way to compute the posterior $p(x|y)$ using the inverse conditional probability $p(y|x)$ (often called the generative model, as it describes how state causes measurements) and the prior $p(x)$.

The denominator $p(y)$ does not depend on x and acts as a normalizer. It is often denoted by η for brevity:

$$p(x|y) = \eta p(y|x)p(x) \quad (2.27)$$

where $\eta = p(y)^{-1}$. Different equations may use the same symbol η for different normalization constants. Bayes rule can be conditioned on other variables, e.g., conditioning on $Z = z$ gives:

$$p(x|y, z) = \frac{p(y|x, z)p(x|z)}{p(y|z)} \quad (2.28)$$

2.15.1 Localizing Orazio

The problem of localizing Orazio given a measured signal strength is a direct application of calculating a posterior distribution using principles related to Bayes rule. Given a measured signal strength s_2 , we want to find the probability distribution over possible locations L_j , i.e., $P(L_j|S_2)$. Using the definition of conditional probability $P(A|B) = P(A, B)/P(B)$, we have:

$$P(L_j|S_2) = \frac{P(S_2, L_j)}{P(S_2)} \quad (2.29)$$

In a joint distribution table $P(S, L)$, $P(S_2, L_j)$ represents entries in the row corresponding to s_2 . The term $P(S_2)$ is the marginal probability of observing signal strength s_2 , obtained by summing the values in that row: $P(S_2) = \sum_j P(S_2, L_j)$.

Dividing each element $P(S_2, L_j)$ by this sum $P(S_2)$ normalizes the row to become a valid probability distribution over locations, representing the probability of being at location L_j given the measured signal S_2 .

2.16 Probability identities

A probability equation derived by applying the axioms and exploiting conditional independence from the domain holds regardless of the specific shape of the densities. A consequence is that the same conditioning term can be added to all terms of a probability equation, and it will still hold. For example, if $p(x, y) = p(x|y)p(y)$ holds, then $p(x, y|z) = p(x|y, z)p(y|z)$ also holds.

2.17 Functions of random variables: statistics

Statistical features of probability distributions, such as expectation and covariance, are important. The **expectation**⁵ of a random variable X , $E[X]$, is its average value.

$$\begin{aligned} E[X] &= \sum x p(x) && \text{Discrete} \\ E[X] &= \int x p(x) dx && \text{Continuous} \end{aligned} \quad (2.30)$$

Expectation is a linear function: $E[aX + b] = aE[X] + b$ for constants a, b . The **covariance** of X , $Cov[X]$, measures the squared expected deviation from the mean:

$$Cov[X] = E[X - E[X]]^2 = E[X^2] - E[X]^2 \quad (2.31)$$

For a multivariate normal distribution $\mathcal{N}(\mathbf{x}; \mu, \Sigma)$, the mean is μ and the covariance is Σ .

2.18 Entropy

Entropy**, $H_p(x)$, is a concept from information theory defined for a probability distribution $p(x)$. It is given by the expectation of $-\log_2 p(x)$:

$$H_p(x) = E[-\log_2 p(x)] \quad (2.32)$$

$$\begin{aligned} H_p(x) &= -\sum p(x) \log_2 p(x) && \text{Discrete} \\ H_p(x) &= -\int p(x) \log_2 p(x) dx && \text{Continuous} \end{aligned} \quad (2.33)$$

Entropy represents the expected information that the value of x carries. In probabilistic robotics, it is used in information gathering to express the information a robot might receive from executing actions.

⁵or expected value

3 Modeling dependencies Bayesian networks

This section discusses modeling dependencies between variables, particularly using Bayesian Networks.

3.1 Considerations about dimensions

When modeling systems with multiple interacting components and observations, the number of possible distinct outcomes or states can grow very large: consider a system with a robot, two access points (APs), and an elevator. If we track the robot's location (100 possibilities), the signal strengths from two APs (4 levels each), and the elevator's floor (6 floors), the total number of distinct outcomes is $4 \times 4 \times 6 \times 100 = 9600$. Adding a simple monochromatic camera with 10x10 resolution and 16 gray levels increases the complexity dramatically, resulting in $9600 \times 16^{(10 \times 10)}$ disjoint events, a number large enough to pose a significant computational challenge.

3.2 Queries

To answer a probabilistic query, such as determining the probability distribution of one variable (e.g., signal strength from AP A, S^A) given the value of another (e.g., signal strength from AP B, S_2^B) $p(S^A|S_2^B)$, we typically follow these steps:

1. start with the joint probability distribution (JPD) over all relevant variables in the system, for example, $P(S^A, S_2^B, I_{1,1}, \dots, I_{10,10}, L, E)$ including signal strengths (S^A and S_2^B), image pixels ($I_{1,1}, \dots, I_{10,10}$), robot location (L), and elevator state (E);
2. eliminate all non-relevant variables through marginalization to obtain the joint distribution over the variables involved in the query, like $P(S^A, S_2^B, I_{1,1}, \dots, I_{10,10}, L, E)$ marginalized to $P(S^A, S_2^B)$. Marginalization involves summing out variables that are not part of the desired query;
3. use Bayes' rule to obtain the answer in the form of the desired conditional probability distribution.

3.3 Conditional independence

Understanding the underlying phenomena allows us to identify relationships and, crucially, conditional independencies. For example, the signal strength from an access point might depend only on the robot's location, and pixel intensities in an image might also depend primarily on location, assuming stable environmental conditions. The elevator's state could be largely independent unless the robot interacts with it. These observed independencies are key to building efficient models.

3.4 Making order

Probabilistic graphical models combine probability theory and graph theory to handle uncertainty and complexity, providing a compact representation of joint probability distributions.

In these models, **nodes** represent **random variables**, and the **arcs** represent conditional **dependence**, meaning the absence of an arc implies conditional independence.

Two main types exist:

1. undirected graphical models (Markov random fields/networks): independence is defined by graph separation; A and B are conditionally independent given C if C separates all paths between A and B;
2. directed graphical models (Bayesian networks/belief networks): these use directed arcs, which can represent causal relationships where A "causes" B. The notion of independence is more complex, taking directionality into account.

For directed models, we need to specify the **conditional probability distribution** (CPD) for each node. For discrete variables, this is a conditional probability table (CPT) listing the probability of a child's values given its parents' values. A node with no parents (a root) has its prior probability specified by its CPT.

The conditional independence encoded by a Bayesian network implies that a node is independent of its non-descendants (its ancestors) given its parents.

This property allows for a significantly more compact representation of the joint distribution. Instead of $O(2^n)$ for n binary nodes in a full JPD, a BN requires $O(n2^k)$ space, where k is the maximum number of parents (fan-in) for any node. This reduction in parameters also aids learning.

Bayesian Networks are named for their use of **Bayes' rule for inference**, not necessarily for adhering to Bayesian statistics for parameter estimation; frequentist methods are commonly used. Storing the conditional probabilities in a BN requires far fewer numbers than storing the full JPD: for the Orazio example, this reduces

the required numbers from $O(10^{124})$ to roughly 160,906. The full joint distribution can still be recovered from the conditional probabilities using the chain rule.

3.5 Example of inference

Let's consider the example of computing the probability distribution of signal strength from AP_A (S_A) given that a specific pixel (1,1) has intensity 7 ($I_{1,1} = 7$). To find $P(S_A|I_{1,1} = 7)$, we can use the structure of the Bayesian Network: assuming signal strengths and pixel intensities are conditionally independent given the robot's location (L), the process involves:

1. writing the joint probability based on the network structure

$$P(S_A, S_B, \{I_{i,j}\}, L, E) = P(S_A|L)P(S_B|L)P(\{I_{i,j}\}|L)P(L)P(E) \quad (3.1)$$

where $P(\{I_{i,j}\}|L)$ is the product of probabilities for individual pixels given the location:

$$P(\{I_{i,j}\}|L) = \prod_{i,j} P(I_{i,j}|L) \quad (3.2)$$

2. marginalizing out variables not in the query (S_B , other pixels, E , and L) to get

$$P(S_A, I_{1,1} = 7) \quad (3.3)$$

3. applying Bayes' rule to get the conditional probability

$$P(S_A|I_{1,1} = 7) = \frac{P(S_A, I_{1,1} = 7)}{P(I_{1,1} = 7)} \quad (3.4)$$

3.6 Considerations

In the pixel intensity example, knowing the location (L) simplifies inference significantly because S_A and $I_{1,1}$ become conditionally independent given L (if L is their only common ancestor on paths connecting them). If we know L , then $P(S_A|I_{1,1} = 7, L) = P(S_A|L)$. Knowing other pixel intensities would not provide additional information about S_A if L is known. If L is unknown, knowing $I_{1,1} = 7$ provides evidence that updates our belief about L , and this updated belief about L then informs our belief about S_A . The elevator's state, being disconnected in the example network, is independent of the other variables.

3.7 Semantics

The conditional independence properties of a Bayesian network are captured by its semantics:

- local semantics: a node is conditionally independent of its non-descendants given its parents;
- global semantics: a node is independent of all other nodes in the network (the rest) given its Markov blanket, which consists of its parents, its children, and the parents of its children.

These semantics allow us to prune the network for inference, ignoring variables that are independent of the query given observed evidence. The process of using evidence (like $I_{1,1} = 7$) to update beliefs about causes (like L) and then effects (like S_A) involves incorporating the new knowledge into the prior belief, altering it to a posterior.

The **Bayes Ball algorithm** (fig. ??) is an intuitive way to determine d-separation (conditional independence) in a BN: a ball can pass between nodes A and B given observed set C if there is a path that is not "blocked". Paths are blocked if:

- X is observed, by:
 - serial connections $A \rightarrow X \rightarrow B$,
 - diverging connections $A \leftarrow X \rightarrow B$,
- X is hidden and none of its descendants are observed, by converging connections $A \rightarrow X \leftarrow B$.

Conversely, paths pass if:

- X is hidden and there are serial or diverging connections,
- X or any of its descendants are observed and there are converging connections.

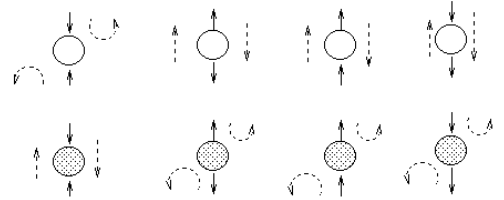


Figure 3.1: Bayesian ball example

3.8 Bayes nets with discrete and continuous nodes

Bayesian networks can model relationships between both discrete and continuous variables:

- discrete nodes often use multinomial distributions (CPTs) or softmax distributions,
- continuous nodes commonly use Gaussian distributions (CPDs).

Complex models can be built by combining these distribution types.

4 Dynamic Bayesian networks (Filtering)

This section covers dynamic systems, their probabilistic modeling using Dynamic Bayesian Networks, and key inference techniques, particularly filtering.

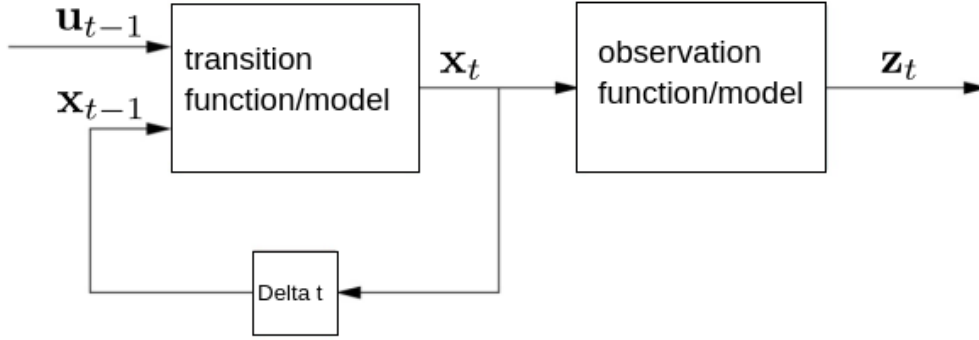


Figure 4.1: Dynamic system, deterministic/probabilistic view

A dynamic system changes state over time. In a **deterministic** view, the evolution is governed by functions:

- transition function: $x_t = f(x_{t-1}, u_{t-1})$ determines the current state x_t from the previous state x_{t-1} and control u_{t-1} ,
- observation function: $z_t = h(x_t)$ determines the current observation z_t from the current state x_t .

In a **probabilistic** view, the evolution and observation are described by probability distributions:

- transition model: $p(x_t|x_{t-1}, u_{t-1})$ gives the probability of reaching state x_t from x_{t-1} given control u_{t-1} ,
- observation model: $p(z_t|x_t)$ gives the probability of observing z_t given the state x_t .

The three key aspects of a dynamic system are:

1. an initial state distribution $p(x_0)$ from which the state evolves,
2. controls u_t that influence the state transition,
3. observations z_t that provide noisy information about the state.

This temporal dependency structure is recurrent.

4.1 Dynamic Bayesian Networks (DBN)

Dynamic Bayesian Networks (DBNs) are directed graphical models of stochastic processes. They have a characteristic recurrent structure across time slices. DBNs represent the state using multiple variables and their interdependencies, generalizing simpler models like Hidden Markov Models (HMMs) and Linear Dynamical Systems (LDSs).

The graphical structure facilitates specifying conditional independencies and enables a compact parameterization. The variables in a DBN (states, controls, observations) can be discrete or continuous.

A DBN is defined by its structure and parameters, typically specified for the first two time slices (a 2TBN), assuming the model is time-invariant.

Key components are:

- intra-slice topology: dependencies within a single time slice;

- inter-slice topology: dependencies between variables in consecutive time slices.

Examples include robot localization, HMMs, and LDSs.

DBNs assume the state x_t is **complete** (Markov), meaning past data doesn't influence the future given the current state. Violations can occur but DBNs are often robust in practice. More complex DBNs like the Bayesian Automated Taxi (BAT) network exist. When controls are viewed as inputs (actions) and observations as outputs (percepts), DBNs can be seen as partially observable Markov decision problems (POMDPs).

4.2 Inference in a DBN

Inference algorithms for DBNs estimate the distribution of states, maintaining a belief about the current or past state; the

- belief $b_{t|t}$ is the posterior probability $p(x_t|z_{1:t}, u_{1:t})$,
- prediction $p(x_t|t-1)$ ⁶ is the posterior $p(x_t|z_{1:t-1}, u_{1:t})$ before incorporating the latest measurement

Storing the belief depends on the state variable domain and distribution characteristics; for

- discrete states, an array of probabilities suffices;
- continuous Gaussian states, the mean and covariance matrix represent the belief;
- unknown continuous distributions, approximate representations are used.

Common inference problems given measurements $z_{1:T}$ and controls $u_{0:T-1}$ (which are usually known):

- **filtering**: compute the belief about the current state $p(x_T|u_{0:T-1}, z_{1:T})$;
- **smoothing**: compute the belief about a past state, $p(x_k|u_{0:T-1}, z_{1:T})$ for $0 < k < T$. This uses future data to improve the estimate of a past state;
- **maximum a posteriori (MAP)**: find the most likely sequence of states $x_{0:T}$, $\text{argmax}_{x_{0:T}} p(x_{0:T}|u_{0:T-1}, z_{1:T})$. Unlike filtering and smoothing, this yields a state sequence, not a distribution.

Directly applying general BN inference techniques is impractical for DBNs due to the potentially infinite state space and variable count. The recurrent structure must be leveraged.

4.3 Filtering: Bayes recursion

Filtering is recursively computed using the Bayes filter algorithm: update the belief $b(x_{t-1})$ at time $t-1$ to $b_{t|t}$ using the control u_t and measurement z_t . The algorithm steps are (for continuous states, the integral is a sum for discrete states):

```

1: Algorithm Bayes_filter(bel(xt-1), ut, zt):
2: for all xt do
3: bel(xt) = integral p(xt | ut, xt-1) bel(xt-1) dxt-1
4: bel(xt) = eta p(zt | xt) bel(xt)
5: endfor
6: return bel(xt)

```

The process involves two steps:

1. prediction: line 3 calculates the predicted belief $p(x_t|t-1)$ by incorporating the control u_t . This is done by integrating the product of the state transition probability $p(x_t|u_t, x_{t-1})$ and the previous belief $b(x_{t-1})$ over all possible previous states x_{t-1} :

$$p(x_t|t-1) = \int p(x_t|u_t, x_{t-1})b(x_{t-1})dx_{t-1} \quad (4.1)$$

2. correction (measurement update): line 4 calculates the updated belief $b_{t|t}$ by incorporating the measurement z_t . It multiplies the predicted belief $p(x_t|t-1)$ by the likelihood of the measurement $p(z_t|x_t)$ and normalizes the result:

$$b_{t|t} = \eta p(z_t|x_t)p(x_t|t-1) \quad (4.2)$$

⁶or $b_{t|t-1}$

The variable η is a normalization constant ensuring the result is a valid probability distribution. It is calculated as the inverse of the sum (or integral) of the non-normalized belief over all states x_t :

$$\eta = \left(\int p(z_t|x_t)p(x_t|t-1)dx_t \right)^{-1} \quad (4.3)$$

The derivation of the Bayes filter equations relies on Bayes' rule and the Markov assumption (complete state). The update step $b_{t|t} = \eta p(z_t|x_t)p(x_t|t-1)$ is derived from

$$p(x_t|z_{1:t}, u_{1:t}) = \eta p(z_t|x_t, z_{1:t-1}, u_{1:t})p(x_t|z_{1:t-1}, u_{1:t}) \quad (4.4)$$

by using the Markov property

$$p(z_t|x_t, u_{0:t-1}, z_{1:t-1}) = p(z_t|x_t) \quad (4.5)$$

The prediction step $p(x_t|t-1) = \int p(x_t|x_{t-1}, u_t)b(x_{t-1})dx_{t-1}$ is derived from $p(x_t|z_{1:t-1}, u_{1:t}) = \int p(x_t|x_{t-1}, z_{1:t-1}, u_{1:t})p(x_{t-1}|z_{1:t-1}, u_{1:t-1})dx_{t-1}$ by using the Markov properties $p(x_t|x_{t-1}, z_{1:t-1}, u_{1:t}) = p(x_t|x_{t-1}, u_t)$ and $p(x_{t-1}|z_{1:t-1}, u_{1:t}) = p(x_{t-1}|z_{1:t-1}, u_{1:t-1})$ for random controls.

4.4 Filtering: alternative formulation

Filtering can be alternatively understood through the prediction and update cycles.

- Predict: compute the predicted belief $p(x_t|t-1)$ from the previous belief $b(x_{t-1})^7$ and control u_t . This uses the transition model

$$p(x_t|x_{t-1}, u_{t-1}) \quad (4.6)$$

The predicted belief is obtained by:

1. computing the joint distribution $p(x_t, x_{t-1})$ using the chain rule

$$p(x_t, x_{t-1}|t-1) = p(x_t|x_{t-1}, u_{t-1})p(x_{t-1}|t-1) \quad (4.7)$$

and then marginalizing out x_{t-1}

$$p(x_t|t-1) = \int p(x_t|x_{t-1}, u_t)b(x_{t-1})dx_{t-1} \quad (4.8)$$

- update: compute the updated belief $b_{t|t}$ from the predicted belief $b_{t|t-1}$ and measurement z_t . This uses the observation model $p(z_t|x_t)$. The updated belief is found by

1. computing the joint distribution $p(x_t, z_t)$ using the chain rule

$$p(x_t, z_t|t) = p(z_t|x_t)p(x_t|t-1) \quad (4.9)$$

2. conditioning on the actual measurement z_t and normalizing

$$b_{t|t} = \frac{p(x_t, z_t|t)}{p(z_t|t)} \quad (4.10)$$

4.4.1 To sum up

Implementing the Bayes filter requires:

- the initial belief $p(x_0)$,
- the transition model $p(x_t|u_t, x_{t-1})$,
- the observation model $p(z_t|x_t)$,
- choosing a suitable representation for the belief distribution.

Exact implementations are limited, often requiring approximations with trade-offs in efficiency, accuracy, and ease of implementation.

⁷or $p(x_{t-1}|t-1)$

5 Gaussian Distribution

5.1 Circles

A circle has equation $x^2 + y^2 = r^2$ and is a slice of a paraboloid.

It's possible to perform some operations on the circle, like

- scaling $a_{xx}x^2 + a_{yy}y^2 = r^2$,
- slanting $a_{xx}x^2 + a_{xy}xy + a_{yy}y^2 = r^2$.

In particular, the last operation can be re-written in matricial form as

$$\begin{pmatrix} x & y \end{pmatrix} \begin{pmatrix} a_{xx} & \frac{a_{xy}}{2} \\ \frac{a_{xy}}{2} & a_{yy} \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = r^2 \quad (5.1)$$

The matrix $A = \begin{pmatrix} a_{xx} & \frac{a_{xy}}{2} \\ \frac{a_{xy}}{2} & a_{yy} \end{pmatrix}$ is a symmetric matrix, therefore can be re-written using the eigenvalue decomposition:

$$A = R^T \begin{pmatrix} \Omega_1 & \\ & \Omega_2 \end{pmatrix} R \quad (5.2)$$

This is because the off diagonal components a_{xy} rotate the ellipsoid. Since an ellipse is nothing more than a slanted circle, it can be rotated, translated and re-written using eigenvalue decomposition:

$$A = \left[R \begin{pmatrix} x - x_c \\ y - y_c \end{pmatrix} \right]^T \begin{pmatrix} \Omega_1 & \\ & \Omega_2 \end{pmatrix} \left[R \begin{pmatrix} x - x_c \\ y - y_c \end{pmatrix} \right] = r^2 \quad (5.3)$$

5.2 Gaussian

The Gaussian PDF is, expressed as the moment form/parametrization,

$$\mathcal{N}(x; \mu, \Sigma) \triangleq \frac{1}{(2\pi)^{n/2} \sqrt{\det \Sigma}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right) \quad (5.4)$$

where:

- $x \in R^n$ is a random vector that's Gaussian distributed,
- $\mu = \frac{1}{N} \sum x^i \in R^n$ is the mean,
- $\Sigma = \frac{1}{N-1} \sum (x^i - \mu)(x^i - \mu)^T \in S_{++}^n$ ⁸ is the covariance. Σ represent the uncertainty: the larger it is, the larger the Gaussian is. Moreover, it is squared and symmetric since it is the sum of symmetric matrices (we are multiplying a row vector - displacement from the mean and the sampled terms - with its transpose)

Another way to represent the parameters is using the expected value:

$$\mu = \int_{\Sigma} x p(x) dx = E[x] \quad \Sigma = \int_{\Sigma} (x - \mu)(x - \mu)^T p(x) dx = E[(x - \mu)(x - \mu)^T]$$

An alternative representation is the information/canonical/natural form of the Gaussian density:

$$\mathcal{N}^{-1}(x; \nu, \Omega) \triangleq \frac{\exp \left(-\frac{1}{2} \nu^T \Omega^{-1} \nu \right)}{(2\pi)^{n/2} \sqrt{\det \Omega^{-1}}} \exp \left(-\frac{1}{2} x^T \Omega x + x^T \nu \right) \quad (5.5)$$

where

- ν is the information vector,
- Ω is the information matrix (the higher it is, the more information I have, so the more certain I am. While Σ is a measure of uncertainty, Ω is a measure of certainty).

⁸ S_{++}^n is the n -dimensional positive definite cone

Equations ?? and ?? are related as

$$\begin{aligned}\mathcal{N}(x; \mu, \Sigma) &= \frac{1}{(2\pi)^{n/2} \sqrt{\det \Sigma}} \exp \left(-\frac{1}{2} x^T \Sigma^{-1} x + x^T \Sigma^{-1} \mu - \frac{1}{2} \mu^T \Sigma^{-1} \mu \right) \\ &= \frac{\exp \left(-\frac{1}{2} \mu^T \Sigma^{-1} \mu \right)}{(2\pi)^{n/2} \sqrt{\det \Sigma}} \exp \left(-\frac{1}{2} x^T \Sigma^{-1} x + x^T \Sigma^{-1} \mu \right)\end{aligned}\quad (5.6)$$

from this relation we get that

$\nu = \Sigma^{-1} \mu$	$\Omega = \Sigma^{-1}$
the information vector is a scaled version of the mean	the information matrix is the inverse of the covariance matrix (also called precision matrix)

An interesting thing to note is that if you picture a 3D Gaussian bell and watch it from above, you will see an ellipse.

Gaussian are extremely useful (some⁹ would say that Gaussian are cool) since are closed under sum, affine transformation, conditioning and marginalization, so to perform this operations we only need to compute the parameters of the result from the parameters of the inputs (since an *insert one of the above operations* of Gaussians is still a Gaussian).

5.2.1 Partitioned Gaussian densities

Assume the random vector, mean and covariance can be partitioned according to what is represented in table 5.1:

$$x = \begin{pmatrix} x_a \\ x_b \end{pmatrix} \quad \mu = \begin{pmatrix} \mu_a \\ \mu_b \end{pmatrix} \quad \Sigma = \begin{pmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{pmatrix}$$

Table 5.1: Partitioned

For reason of symmetry, $\Sigma_{ba} = \Sigma_{ab}^T$. Since $\Omega = \Sigma^{-1}$, it's possible to write the partitioned information matrix as well, which is also symmetric ($\Omega_{ab} = \Omega_{ba}^T$): $\Omega = \begin{pmatrix} \Omega_{aa} & \Omega_{ab} \\ \Omega_{ba} & \Omega_{bb} \end{pmatrix}$

5.2.2 Marginalization

Consider the (moment form) Gaussian distribution partitioned as seen before, the marginal density is

$$p(x_a) = \mathcal{N}(x_a; \mu_a, \Sigma_{aa}) \quad (5.7)$$

5.2.3 Conditioning

Consider the (moment form) Gaussian distribution partitioned as seen before, the conditional density is

$$p(x_a | x_b) = \mathcal{N}(x_a; \mu_{a|b}, \Sigma_{a|b}) = \frac{p(x_a, x_b)}{\int_{x_a} p(x_a, x_b) dx_a} \quad (5.8)$$

where

- $\mu_{a|b} = \mu_a + \Sigma_{aa} \Sigma_{bb}^{-1} (x_b - \mu_b) = \mu_a - \Omega_{aa}^{-1} \Omega_{ab} (x_b - \mu_b)$,
- $\Sigma_{a|b} = \Sigma_{aa} - \Sigma_{ab} \Sigma_{bb}^{-1} \Sigma_{ba} = \Omega_{aa}^{-1}$

The cross correlation Σ_{ab} indicates the Ellipse/Gaussian is crooked. If it's:

- 0, than we always have a Gaussian with the same parameters,
- not 0, we have different Gaussians with different parameters.

The larger the covariance, the more uncertain we are. Since we are integrating knowledge, there is a subtraction here.

⁹Professor Grisetti, he says this in is slides

5.2.4 Chain rule

Given the Gaussian distribution of the random variable x_a $p(x_a) = \mathcal{N}(x_a; \mu_a, \Sigma_a)$ and the conditional density $p(x_b|x_a) = \mathcal{N}(x_b; Ax_a + c, \Sigma_{b|a})$, we want to compute $p(x_a, x_b) = \mathcal{N}(x_a, x_b; \mu_{a,b}, \Sigma_{a,b})$.

The parameters are

$$\mu_{a,b} = \begin{pmatrix} \mu_a \\ \mu_b \end{pmatrix} = \begin{pmatrix} \mu_a \\ A\mu_a + c \end{pmatrix} \quad \Sigma_{a,b} = \begin{pmatrix} \Sigma_a & \Sigma_a A^T \\ A\Sigma_a & \Sigma_{b|a} + A\Sigma_a A^T \end{pmatrix}$$

We notice how the covariance is increasing, this is because when applying the chain rule we increase uncertainty. On the other hand, certainty decreases: $\Omega_{a,b} = \begin{pmatrix} A^T \Omega_{b|a} A + \Omega_a & -A^T \Sigma_{b|a} \\ -\Omega_{b|a} A^T & \Omega_{b|a} \end{pmatrix}$

5.2.5 Affine transformations

Assume we have the marginal density $p(x_a)$ and the conditional density $p(x_b|x_a)$ (affine in x_a), we want to derive the expression for the joint density $p(x_a, x_b)$, the marginal density $p(x_b)$ and the conditional density $p(x_a|x_b)$. Assume that x_a and x_b conditioned on x_a are Gaussian distributed as

$$p(x_a) = \mathcal{N}(x_a; \mu_a, \Sigma_a) \quad | \quad p(x_b|x_a) = \mathcal{N}(x_b; Mx_a + b, \Sigma_{b|a})$$

where M is a matrix and b is a constant vector, the **joint distribution** is given by

$$p(x_a, x_b) = \mathcal{N} \left(\begin{pmatrix} x_a \\ x_b \end{pmatrix}; \begin{pmatrix} \mu_a \\ M\mu_a + b \end{pmatrix}, R \right) \quad (5.9)$$

where

$$R = \begin{pmatrix} M^T \Sigma_{b|a}^{-1} M + \Sigma_a^{-1} & -M^T \Sigma_{b|a}^{-1} \\ -\Sigma_{b|a}^{-1} M & \Sigma_{b|a}^{-1} \end{pmatrix}^{-1} = \begin{pmatrix} \Sigma_a & \Sigma_a M^T \\ M \Sigma_a & \Sigma_{b|a} + M \Sigma_a M^T \end{pmatrix} \quad (5.10)$$

Assumed that $x_b = f(x_a) = Mx_a + b$, the **marginal** density is given by $p(x_b) = \mathcal{N}(x_b; \mu_b, \Sigma_b)$, where

- $\mu_b = M\mu_a + b$,
- $\Sigma_b = \Sigma_{b|a} + M\Sigma_a M^T$.

The **conditional density** is

$$p(x_a|x_b) = \mathcal{N}(x_a; \mu_{a|b}, \Sigma_{a|b}) \quad (5.11)$$

where

- $\mu_{a|b} = \Sigma_{a|b} \left(M^T \Sigma_{b|a}^{-1} (x_b - b) + \Sigma_a^{-1} \mu_a \right) = \mu_a + \Sigma_a M^T \Sigma_b^{-1} (x_b - b - M\mu_a)$
- $\Sigma_{a|b} = \left(\Sigma_a^{-1} + M^T \Sigma_{b|a}^{-1} M \right)^{-1} = \Sigma_a - \Sigma_a M^T \Sigma_b^{-1} M \Sigma_a$.

5.2.6 Taylor expansion

Assumed we have a non-linear transformation, we can approximate the function around a linearized point x_0 using Taylor expansion:

$$f(x) \simeq f(x_0) + \frac{\partial f(x)}{\partial x} (x - x_0) = Ax + f(x_0) - Ax_0 = Ax + b \quad (5.12)$$

where $A = \frac{\partial f(x)}{\partial x}$.

This reduces the transformation to an affine one; note that this is Taylor approximation, so this approximation only holds around the linearized point and the further we go, the worst the approximation it is.

6 Kalman filters

Knowledge of the robot configuration, q , is required for planning and feedback control, and this can be obtained via localization, which is a procedure for estimating the robot configuration, including in real-time. Probabilistic localization involves maintaining a probability distribution over the space of all possible hypotheses regarding the configuration, instead of maintaining a single hypothesis. Effective localization methods utilize both proprioceptive and exteroceptive sensors. If an environment map is known, actual sensor readings can be compared with those predicted using the current estimate. A Kalman Filter is one possible approach for localization.

6.1 Odometric localization

Odometric localization involves estimating the robot configuration by iterative integration of the kinematic model. This process is subject to an error that diverges over time. Wheeled Mobile Robots (WMBRs) are equipped with encoders that report the position of the wheels, not the robot's position.

Consider a unicycle under constant velocity inputs v_k, ω_k in the interval $[t_k, t_{k+1}]$, as in a digital control implementation, in each sampling interval the robot moves along an arc of a circle of radius v_k/ω_k (if $\omega_k = 0$, the robot moves on a line segment), assuming the current configuration q_k and velocities v_k, ω_k are known, the next configuration q_{k+1} can be computed via integration of the robot dynamics:

$$\dot{q} = F(q, u) \quad \text{with } t \in [t_k, t_{k+1}] \quad (6.1)$$

Integrating this gives:

$$q_{k+1} - q_k = \int_{t_k}^{t_{k+1}} F(q, u) dt \implies q_{k+1} = q_k + \int_{t_k}^{t_{k+1}} F(q, u) dt \quad (6.2)$$

Integrating this quantity is not trivial, luckily there are different possibilities for this integration, like:

- Euler integration,
- 2nd order Runge-Kutta integration,
- exact integration.

6.1.1 Euler integration

The idea behind Euler integration is to make an assumption based on a small sampling interval ($T_s = t_{k+1} - t_k$) to easily compute a nonlinear integral. The assumption for Euler integration is

$$F(q, u) \approx f(q_k, u_k) \quad (6.3)$$

and the resulting update equations are:

$$\begin{aligned} x_{k+1} &= x_k + v_k T_s \cos \theta_k \\ y_{k+1} &= y_k + v_k T_s \sin \theta_k \\ \theta_{k+1} &= \theta_k + \omega_k T_s \end{aligned} \quad (6.4)$$

In this method, x_{k+1} and y_{k+1} are approximate, while θ_{k+1} is exact.

6.1.2 2nd order Runge-Kutta integration

The assumption for Runge-Kutta integration is

$$F(q, u) \approx L \left(q_k + \frac{T_s}{2} f(q_k, u_k), u_k \right) \quad (6.5)$$

During this integration, the average orientation during $[t_k, t_{k+1}]$ is used. Consequently, x_{t+1} and y_{t+1} are still approximate but more accurate than with Euler integration. The update equations are:

$$\begin{aligned} x_{k+1} &= x_k + v_k T_s \cos \left(\theta_k + \frac{\omega_k T_s}{2} \right) \\ y_{k+1} &= y_k + v_k T_s \sin \left(\theta_k + \frac{\omega_k T_s}{2} \right) \\ \theta_{k+1} &= \theta_k + \omega_k T_s \end{aligned} \quad (6.6)$$

For $\omega_k \approx 0$, a conditional instruction may be used in the implementation.

6.1.3 Exact integration

A third option is exact integration, which gives the exact solution. The equations are:

$$\begin{aligned}x_{k+1} &= x_k + \frac{v_k}{\omega_k} T_s (\sin \theta_{k+1} - \sin \theta_k) \\y_{k+1} &= y_k - \frac{v_k}{\omega_k} T_s (\cos \theta_{k+1} - \cos \theta_k) \\\theta_{k+1} &= \theta_k + \omega_k T_s\end{aligned}\tag{6.7}$$

However, there is a possible singularity since ω is at the determinant: if $\omega_k = 0$, x_{k+1} and y_{k+1} are still defined and coincide with the solution by Euler and Runge-Kutta.

In practice, due to the non-ideality of actuation systems, the commanded inputs v_k and ω_k are not directly used; instead, the effect of the actual inputs is measured.

6.2 System model

A little recap on notation:

- u for controls,
- x for state,
- z for measurement,
- f for the transition function,
- h for the observation function.

In certain derivations, perfect knowledge of inputs, measurements, transition model, and observation model is assumed. For compactness, noise is sometimes embedded within the control input or measurements.

6.2.1 Linear discrete-time system (no noise)

A linear discrete-time system without noise can be considered. In this case, the transition and observation functions are affine transforms:

$$\begin{aligned}x_{k+1} &= A_k x_k + B_k u_k \\y_k &= C_k x_k\end{aligned}\tag{6.8}$$

Here, A_k is the state transition matrix, B_k is the input matrix, and C_k is the observation matrix. These matrices can change over time.

6.2.2 Linear discrete-time system with Gaussian noise

This type of system includes white Gaussian process noise v_k and measurement noise w_k , both assumed to have zero mean and covariance matrices V_k and W_k respectively. The system equations are:

$$\begin{aligned}x_{k+1} &= A_k x_k + B_k u_k + v_k \\y_k &= C_k x_k + w_k\end{aligned}\tag{6.9}$$

Transition and observation functions can also be described using conditional distributions, assuming Gaussian noise:

$$\begin{aligned}p(x_t | x_{t-1}, u_{t-1}) &= \mathcal{N}(x_t; A_t x_{t-1} + B_t u_{t-1} + c_t, \Sigma_x) \\p(z_t | x_t) &= \mathcal{N}(z_t; C_t x_t + d_t, \Sigma_z)\end{aligned}\tag{6.10}$$

where c_t and d_t represent potential offsets, and Σ_x, Σ_z are covariance matrices. Input error v_k and observation error w_k are assumed white Gaussian noise with zero mean.

6.2.3 Nonlinear discrete-time system with noise

A nonlinear discrete-time system with noise has the following form:

$$\begin{aligned}x_{k+1} &= f_k(x_k, u_k) + v_k \\y_k &= h_k(x_k) + w_k\end{aligned}\tag{6.11}$$

where f_k and h_k are nonlinear functions, and v_k, w_k are noise terms. f_k and h_k are assumed to be continuously differentiable for each k .

6.3 Linear systems and Gaussian noise

Gaussian distributions possess properties that are relevant to the Kalman Filter: they are closed under affine transformation, chain rule, marginalization, and conditioning.

An affine transformation $y = Ax + c$ of a Gaussian random variable $x \sim \mathcal{N}(\mu_x, \Sigma_x)$ results in a Gaussian random variable y with mean $\mu_y = A\mu_x + c$ and covariance $\Sigma_y = A\Sigma_x A^T$.

6.4 Kalman Filter

The Kalman Filter (KF) is arguably the best-studied technique for implementing Bayes filters. It serves as an optimal observer of the state for linear systems under Gaussian noise. The KF is a Bayesian Filter that operates within a closed Gaussian world, estimating the parameters (mean and covariance) of the state distribution as new measurements and controls become available. The KF represents beliefs using the moments parameterization: the belief at time t is represented by the mean μ_t and the covariance Σ_t . It is not applicable to discrete or hybrid state spaces.

For the posterior belief to remain Gaussian, in addition to the Markov assumptions of the Bayes filter, three properties must hold:

1. the state transition probability $p(x_t|u_t, x_{t-1})$ must be a linear function of its arguments with added Gaussian noise:

$$x_t = A_t x_{t-1} + B_t u_t + \epsilon_t \quad (6.12)$$

where ϵ_t is a Gaussian random vector;

2. the measurement probability $p(z_t|x_t)$ must also be linear in its arguments with added Gaussian noise. This is expressed as

$$z_t = C_t x_t + \delta_t \quad (6.13)$$

where δ_t is a multivariate Gaussian with zero mean and covariance Q_t ;

3. the initial belief $bel(x_0)$ must be normally (Gaussian) distributed.

These three assumptions ensure that the posterior belief $bel(x_t)$ is always a Gaussian for any point in time t .

The Kalman filter algorithm proceeds in two steps (like all filters): prediction and correction (or update).

6.4.1 Kalman Filter without noise

Consider a linear discrete-time system with no noise:

$$\begin{aligned} x_{k+1} &= A_k x_k + B_k u_k \\ y_k &= C_k x_k \end{aligned} \quad (6.14)$$

The goal is to build a recursive observer that computes an estimate $\hat{x}_{k+1}$ of x_{k+1} from u_k , y_{k+1} and the previous estimate $\hat{x}_k$.

The **prediction** step generates an intermediate estimate $\hat{x}_{k+1|k}$ by propagating $\hat{x}_k$ using the process dynamics. Assuming A_k , B_k and u_k are known, the prediction is:

$$\hat{x}_{k+1|k} = A_k \hat{x}_k + B_k u_k \quad (6.15)$$

This is basically Euler integration.

The **correction** step corrects the prediction based on the measured output y_{k+1} . To be consistent with the measured output, x_{k+1} must belong to the hyperplane

$$\Omega = \{x : C_{k+1}x = y_{k+1}\} \quad (6.16)$$

Thus, the correction Δx must satisfy

$$C_{k+1}(\hat{x}_{k+1|k} + \Delta x) = y_{k+1} \quad (6.17)$$

The goal is to minimize Δx , specifically to find the Δx that is closest to the prediction, which is the one orthogonal to the hyperplane. This involves solving the minimization problem

$$\min \|\Delta x\| \quad \text{subject to} \quad C_{k+1}\Delta x = y_{k+1} - C_{k+1}\hat{x}_{k+1|k} \quad (6.18)$$

We can define

$$\Delta x = C_{k+1}^\dagger (y_{k+1} - C_{k+1}\hat{x}_{k+1|k}) = C_{k+1}^\dagger \nu_{k+1} \quad (6.19)$$

Where:

- $C_{k+1}^\dagger = C_{k+1}^T (C_{k+1} C_{k+1}^T)^{-1}$ is the pseudoinverse of C_{k+1} ,
- $\nu_{k+1} = y_{k+1} - C_{k+1} \hat{x}_{k+1|k}$ is the innovation, which is the information from the sensor minus what we expect to see.

The two-step observer without noise is summarized as:

- **prediction:** $\hat{x}_{k+1|k} = A_k \hat{x}_k + B_k u_k$
- **correction:** $\hat{x}_{k+1} = \hat{x}_{k+1|k} + C_{k+1}^\dagger \nu_{k+1}$

In general, the estimate $\hat{x}_{k+1}$ will not converge to the true value because the correction is naive; estimation errors directed along Ω are not corrected.

6.4.2 Kalman Filter with process noise

Let's now include white Gaussian process noise v_k with zero mean and covariance matrix V_k :

$$\begin{aligned} x_{k+1} &= A_k x_k + B_k u_k + v_k \\ y_k &= C_k x_k \end{aligned} \quad (6.20)$$

Since this is a random process, we need to estimate the state x_{k+1} and the associated covariance P_{k+1} (using P for covariance as in some sources). The structure remains the same: prediction and correction.

Since v_k has zero mean, the state **prediction** is the same as in the no-noise case:

$$\hat{x}_{k+1|k} = A_k \hat{x}_k + B_k u_k \quad (6.21)$$

The covariance prediction depends on the linearity of the expected value and the independence of v_k on x_k :

$$P_{k+1|k} = E((x_{k+1} - \hat{x}_{k+1|k})(x_{k+1} - \hat{x}_{k+1|k})^T) = A_k P_k A_k^T + V_k \quad (6.22)$$

For the **correction** step, we choose Δx to obtain the most likely x belonging to the hyperplane ??

This means choosing the x that maximizes the Gaussian distribution

$$p(x) = \frac{1}{\sqrt{(2\pi)^n |P_{k+1|k}|}} e^{-\frac{1}{2}(x - \hat{x}_{k+1|k})^T P_{k+1|k}^{-1} (x - \hat{x}_{k+1|k})} \quad (6.23)$$

Since there is a minus sign in the exponent, $p(x)$ is maximized when the exponent is minimized. Defining the Mahalanobis distance $\Delta x^T P_{k+1|k}^{-1} \Delta x = \|\Delta x\|_M^2$, the new optimization goal is to minimize the Mahalanobis distance subject to the hyperplane constraint:

$$\min \|\Delta x\|_M \quad \text{s.t.} \quad C_{k+1} \Delta x = y_{k+1} - C_{k+1} \hat{x}_{k+1|k} \quad (6.24)$$

The **correction** is defined as

$$\begin{aligned} \Delta x &= C_{k+1,M}^\dagger (y_{k+1} - C_{k+1} \hat{x}_{k+1|k}) = C_{k+1,M}^\dagger \nu_{k+1} \\ C_{k+1,M}^\dagger &= P_{k+1|k} C_{k+1}^T (C_{k+1} P_{k+1|k} C_{k+1}^T)^{-1} \end{aligned} \quad (6.25)$$

where $C_{k+1,M}^\dagger$ is the weighted pseudoinverse of C_{k+1} . The "best" correction is the one closest to the prediction according to the current covariance estimate.

Using the definition of the covariance matrix and the state correction, the covariance correction is obtained:

$$P_{k+1} = P_{k+1|k} - C_{k+1,M}^\dagger C_{k+1} P_{k+1|k} \quad (6.26)$$

The two-step Kalman filter with process noise only is summarized as:

$$\begin{array}{l|l} \text{prediction} & \hat{x}_{k+1|k} = A_k \hat{x}_k + B_k u_k \\ \text{correction} & \hat{x}_{k+1} = \hat{x}_{k+1|k} + C_{k+1,M}^\dagger \nu_{k+1} \end{array} \quad \left| \quad \begin{array}{l} P_{k+1|k} = A_k P_k A_k^T + V_k \\ P_{k+1} = P_{k+1|k} - C_{k+1,M}^\dagger C_{k+1} P_{k+1|k} \end{array} \right.$$

6.4.3 Full Kalman filters

When using sensors for localization, we must also consider measurement noise w_k , in addition to process noise v_k . Both are assumed to be white Gaussian with zero mean and covariance matrices V_k and W_k respectively. The system equations are:

$$\begin{aligned} x_{k+1} &= A_k x_k + B_k u_k + v_k \\ y_k &= C_k x_k + w_k \end{aligned} \quad (6.27)$$

The **prediction** step remains the same as before:

$$\begin{aligned} \hat{x}_{k+1|k} &= A_k \hat{x}_k + B_k u_k \\ P_{k+1|k} &= A_k P_k A_k^T + V_k \end{aligned} \quad (6.28)$$

Due to the sensor noise, the output value y_{k+1} is no longer certain: we only know that it is drawn from a Gaussian distribution with mean value $C_{k+1} \hat{x}_{k+1|k}$ and covariance matrix W_{k+1} .

1. compute the most likely output value y_{k+1}^* given the predictions and the measured output y_{k+1} ,
2. compute the associated most likely hyperplane $\Omega^* = \{x : C_{k+1}x = y_{k+1}^*\}$,
3. compute the correction Δx .

The resulting Kalman Filter equations are:

$$\begin{array}{l|l} \text{prediction} & \hat{x}_{k+1|k} = A_k \hat{x}_k + B_k u_k \\ \text{correction} & \hat{x}_{k+1} = \hat{x}_{k+1|k} + K_{k+1} \nu_{k+1} \end{array} \quad \left| \quad \begin{array}{l} P_{k+1|k} = A_k P_k A_k^T + V_k \\ P_{k+1} = P_{k+1|k} - K_{k+1} C_{k+1} P_{k+1|k} \end{array} \right.$$

where

- $\nu_{k+1} = y_{k+1} - C_{k+1} \hat{x}_{k+1|k}$ is the innovation,
- $K_{k+1} = P_{k+1|k} C_{k+1}^T (C_{k+1} P_{k+1|k} C_{k+1}^T + W_{k+1})^{-1}$ is the Kalman gain matrix.

The Kalman gain matrix K_{k+1} weighs the accuracy of the prediction versus the accuracy of the measurement: a larger K means the measurements are more reliable, while a smaller K means the prediction is more reliable.

The KF provides an optimal estimate in the sense that $E(x_{k+1} - \hat{x}_{k+1})$ is minimized for each k . The KF is also correct because it provides the mean value and covariance of the posterior Gaussian distribution. Moreover, the KF is still the best linear filter even if the noise is non-Gaussian: if the process is observable, the estimate produced by the KF converges, meaning $E(x_{k+1} - \hat{x}_{k+1})$ is bounded for all k .

6.5 Stochastic approach

Considering a stochastic approach, we incorporate the control by computing the Gaussian distribution of the next state given the input. Starting from the prediction step, the current state is related to the previous state and control by

$$x_t = \begin{pmatrix} A_t & B_t \end{pmatrix} \begin{pmatrix} x_{t-1} \\ u_{t-1} \end{pmatrix} + c_t \quad (6.29)$$

Assuming previous state and control are distributed according to a Gaussian:

$$\begin{pmatrix} x_{t-1|t-1} \\ u_{t-1} \end{pmatrix} \sim \mathcal{N} \left(\begin{pmatrix} \mu_{t-1|t-1} \\ \mu_{u,t-1} \end{pmatrix}; \begin{pmatrix} \Sigma_{t-1|t-1} & 0 \\ 0 & \Sigma_{u,t-1} \end{pmatrix} \right) \quad (6.30)$$

Applying the rule for affine transformations of a Gaussian, where $\mathbf{y} = \mathbf{A}\mathbf{x}_a + \mathbf{c}$ transforms $\mathbf{x}_a \sim \mathcal{N}(\mu_a, \Sigma_a)$ to $\mathbf{y} \sim \mathcal{N}(\mu_b, \Sigma_b)$ with $\mu_b = A\mu_a + c$ and $\Sigma_b = A\Sigma_a A^T$. The results of the **predict step** are:

$$\begin{aligned} p(x_{t|t-1}) &= \mathcal{N}(x_{t|t-1}; \mu_{t|t-1}, \Sigma_{t|t-1}) \\ \mu_{t|t-1} &= A_t \mu_{t-1|t-1} + B_t \mu_{u,t-1} + c_t \\ \Sigma_{t|t-1} &= A_t \Sigma_{t-1|t-1} A_t^T + B_t \Sigma_{u,t-1} B_t^T \end{aligned} \quad (6.31)$$

For the update, given the:

- observation $p(z_t|x_t) = \mathcal{N}(z_t; C_t x_t + d_t, \Sigma_z)$,
- predicted state probability $p(x_t) = p(x_{t|t-1}) = \mathcal{N}(x_{t|t-1}; \mu_{t|t-1}, \Sigma_{t|t-1})$

we use the chain rule to compute the joint

$$p(z_t, x_t) = \mathcal{N} \left(\begin{pmatrix} \mu_{t|t-1} \\ \mu_z \end{pmatrix}; \begin{pmatrix} \Sigma_{t|t-1} & \Sigma_{t|t-1} C_t^T \\ C_t \Sigma_{t|t-1} & \Sigma_z + C_t \Sigma_{t|t-1} C_t^T \end{pmatrix} \right) \quad (6.32)$$

and conditioning to remove the unwanted dimensions from the Gaussian

$$\begin{aligned} p(x_{t|t}|z_t) &= \mathcal{N}(x_{t|t}; \mu_{t|t}, \Sigma_{t|t}) \\ \mu_{t|t} &= \mu_{t|t-1} + \Sigma_{t|t-1} C_t^T (\Sigma_z + C_t \Sigma_{t|t-1} C_t^T)^{-1} (z_t - \mu_z) \\ \Sigma_{t|t} &= \Sigma_{t|t-1} - \Sigma_{t|t-1} C_t^T (\Sigma_z + C_t \Sigma_{t|t-1} C_t^T)^{-1} C_t \Sigma_{t|t-1} \end{aligned} \quad (6.33)$$

This way we determined the resulting probability distribution of the state given all information up to time t

$$p(x_{t|t}|z_t) = \mathcal{N}(x_{t|t}; \mu_{t|t}, \Sigma_{t|t}) \quad (6.34)$$

6.6 To sum up

The Kalman filter algorithm can be illustrated for a simple one-dimensional localization scenario, such as a robot moving along a horizontal axis:

1. start with an initial belief (a normal distribution)

$$\begin{aligned} \mu_{t|t-1} &= A_t \mu_{t-1|t-1} + B_t \mu_{u,t-1} + c_t \\ \Sigma_{t|t-1} &= A_t \Sigma_{t-1|t-1} A_t^T + B_t \Sigma_u B_t^T \end{aligned} \quad (6.35)$$

2. incorporate a measurement (with its associated uncertainty),
3. update the belief using the KF algorithm

$$\begin{aligned} \mu_z &= C_t \mu_{t|t-1} + d_t \\ K_t &= \Sigma_{t|t-1} C_t^T (\Sigma_z + C_t \Sigma_{t|t-1} C_t^T)^{-1} \text{ is the Kalman gain} \\ \mu_{t|t} &= \mu_{t|t-1} + K_t (z_t - \mu_z) \\ \Sigma_{t|t} &= (I - K_t C_t) \Sigma_{t|t-1} \end{aligned} \quad (6.36)$$

4. propagate the belief based on motion (which introduces uncertainty),
5. incorporate a new measurement,
6. obtain the resulting belief.

The equidensity contours of a multivariate Gaussian distribution are ellipsoids. Their principal axes are directed as the eigenvectors of the covariance matrix, and their squared relative lengths are given by the corresponding eigenvalues.

6.6.1 Mathematical derivation of the KF

The derivation of the KF involves manipulating quadratic expressions, particularly when multiplying Gaussians, as their exponents are quadratic and additive. The process involves decomposing the exponent of the belief into functions that depend on different variables, allowing integration. For the prediction step, the integral over the previous state variable yields a constant that can be subsumed into the normalizer. The remaining function, which depends on the current state variable, is also quadratic, indicating the resulting belief is Gaussian. The mean and covariance of this Gaussian correspond to the prediction step formulas in the KF algorithm.

For the measurement update step, the derivation incorporates the measurement probability (also Gaussian) with the predicted belief (Gaussian). The product of two Gaussians is proportional to a Gaussian, and its exponent is quadratic. Calculating the first and second derivatives of this exponent with respect to the state allows determining the mean and covariance of the updated belief.

The Kalman gain matrix emerges during the calculation of the updated mean. Using the inversion lemma can simplify the expression for the updated covariance, avoiding the need to invert the state covariance matrix, which is essential for high-dimensional state spaces.

6.7 Non-Linear systems and Gaussian noise

When dealing with nonlinear discrete-time systems with noise, described by

$$\begin{aligned} x_{k+1} &= f_k(x_k, u_k) + v_k \\ y_k &= h_k(x_k) + w_k \end{aligned} \quad (6.37)$$

the assumption of linearity in the state transition and measurement functions is violated. Unfortunately, with arbitrary nonlinear functions f_k and h_k , the belief is generally no longer a Gaussian. Performing the belief update exactly is usually impossible, and the Bayes filter does not possess a closed-form solution. A nonlinear transformation can distort the density of a Gaussian random variable, destroying its Gaussian shape.

6.8 Extended Kalman filter

The Extended Kalman Filter (EKF) extends the Kalman Filter to nonlinear problems (eq. ??). It relaxes the linearity assumption, using nonlinear functions f_k and h_k for the state transition and measurement probabilities.

The EKF calculates a Gaussian **approximation** to the true belief: its goal is shifted from computing the exact posterior to efficiently estimating its mean and covariance, which are still approximate.

The key idea underlying the EKF approximation is linearization via Taylor expansion: the EKF approximates the nonlinear functions f_k and h_k by linear functions that are tangent to the original functions around the current estimate (typically the mean). It linearizes around

- $\mu_{t-1|t-1}$, for the state transition f_k ;
- $\mu_{t|t-1}$, for the measurement function h_k .

Jacobian matrices are used in the linearization. G_t (or A_t) is the Jacobian of f_k with respect to the state x_k , evaluated at the current mean estimate $\mu_{t-1|t-1}$. H_t (or C_t) is the Jacobian of h_k with respect to the state x_k , evaluated at the predicted mean estimate $\mu_{t|t-1}$.

The resulting Extended Kalman Filter (EKF) algorithm is similar in structure to the Kalman filter, but the linear predictions in KF are replaced by their nonlinear generalizations in EKF, and Jacobians are used instead of the linear system matrices.

The EKF algorithm is summarized as follows:

- **predict:** incorporate new control input:

$$\begin{aligned} \mu_{t|t-1} &= f(\mu_{t-1|t-1}, \mu_{u,t-1}) \\ A_t &= \frac{\partial f(x, u)}{\partial x} \Big|_{x=\mu_{t-1|t-1}} \\ B_t &= \frac{\partial f(x, u)}{\partial u} \Big|_{u=\mu_{u,t-1}} \\ \Sigma_{t|t-1} &= A_t \Sigma_{t-1|t-1} A_t^T + B_t \Sigma_u B_t^T \end{aligned} \quad (6.38)$$

($\Sigma_{t|t-1}$ is the Jacobian w.r.t. input in the covariance prediction, Σ_u is input noise covariance)

- **update:** incorporate new measurement z_t .

$$\begin{aligned} \mu_z &= h(\mu_{t|t-1}) \\ C_t &= \frac{\partial h(x)}{\partial x} \Big|_{x=\mu_{t|t-1}} \\ K_t &= \Sigma_{t|t-1} C_t^T (\Sigma_z + C_t \Sigma_{t|t-1} C_t^T)^{-1} \\ \mu_{t|t} &= \mu_{t|t-1} + K_t (z_t - \mu_z) \\ \Sigma_{t|t} &= (I - K_t C_t) \Sigma_{t|t-1} \end{aligned} \quad (6.39)$$

where C_t is the Jacobian of the measurement function.

The Kalman gain matrix K_t is calculated using the weighted pseudoinverse involving the predicted covariance and measurement covariance.

6.8.1 Mathematical derivation of the EKF

The mathematical derivation of the EKF parallels that of the Kalman filter but applies the steps to the linearized system equations. It involves decomposing the exponent of the belief and integrating, similar to the KF derivation. The measurement update is derived analogously to the KF update, using the linearized measurement function.

6.8.2 Practical considerations

The EKF has become one of the most popular tools for state estimation in robotics due to its simplicity and computational efficiency. Each update typically requires time complexity on the order of $O(k^{2.4} + n^2)$, where k is the dimension of the measurement vector and n is the dimension of the state vector. This efficiency stems from representing the belief as a multivariate Gaussian distribution, which is unimodal. A Gaussian can be seen as a single estimate with an associated uncertainty ellipse. In many practical problems, Gaussians are robust estimators.

However, the EKF's reliance on a unimodal Gaussian representation is a limitation; it cannot effectively represent multimodal beliefs, where the robot might have multiple distinct hypotheses about its location. In such cases, the arithmetic mean of hypotheses is not necessarily a likely state. An extension called the Multi-Hypothesis Extended Kalman Filter (MHEKF) addresses this by representing posteriors using mixtures (sums) of Gaussians.

The quality of the EKF approximation is affected by the uncertainty of the Gaussian belief and the degree of nonlinearity in the functions. If the nonlinear functions are approximately linear around the mean of the estimate, the EKF approximation tends to be good. However, the wider the Gaussian belief (i.e., the less certain the robot), the more it is distorted by nonlinearities, leading to larger approximation errors. Similarly, the more locally nonlinear the function is around the mean, the larger the error. Therefore, in practice, it is important to keep the uncertainty of the state estimate small when applying EKFs.

Compared to the standard KF for linear systems, the EKF for nonlinear systems loses the property of optimality.

6.9 Summary

The Kalman Filter is an optimal observer for the state of linear systems subject to Gaussian noise. It functions as a Bayesian Filter that maintains the belief as a Gaussian distribution by updating its mean and covariance based on controls and measurements. For nonlinear systems, the Extended Kalman Filter (EKF) can be used, which applies KF equations to a linearized approximation of the system dynamics and measurement functions. However, the EKF loses the optimality guarantee of the standard KF. Both KF and EKF are widely used in practice.

7 EKF SLAM applications

7.1 SLAM definition

The Simultaneous Localization and Mapping problem, commonly abbreviated as SLAM, or Concurrent Mapping and Localization (CML), arises when a robot operates without a pre-existing map of its environment or knowledge of its own pose; instead, the robot is provided solely with measurements ($z_{1:t}$) and controls ($u_{1:t}$).

In SLAM, the robot is tasked with acquiring a map of its environment while simultaneously localizing itself relative to this developing map and this tasks must be estimated concurrently.

From a probabilistic perspective, the SLAM problem has two main forms:

1. online,
2. full.

7.1.1 Online SLAM problem

The online SLAM problem involves estimating the posterior distribution over the momentary pose (x_t) along with the map (m):

$$p(x_t, m | z_{1:t}, u_{1:t}) \quad (7.1)$$

This is referred to as the online problem because it focuses on estimating variables that persist at time t . Many algorithms for online SLAM are incremental, discarding past measurements and controls after processing.

7.1.2 Full SLAM problem

The full SLAM problem aims to calculate a posterior over the robot's entire path ($x_{1:t}$) up to time t , in addition to the map:

$$p(x_{1:t}, m | z_{1:t}, u_{1:t}) \quad (7.2)$$

This differs from online SLAM, which only estimates the current pose x_t . The online SLAM posterior can be obtained by integrating out past poses from the full SLAM posterior:

$$p(x_t, m | z_{1:t}, u_{1:t}) = \int \cdots \int p(x_{1:t}, m | z_{1:t}, u_{1:t}) dx_1 dx_2 \cdots dx_{t-1} \quad (7.3)$$

Integrating out past poses causes changes in the dependency structures in SLAM. The graphical model for full SLAM shows the dependencies across the entire path and the map.

7.1.3 Continuous and discrete components

SLAM problems contain both continuous and discrete estimation components:

- the continuous aspect relates to the location of objects in the map and the robot's pose variables,
- the discrete aspect pertains to correspondence: when an object is detected, the algorithm must determine if it's a previously seen object or a new one (this reasoning is typically discrete).

Explicit correspondence variables (c_t for current, $c_{1:t}$ for full path) can be included in the posteriors.

7.1.4 Challenges

Estimating the full posterior for either online or full SLAM is usually infeasible in practice due to the high dimensionality of the continuous parameter space (map features, pose) and the large number of discrete correspondence variables; moreover, the number of possible correspondence assignments grows exponentially with time. Practical SLAM algorithms rely on **approximations** to handle the correspondence problem, EKF SLAM is one such approximation method.

7.2 Application

The Extended Kalman Filter (EKF) approach was historically the earliest and arguably most influential SLAM algorithm; it applies the EKF to the online SLAM problem, typically using maximum likelihood data association.

The EKF SLAM approach involves several key aspects:

- scenario description;
- controls and observations definitions;
- handling non-linear systems and Gaussian noise assumptions;
- the EKF SLAM algorithm details.

7.2.1 Scenario

Consider a robot moving on a 2D plane. The robot is controlled by translational and rotational velocities, it is equipped with a sensor that can detect a set of uniquely distinguishable landmarks, measuring their relative position (range and bearing). Crucially, the location of these landmarks in the world is **unknown**.

The goal is to develop a KF-based algorithm that simultaneously tracks the robot's position (localization) and the position of observed landmarks (mapping). The inputs to the algorithm are velocity measurements (controls) and landmark measurements (observations). There is no prior knowledge of the map.

7.2.2 EKF

The EKF framework for SLAM follows the standard predict-correct cycle, but a new step is added:

1. **predict**: incorporate the new control input to predict the state and its uncertainty (eq. ??).
2. **correct**: incorporate the new measurement to update the state estimate and reduce uncertainty (innovation $z_t - h(\mu_{t|t-1})$) (eq. ??),
3. **SLAM**: add new landmarks to the state.

7.2.3 Domains: state space

In EKF SLAM, the state vector is extended to include the robot's pose and the positions of all observed landmarks: if the robot pose $X_t^{[r]}$ is 3D and there are N landmarks, each with 2D position $x_t^{[N]}$, the full state vector can be defined as:

$$\begin{aligned} X_t^{[r]} = [R_t | t_t] \in SE(2) &\implies x_t^{[r]} = (x_t \quad y_t \quad \theta_t)^T \in \mathcal{R}^3 \\ x_t^{[N]} = (x_t^{[N]} \quad y_t^{[N]})^T &\in \mathcal{R}^2 \implies x_t = \begin{pmatrix} x_t^{[r]} \\ x_t^{[N]} \end{pmatrix} \in \mathcal{R}^{(3+2N)} \end{aligned} \quad (7.4)$$

This matches the structure where the robot pose variables come first, followed by the coordinates of each landmark.

7.2.4 Domains: control and measurements

The **controls**, similar to EKF Localization, typically represent the robot's velocity inputs¹⁰:

$$u_t = \begin{pmatrix} u_t^{[1]} \\ u_t^{[2]} \end{pmatrix} \in \mathcal{R}^2 \quad (7.5)$$

Measurements correspond to observations of landmarks by the robot's sensor: assuming a 2D landmark sensor, each measurement can be the relative position (e.g., range and bearing) of a landmark with respect to the robot. If M landmarks are observed at time t , the measurement vector z_t consists of M blocks. For the m -th measurement block, corresponding to the $j(m)$ -th landmark in the state, the measurement $z_t^{[m]}$ might be its position relative to the robot:

$$z_t^{[m]} = \begin{pmatrix} x_t^{[m]} \\ y_t^{[m]} \end{pmatrix} \in \mathcal{R}^2, \quad m = 1, \dots, M \quad (7.6)$$

Landmark measurements can also include range, bearing, and potentially signature $(r_t^i, \phi_t^i, s_t^i)^T$.

7.2.5 Transition function

The transition function describes how the state vector changes over time based on the control input. For the augmented state vector, only the robot's pose changes according to the motion model; the landmark positions

¹⁰For a differential drive or unicycle-like robot, this might be translational and rotational velocity

remain constant as they are static entities in the world:

$$x_t = f(x_{t-1}, u_{t-1}) = \begin{pmatrix} x_{t-1} + u_{t-1}^{[1]} \cos(\theta_{t-1}) \\ y_{t-1} + u_{t-1}^{[1]} \sin(\theta_{t-1}) \\ \theta_{t-1} + u_{t-1}^{[2]} \\ x_{t-1}^{[1]} \\ x_{t-1}^{[2]} \\ \vdots \\ x_{t-1}^{[N]} \end{pmatrix} \quad (7.7)$$

This assumes a simple kinematic model for the robot's pose update. More generally, using a velocity model:

$$y_t = y_{t-1} + F_x \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \theta - \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix} \quad (7.8)$$

where y_t is the combined state vector, and F_x is a matrix that maps the robot's 3D pose change into the $(3 + 2N)$ -dimensional state vector space.

7.2.6 Measurement function

The measurement function $h(x_t)$ predicts the sensor measurement given the robot's pose and the landmark locations from the state vector. For the N -th potential measurement (from the N -th landmark), it calculates the predicted relative position of the landmark with respect to the robot pose $x_t^{[r]}$ at time t :

$$\begin{aligned} z_t^{[n]} &= h^{[n]}(x_t) = h(x_t^{[r]}, x_t^{[n]}) = R_t^T (x_t^{[n]} - t_t) \\ &= \begin{pmatrix} \cos \theta_t (x_t^{[n]} - x_t) + \sin \theta_t (y_t^{[n]} - y_t) \\ -\sin \theta_t (x_t^{[n]} - x_t) + \cos \theta_t (y_t^{[n]} - y_t) \end{pmatrix} \end{aligned} \quad (7.9)$$

where

- $x_t^{[r]} = (x_t, y_t, \theta_t)^T$ is the robot pose,
- $x_t^{[N]} = (x_t^{[N]}, y_t^{[N]})^T$ is the landmark position,
- $t_t = (x_t, y_t)^T$ is the robot's translational position,
- R_t is the rotation matrix corresponding to θ_t .

For a range and bearing sensor, the measurement function for the j -th landmark would be:

$$h(y_t, j) = \begin{pmatrix} \sqrt{(m_{j,x} - x)^2 + (m_{j,y} - y)^2} \\ \text{atan2}(m_{j,y} - y, m_{j,x} - x) - \theta \\ m_{j,s} \end{pmatrix} \quad (7.10)$$

Here (x, y, θ) is the robot pose from y_t , $(m_{j,x}, m_{j,y}, m_{j,s})$ are the landmark properties from y_t , where j is the index of the landmark.

7.2.7 Control noise

The control inputs are assumed to be affected by zero-mean Gaussian noise. This noise can have two components:

- a constant noise σ_u ,
- velocity-dependent terms whose standard deviation grows with speed:
 - translational noise standard deviation $\sigma_T = u_t^{[1]}$,
 - rotational noise standard deviation $\sigma_R = u_t^2$.

The noise is typically represented by a covariance matrix Σ_u .

$$n_{u,t} \sim \mathcal{N} \left(n_{u,t}; 0, \begin{pmatrix} \sigma_u^2 + \sigma_T^2 & 0 \\ 0 & \sigma_u^2 + \sigma_R^2 \end{pmatrix} \right) = \mathcal{N}(n_{u,t}; 0, \Sigma_u) \quad (7.11)$$

7.2.8 Measurement noise

For each landmark measurement, a (zero mean and constant) Gaussian noise is considered. For a 2D measurement (e.g., x and y relative position, or range and bearing), the noise covariance matrix Σ_z is used.

$$n_z \sim \mathcal{N}\left(n_z; 0, \begin{pmatrix} \sigma_z^2 & 0 \\ 0 & \sigma_z^2 \end{pmatrix}\right) \quad (7.12)$$

An alternative notation of the measurement noise covariance matrix or range, bearing, and signature as Q_t is:

$$Q_t = \begin{pmatrix} \sigma_r^2 & 0 & 0 \\ 0 & \sigma_\phi^2 & 0 \\ 0 & 0 & \sigma_s^2 \end{pmatrix} \quad (7.13)$$

7.2.9 Jacobian 1: state

For the prediction step of the EKF, the transition function is linearized around the previous state estimate ($\mu_{t-1|t-1}$). The Jacobian with respect to the state vector x_t , denoted A_t , is calculated.

Since only the robot's pose changes, the derivative with respect to landmark states is an identity block for that landmark and zero for others.

The Jacobian has a block structure:

$$A_t = \frac{\partial f(\cdot)}{\partial x} \Big|_{x=\mu_{t-1|t-1}} = \begin{pmatrix} \frac{\partial f(\cdot)}{\partial x^{[r]}} & \frac{\partial f(\cdot)}{\partial x^{[1]}} & \frac{\partial f(\cdot)}{\partial x^{[2]}} & \cdots & \frac{\partial f(\cdot)}{\partial x^{[N]}} \end{pmatrix} \quad (7.14)$$

The derivative of the transition model over the robot state is:

$$\frac{\partial f(\cdot)}{\partial x^{[r]}} = \begin{pmatrix} 1 & 0 & -u_{t-1}^{[1]} \sin(\theta_{t-1}) \\ 0 & 1 & u_{t-1}^{[1]} \cos(\theta_{t-1}) \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ \vdots & \vdots & \vdots \\ 0 & 0 & 0 \end{pmatrix} \quad (7.15)$$

The derivative of the transition model over a landmark state (e.g., the 2nd landmark) is an identity matrix in its block position:

$$\frac{\partial f(\cdot)}{\partial x^{[2]}} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ \vdots & \vdots \\ 1 & 0 \\ 0 & 1 \\ \vdots & \vdots \\ 0 & 0 \end{pmatrix} \quad (7.16)$$

A has dimension $N \times N$, where N is the dimension of the state vector.

7.2.10 Jacobian 2: controls

The Jacobian of the transition function with respect to the control input u_t , denoted B_t , is also calculated. For a 2-dimensional control vector and an N -dimensional state, this is an $M \times N$ Jacobian (where $M = 2$ is the control dimension, N is the state dimension):

$$B_t = \frac{\partial f(x, u)}{\partial u} \Big|_{u=u_{t-1}} = \begin{pmatrix} \cos(\theta_{t-1}) & 0 \\ \sin(\theta_{t-1}) & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \\ \vdots & \vdots \\ 0 & 0 \end{pmatrix} \quad (7.17)$$

7.2.11 Jacobian 3: measurements

For the correction step, the measurement function $h(x_t)$ is linearized around the predicted state estimate $(\mu_{t|t-1})$. The Jacobian with respect to the full state vector x_t needs to be calculated for each potential measurement (each landmark).

The Jacobian for the n -th measurement, denoted $C_t^{[n]}$, has a block structure because the predicted measurement of landmark n depends only on the robot's pose and the position of landmark n :

$$C_t^{[n]} = \frac{\partial h^{[n]}(\cdot)}{\partial x} \Big|_{x=\mu_{t|t-1}} = \begin{pmatrix} \frac{\partial h^{[n]}}{\partial x^{[r]}} & 0 & \dots & \frac{\partial h^{[n]}}{\partial x^{[n]}} & \dots & 0 \end{pmatrix} \quad (7.18)$$

Here, $\frac{\partial h^{[n]}}{\partial x^{[r]}}$ is the derivative with respect to the robot pose block, and $\frac{\partial h^{[n]}}{\partial x^{[n]}}$ is the derivative with respect to the n -th landmark's position block.

All other blocks are zero because the measurement only depends on the robot and the observed landmark.

Calculating the derivatives for the non-zero blocks, remembering that the measurement function is $h^{[n]}(x_t) = R_t^T(x_t^{[n]} - t_t)$:

$$\begin{array}{ll} \text{w.r.t. the robot pose block:} & \text{w.r.t. the landmark block}^{11} \\ \frac{\partial h^{[n]}}{\partial x^{[r]}} = \begin{pmatrix} -R_t^T & \frac{\partial R_t^T}{\partial \theta_t}(x_t^{[n]} - t_t) \end{pmatrix} & \frac{\partial h^{[n]}}{\partial x^{[n]}} = R_t^T \end{array} \quad (7.19)$$

The specific Jacobian for the range, bearing, signature measurement function is:

$$h_{i \ t} = \frac{1}{q} \begin{pmatrix} -\sqrt{q}\delta_x & -\sqrt{q}\delta_y & 0 & \sqrt{q}\delta_x & \sqrt{q}\delta_y & 0 \\ \delta_y & -\delta_x & -q & -\delta_y & \delta_x & 0 \\ 0 & 0 & 0 & 0 & 0 & q \end{pmatrix} \quad (7.20)$$

where $\delta = (\delta_x, \delta_y)^T = (\bar{\mu}_{j,x} - \bar{\mu}_{t,x}, \bar{\mu}_{j,y} - \bar{\mu}_{t,y})^T$ and $q = \delta^T \delta$. This Jacobian is applied to the matrix $F_{x,j}$ which maps the 6-dimensional space (robot pose + landmark pose) to the full state space.

7.2.12 Data association

A significant challenge in SLAM, especially when landmark identities are not known (unknown correspondence), is data association. The observed measurements at time t (z_t) originate from a subset of the landmarks currently included in the state, but the order of measurements may not match the order of landmarks in the state.

A data association function $j(m)$ is needed to identify which landmark in the state corresponds to the m -th measurement block. For now, we assume this assignment is known or determined.

When correspondence is unknown, the problem is to determine the association map: a common approach is to associate each observation to the landmark that minimizes the magnitude of the innovation.

The innovation (n_{ij}) is the difference between the actual measurement ($y_{i,k+1}$) and the expected measurement if the observation refers to the j -th landmark ($h(\hat{q}_{k+1|k}, \text{landmark } j)$). The smaller the innovation, the more likely the correspondence.

This innovation magnitude must be weighted by the measurement uncertainty, encoded in the matrix Ψ_k in the EKF.

The EKF SLAM algorithm with unknown correspondences uses an incremental maximum likelihood (ML) estimator for correspondence: it calculates a likelihood score (based on Mahalanobis distance) for associating a measurement with each existing landmark in the map, and also considers the possibility of a new landmark. The Mahalanobis distance for measurement z_t^i and landmark k is

$$\pi_k = (z_t^i - \hat{z}_t^k)^T \Psi_k^{-1} (z_t^i - \hat{z}_t^k) \quad (7.21)$$

where Ψ_k is the innovation covariance. The correspondence is selected by minimizing this distance.

7.2.13 Populating the map

As the robot moves and observes new landmarks, these need to be added to the map, which means extending the state vector. This process is iterative:

1. initially, the state only contains the robot's pose. Map-related information (like ID-to-state and state-to-ID maps) is initialized to indicate no landmarks are present;
2. when the first landmark (say, with ID 7) is observed at time $t + 1$, its estimated position is added to the state vector, which is then augmented. The ID-to-state map is updated to show that landmark 7 corresponds to the first landmark entry in the state, and the state-to-ID map shows that the first state entry corresponds to landmark 7;

3. when a second new landmark (say, with ID 9) is observed at time $t + 2$, its estimated position is added, further augmenting the state vector. The maps are updated accordingly;
4. this process continues as more new landmarks are observed, populating the map.

When a measurement is not associated with any existing landmark via the ML correspondence check (its Mahalanobis distance is above a threshold compared to the "new landmark" score), a new landmark is hypothesized and added to the map. The location of a new landmark is initialized based on the robot's current pose estimate and the measurement (range and bearing).

This initialization is important for linearization in EKF.

7.2.14 Updating the map

When new measurements arrive, the sensed landmarks are partitioned into two classes:

1. already known landmarks: these are landmarks already present in the state vector. Their measurements are used to perform an EKF correction step, updating the robot's pose and the locations of the known landmarks in the state;
2. new landmarks: these are landmarks not yet in the state vector. Based on data association, if a measurement is determined to come from a new landmark, this landmark is added to the state *after* the EKF correction using known landmarks is performed.

Practical EKF SLAM implementations often include map management techniques to deal with spurious measurements (outliers) and manage the growing map size. These include:

- maintaining a provisional landmark list for newly observed features before adding them permanently to the map. Provisional landmarks don't affect the robot pose estimate until they are consistently observed;
- maintaining a landmark existence probability or counter, incremented upon observation and decremented when a landmark is expected but not seen, removing landmarks below a threshold.

These techniques lead to leaner maps and improve robustness against noise. Initializing new landmark covariance with very large values can cause numerical instability; an explicit initialization step based on the measurement and robot pose estimate is preferred.

7.3 Limitations of EKF SLAM

EKF SLAM is subject to several limitations:

- Gaussian noise assumption: EKF's assume Gaussian noise for motion and perception, which may not always hold.
- Unimodal distribution: KF/EKF assumes the state probability distribution is unimodal (a single peak). If the initial position is unknown or in the presence of aliasing (where distinct locations appear similar to the sensor), the distribution can become multimodal (multiple peaks), which EKF's cannot handle well (the "kidnapped robot problem").
- Linearization errors: EKF's rely on linearizing nonlinear motion and measurement models around the current estimate. Significant uncertainty or highly nonlinear functions can lead to intolerable errors.
- Data association brittleness: the incremental maximum likelihood data association, while efficient, makes EKF SLAM brittle to landmark confusion. It assumes the ML association is always correct and cannot revise past decisions. Distinctive landmarks or spatial separation help mitigate this.
- Quadratic complexity: the update time for EKF SLAM is quadratic in the number of landmarks ($O(N^2)$). This limits the size of the map that can be managed to typically less than 1,000 features. Larger maps (10^6 features or more) are computationally infeasible.
- Limited information usage: EKF SLAM often relies on extracting specific landmarks, discarding much of the raw sensor data, which leads to information loss compared to methods using the full sensor stream.
- Difficult data association in Sparse Maps: the quadratic complexity necessitates sparse maps, but sparse maps make data association harder due to the increased chance of confusing features. This is the fundamental dilemma of EKF SLAM.

- Inapplicable to full SLAM: the basic EKF SLAM algorithm described is for online SLAM. Applying it directly to full SLAM would require the state vector and covariance to grow indefinitely with time, making updates computationally prohibitive.

Despite these limitations, EKF SLAM has been applied successfully in various robotic scenarios (airborne, underwater, indoor) when landmarks are sufficiently distinct and the map size is manageable.

7.4 g2o wrapper

Datasets for testing SLAM algorithms, such as those acquired from simulations (like V-REP) or real robots, often contain control inputs (transitions) and sensor observations (measurements) related to poses and landmarks. Formats like g2o can store this data, where transitions link poses, and observations link poses to landmarks. Load functions can parse this data into structured arrays for use in an EKF SLAM algorithm. Unlike localization datasets where landmark locations are known, SLAM datasets require estimating these locations.

8 Data association

Assume we don't know which observation was generated by which landmark (so the landmarks are not uniquely identifiable), this is intrinsically a Markovian problem. A mixture of Gaussian approximate a multi-data process.

In this case, we can choose the best possible association, this can however lessen robustness.

8.1 Finding the best association

Data association determines which state variable (if any) is responsible of a measurement, based on the current estimate of the state and on the history of measurements.

Assumed we have n landmarks and m measurements, with $n > m$, the number of possible associations is

$$\frac{n!}{m!(n-m)} = \binom{n}{m} \quad (8.1)$$

For simple cases, we try all the combinations and find the best one, but in SLAM is more complex: when we evolve the system, the distributions grow exponentially.

The best association j^* is the one that maximises the joint likelihood of the measurements:

$$j^* = \arg \max_j p\left(\hat{z}^{[j(1)]} = z^{[1]}, \dots, \hat{z}^{[j(M)]} = z^{[M]}\right) \quad (8.2)$$

where:

- $z^{[i]}$ is the element in the cell i , which tells us which landmark is responsible for observation i ,
- $p\left(\hat{z}^{[j(1)]} = z^{[1]}, \dots, \hat{z}^{[j(M)]} = z^{[M]}\right)$ is the probability of assignment j ,
- $j(i) \in [0, 1, \dots, N]$. $j(3) = 5$ means measurement 3 originates from landmark 5. $j(n) = 0$ means no correspondence,
- $X = x^r, x^{[1]}, \dots, x^{[N]}$ is the state (robot position, sensor position, landmarks),
- $z = z^{[1]}, \dots, z^{[M]}$ are the measurements $(1, \dots, M)$ ¹²
- $\hat{z} = \hat{z}^{[1]}, \dots, \hat{z}^{[N]}$ are the predictions $(1, \dots, N)$ ¹³.

8.1.1 Evaluation an association

Assumed the state is known, we can write down the steps to compute the likelihood of an association j :

$$\begin{aligned} p\left(\hat{z}^{[j(1)]} = z^{[1]}, \dots, \hat{z}^{[j(M)]} = z^{[M]}\right) &= \int_X p\left(\hat{z}^{[j(1)]} = z^{[1]}, \dots, \hat{z}^{[j(M)]} = z^{[M]} | X\right) p(X) dX \\ &= \int_{X \in \Omega} \prod_{m=1}^M p\left(\hat{z}^{[j(m)]} = z^{[m]} | x^{[j(m)]}, x^r\right) p(x^{[j(m)]}, x^r) dX \end{aligned} \quad (8.3)$$

We are decorrelating one measurement from the next (x^r is a measurement of independence).

We need to repeat this for each association, which is quite a cumbersome and heavy process, we can simplify it.

8.2 Joint prediction

First thing, we produce the joint prediction on all the possible landmarks; let's reason with single data association: we notice that the distribution of the prediction is not dependent on the specific assignment

$$\begin{aligned} p(\hat{z}) &= \int_X p(\hat{z} | X) p(X) dX && \text{chain rule and marginalization} \\ &= \int_{X \in \Omega} \prod_{n=1}^N p(\hat{z}^{[n]} | X^{[n]}, x^r) p(X) dX && \text{measurement independence} \end{aligned} \quad (8.4)$$

$p(\hat{z})$ has N blocks, as many as the landmark. We can compute this distribution once, and then evaluate it for each assignment.

¹²We can have more landmark then measurement just like more measurement than landmark

¹³ $\#states = \#predictions$

8.2.1 Independence loss

Marginalizing out the states renders the landmark observations not independent, thus we cannot “estimate” the likelihood of a single landmark assignment as the product of the single likelihoods:

$$p(\hat{z}^{[j]} = z) \neq \prod_{m=1}^M p(\hat{z}^{[j(m)]} = z^{[m]}) \quad (8.5)$$

After having defined a mapping from landmarks to measurement, can we define the inverse mapping from measurement to landmarks?

$$j(m) = n \quad j^{-1}(n) = m$$

In general, no. Especially if some landmarks do not appear in the observations, since in this case $j(n)$ will return an invalid value.

8.2.2 Using the joint prediction

We can use the prediction of all landmarks to determine how likely one specific assignment is; to do so, we can reshuffle the measurement vector by reordering the measures according to the inverse assignment j^{-1}

$$z^{[j^{-1}]} = \begin{pmatrix} z^{[j^{-1}(1)]} \\ z^{[j^{-1}(2)]} \\ \vdots \\ z^{[j^{-1}(N)]} \end{pmatrix} \quad z^{[j]} = \begin{pmatrix} z^{[j^{-1}(1)]} \\ \vdots \\ z^{[j^{-1}(N)]} \end{pmatrix} \quad (8.6)$$

Notice how, since the mapping is not invertible, we will have “holes” for landmarks not seen or unmatched measurements.

8.3 Gaussian case

In the Gaussian case we assume the observation model is distributed according to a Gaussian distribution, we can apply marginalization and chain rule in one go:

$$\begin{aligned} p(z|x) &= \mathcal{N}(z; Cx + d, \Sigma_{z|x}) && \text{conditional over all landmarks} \\ p(x) &= \mathcal{N}(x; \mu_x, \Sigma_x) && \text{prior on pose and landmarks} \end{aligned} \quad (8.7)$$

Where

$$\begin{aligned} \mu &= \begin{pmatrix} \mu^r \\ \mu^{[1]} \\ \mu^{[2]} \\ \vdots \\ \mu^{[N]} \end{pmatrix} && \Sigma = \begin{pmatrix} \Sigma^{[rr]} & \Sigma^{[r1]} & \Sigma^{[r2]} & \dots \Sigma^{[rN]} \\ \Sigma^{[1r]} & \Sigma^{[11]} & \Sigma^{[12]} & \dots \Sigma^{[1N]} \\ \Sigma^{[2r]} & \Sigma^{[21]} & \Sigma^{[22]} & \dots \Sigma^{[2N]} \\ \vdots & & & \vdots \\ \Sigma^{[Nr]} & \Sigma^{[N1]} & \Sigma^{[N2]} & \dots \Sigma^{[NN]} \end{pmatrix} \\ \mu_z &= C\mu_x + d && \Sigma_z = C\Sigma_x C^T + \Sigma_{z|x} = \begin{pmatrix} \Sigma_z^{[11]} & \Sigma_z^{[12]} & \dots \Sigma_z^{[1N]} \\ \Sigma_z^{[21]} & \Sigma_z^{[22]} & \dots \Sigma_z^{[2N]} \\ \vdots & & \vdots \\ \Sigma_z^{[N1]} & \Sigma_z^{[N2]} & \dots \Sigma_z^{[NN]} \end{pmatrix} \end{aligned}$$

the chain rule will have dimension $\mu_x + \mu_z$

Joint covariance of the measurements

We want to eliminate Σ_x via marginalization. Note that we are projecting every single individual landmark: no data association so far.

Let's now evaluate the association; to do so, we can reshuffle the measurement vector by reordering the measures according to the inverse assignment j^{-1}

$$z^{[j]} = \begin{pmatrix} z^{[j^{-1}(1)]} \\ z^{[j^{-1}(3)]} \\ \vdots \\ z^{[j^{-1}(n)]} \\ \vdots \\ z^{[j^{-1}(N)]} \end{pmatrix} \quad (8.8)$$

Notice how, since the mapping is not invertible, we will have “holes” for landmarks not seen or unmatched measurements. We can suppress all holes in measurements, predictions and covariance matrix using marginalization, just like we do in Gaussian.

Before suppression

After suppression

$$\begin{aligned}
z^{[j]} &= \begin{pmatrix} z^{[j^{-1}(1)]} \\ z^{[j^{-1}(3)]} \\ \vdots \\ z^{[j^{-1}(n)]} \\ \vdots \\ z^{[j^{-1}(N)]} \end{pmatrix} & \tilde{z}^{[j]} &= \begin{pmatrix} z^{[j^{-1}(1)]} \\ z^{[j^{-1}(3)]} \\ \vdots \\ z^{[j^{-1}(n)]} \\ \vdots \\ z^{[j^{-1}(N)]} \end{pmatrix} \\
\mu_z &= \begin{pmatrix} \mu_z^{[1]} \\ \vdots \\ \mu_z^{[N]} \end{pmatrix} & \tilde{\mu}_z &= \begin{pmatrix} \mu_z^{[1]} \\ \vdots \\ \mu_z^{[N]} \end{pmatrix} \\
\Sigma_z &= \begin{pmatrix} \Sigma_z^{[11]} & \dots & \Sigma_z^{[1N]} \\ \vdots & & \vdots \\ \Sigma_z^{[N1]} & \dots & \Sigma_z^{[NN]} \end{pmatrix} & \tilde{\Sigma}_z &= \begin{pmatrix} \Sigma_z^{[11]} & \dots & \Sigma_z^{[1N]} \\ \vdots & & \vdots \\ \Sigma_z^{[N1]} & \dots & \Sigma_z^{[NN]} \end{pmatrix}
\end{aligned}$$

We can now evaluate the Gaussian:

$$p\left(\hat{z}^{[j(1)]} = z^{[1]}, \dots, \hat{z}^{[j(M)]} = z^{[M]}\right) \propto \frac{1}{\dots} \exp\left(-(\tilde{z}^{[j]} - \tilde{\mu}_z)^T \tilde{\Sigma}_z^{-1} (\tilde{z}^{[j]} - \tilde{\mu}_z)\right) \quad (8.9)$$

presumably proportional to how likely the association is.

Given some associations, we can have a certain error that grows with the associations (the more landmark association we have, the more the error grows). The association we just saw favors low error association, so with zero or few landmarks. To account for this, we multiply the resulting value with a factor $p_{miss} < 1$ exponential in the number of holes:

$$p\left(\hat{z}^{[j(1)]} = z^{[1]}, \dots, \hat{z}^{[j(M)]} = z^{[M]}\right) \propto \frac{1}{\dots} \exp\left(-(\tilde{z}^{[j]} - \tilde{\mu}_z)^T \tilde{\Sigma}_z^{-1} (\tilde{z}^{[j]} - \tilde{\mu}_z)\right) p_{miss}^K \quad (8.10)$$

p_{miss}^K is the probability of not seeing k landmarks and decreases with the number of unmatched measurements K . Also associations with unmatched measurements need to be tried!

8.4 (En)forcing independence

We cut off all the assumption we don't like: first one is dependence of landmarks. **Assume** the landmarks are independent, this way we don't need to evaluate the entire measurement vector, starting by ignoring the off diagonal blocks of the covariance matrix. Since we ignore these entries, we don't need to compute them and can be set to 0:

$$\begin{aligned}
\mu_z &= \begin{pmatrix} \mu_z^{[1]} \\ \mu_z^{[2]} \\ \vdots \\ \mu_z^{[N]} \end{pmatrix} & \Sigma_z &= \begin{pmatrix} \Sigma_z^{[11]} & 0 & \dots & 0 \\ 0 & \Sigma_z^{[22]} & \dots & 0 \\ \vdots & & & \vdots \\ 0 & 0 & \dots & \Sigma_z^{[NN]} \end{pmatrix}
\end{aligned} \quad (8.11)$$

We want to maximize

$$p(z = \hat{z}^{[j]}) = \prod_{m=1}^M p(z^{[m]} = \hat{z}^{[j(m)]}) \quad (8.12)$$

which is equivalent to minimizing its logarithm

$$\log p(z = \hat{z}^{[j]}) = \sum_{m=1}^M \log p(z^{[m]} = \hat{z}^{[j(m)]}) \quad (8.13)$$

By considering the log, we also get rid of the multiplications (which becomes now a sum):

$$\sum_{m=1}^M (z^{[m]} - \mu_z^{[j(m)]})^T (\Sigma^{[j(m), j(m)]})^{-1} (z^{[m]} - \mu_z^{[j(m)]}) \quad (8.14)$$

From this we get that

- each measurement can be associated with one of the N landmarks;
- the contribution to the log likelihood is additive;
- we can precompute the likelihood for each landmark/measurement pair

$$a_{mn} = (z^{[m]} - \mu_z^{[n]})^T \Omega^{[n, n]} (z^{[m]} - \mu_z^{[n]}) \quad (8.15)$$

degree of penalty, the penalty is 0 when the landmark projects perfectly in the desired position;

- the likelihood of an association is

$$\sum_m a_{m, j(m)} \quad (8.16)$$

8.4.1 Independence and assignment

We can now organize the costs in a cost matrix A :

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1N} \\ a_{21} & a_{22} & \dots & a_{2N} \\ \vdots & & & \vdots \\ a_{M1} & a_{M2} & \dots & a_{MN} \end{pmatrix} \quad (8.17)$$

The **assignment problem** tells us that we want to assign to the agent the task that costs less. So we want to find exactly one element for each columns such that

- the sum of all entries is minimized,
- no column is selected more than once,
- the matrix is made square by padding with a default value l_{miss} modeling the cost of a missed landmark (penalty of not seeing the landmark)

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1N} \\ a_{21} & a_{22} & \dots & a_{2N} \\ \vdots & & & \vdots \\ a_{M1} & a_{M2} & \dots & a_{MN} \\ l_{miss} & l_{miss} & l_{miss} & l_{miss} \end{pmatrix} \quad (8.18)$$

8.5 Nearest neighbor

If the the measurement covariances have equal eigenvalues, the greedy association degenerates to the nearest neighbor strategy, a fast and good enough approach that assigns to each landmark the closest measurement.

In case of nearest neighbor, the entries of the association matrix are the squared distances between prediction and measurement

$$a_{mn} = (z^{[m]} - \mu_z^{[n]})^T \Omega^{[n, n]} (z^{[m]} - \mu_z^{[n]}) \quad A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1N} \\ a_{21} & a_{22} & \dots & a_{2N} \\ \vdots & & & \vdots \\ a_{M1} & a_{M2} & \dots & a_{MN} \\ l_{miss} & l_{miss} & l_{miss} & l_{miss} \end{pmatrix} \quad (8.19)$$

Even though its simple, computing the association matrix is expensive for large M, N .

8.6 Greedy association

A simpler solution is selecting for each column the row with the smaller value.

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1N} \\ a_{21} & a_{22} & \dots & a_{2N} \\ \vdots & & & \vdots \\ a_{M1} & a_{M2} & \dots & a_{MN} \end{pmatrix}$$

However, this may result in undesired configurations, like:

- multiple measurements can be assigned to the same landmark,
- we can assign a landmark to a far measurement because nothing better is found.

8.6.1 Pruning heuristic

To avoid the bad association of the previous case, we can use three heuristics:

- **gating**: ignore all associations whose cost is higher than a threshold,
- **best friends**¹⁴: an association should be the best (i.e. minimum) of both row and column,
- **lonely best friends**: one measurement should be only assigned to one single landmark (a match is a clear match compared to all the other matches around) and the difference between the minimum and the second minimum of a row/column should be higher than a threshold.

All associations that fail the test are dropped and the measurements ignored.

8.7 To sum up

Data association is hard because the measurements are not independent without knowing the state. It would require to evaluate all possible assignments.

Still errors are possible. Forcing independence renders the problem tractable in polynomial time. Acceptable solutions for SLAM can be found through greedy assignments, and potentially ambiguous situations can be avoided by dropping measurements (pruning).

¹⁴The profs favorite

9 Unscented transform

9.1 Gaussian

The multivariate Gaussian (also known as Normal) density function can be parameterized in different ways; the most common form is the moment form, parameterized by the mean value $\mu \in \mathbb{R}^n$ and the covariance matrix $\Sigma \in \mathbb{S}_{++}^n$. The probability density function (PDF) is given by:

$$p(\mathbf{x}) = \mathcal{N}(\mathbf{x}; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} \sqrt{\det \Sigma}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu)^T \Sigma^{-1} (\mathbf{x} - \mu) \right) \quad (9.1)$$

where $\mathbf{x} \in \mathbb{R}^n$ is a random vector that is Gaussian distributed.

An alternative parameterization is the **information form**, also referred to as the canonical form or natural form; this parameterization is based on the information vector ν and the (Fisher) information (or precision) matrix Ω . The PDF in information form is given by:

$$\mathcal{N}^{-1}(\mathbf{x}; \nu, \Omega) = \frac{\exp \left(-\frac{1}{2} \nu^T \Omega^{-1} \nu \right)}{(2\pi)^{n/2} \sqrt{\det \Omega^{-1}}} \exp \left(-\frac{1}{2} \mathbf{x}^T \Omega \mathbf{x} + \mathbf{x}^T \nu \right) \quad (9.2)$$

The relationship between the moment and information forms is straightforward. The parameters in the information form are related to the moment form parameters by:

$$\nu = \Sigma^{-1} \mu \quad \Omega = \Sigma^{-1} \quad (9.3)$$

The parameters Ω and ν (or ξ) provide a complete parameterization of a Gaussian, and the mean and covariance can be easily obtained from them. The difference in parameterization between the moment form and the information form leads to different computational complexities for various operations.

9.1.1 Unscented Gaussian

Instead of directly using the mean and covariance, a Gaussian can be represented using a set of control points (also called sigma points) and associated weights. For an n -dimensional Gaussian, this representation typically involves $2n + 1$ points, strategically chosen from the Gaussian distribution.

9.2 Sigma points from parameters

The location of the sigma points and their weights are computed deterministically from the mean μ and covariance Σ of the Gaussian. The goal is to select these points and weights such that they accurately capture the properties of the original distribution; specifically, the first two moments (mean and covariance) of the set of weighted sigma points are chosen to match those of the prior distribution.

For an n -dimensional Gaussian with mean μ and covariance Σ , a common set of $2n + 1$ sigma points, denoted as $\mathcal{X}^{[i]}$, is chosen according to the rule:

$$\begin{aligned} \mathcal{X} &= \mu \\ \mathcal{X}^{[i]} &= \mu + \sqrt{(n + \lambda) \Sigma_i} \quad \text{for } i = 1, \dots, n \\ \mathcal{X}^{[i]} &= \mu - \sqrt{(n + \lambda) \Sigma_{i-n}} \quad \text{for } i = n + 1, \dots, 2n \end{aligned} \quad (9.4)$$

Here, $\sqrt{(n + \lambda) \Sigma_i}$ denotes the i -th column of the matrix square root of $(n + \lambda) \Sigma$. The matrix square root can be computed using the **Cholesky decomposition**: for a symmetric positive definite matrix A , the Cholesky decomposition yields a lower triangular matrix L such that $A = LL^T$ ¹⁵.

The parameter λ is calculated as $\lambda = \alpha^2(n + \kappa) - n$:

- α and κ are scaling parameters that determine how far the sigma points are spread from the mean,
- α typically influences how far the points are from the mean and is often chosen to be a small positive value, such as 10^{-3} ,
- β can encode prior knowledge about the distribution; for an exact Gaussian, $\beta = 2$ is the optimal choice since it minimizes errors.

Each sigma point $\mathcal{X}^{[i]}$ has two weights associated with it:

¹⁵We can call L the square root of A : $L = \sqrt{A}$

- $\mathbf{w}_m^{[i]}$: used for computing the mean.
- $\mathbf{w}_c^{[i]}$: used for recovering the covariance.

The weights are calculated as follows:

$$\mathbf{w}_m^{[0]} = \frac{\lambda}{n + \lambda} \quad \mathbf{w}_c^{[0]} = \frac{\lambda}{n + \lambda} + (1 - \alpha^2 + \beta) \quad \mathbf{w}_m^{[i]} = \mathbf{w}_c^{[i]} = \frac{1}{2(n + \lambda)} \quad \text{for } i = 1, \dots, 2n \quad (9.5)$$

The sigma points can be seen as weighted control points used to control the shape of the Gaussian. We can reconstruct the mean and covariance from the sigma points and their weights:

$$\mu = \sum_{i=0}^{2n} \mathbf{w}_m^{[i]} \mathcal{X}^{[i]} \quad \Sigma = \sum_{i=0}^{2n} \mathbf{w}_c^{[i]} (\mathcal{X}^{[i]} - \mu)(\mathcal{X}^{[i]} - \mu)^T \quad (9.6)$$

The method of choosing sigma points allows the transformation to be inverted, meaning we can go back and forth between the parametric (mean/covariance) and unscented (sigma points/weights) representations.

The scaled unscented transformation introduces a positive scaling parameter α (distinct from the α used in λ) that can be made arbitrarily small to minimize higher-order effects. It is equivalent to the conventional unscented transformation followed by a simple post-processing step or modifying the weights.

The predicted mean and covariance are accurate to the second order for any value of this scaling α . Positive semi-definiteness of the predicted covariance is guaranteed if all original weights $\mathbf{w}_c^{[i]}$ are non-negative. The scaled unscented transform can also partially incorporate information about the fourth-order term in the Taylor series expansion of the covariance. The numerical costs are the same as the standard unscented transform.

9.3 Operations

9.3.1 Unscented marginalization

Let $\mathbf{x}^T = (\mathbf{x}_a^T \quad \mathbf{x}_b^T)$ be a random variable represented by an unscented transform (UT):

$$\mathbf{x} \sim \text{UT}(\mathbf{x}; x^{[i]}, \mathbf{w}_m^{[i]}, \mathbf{w}_c^{[i]}) \quad (9.7)$$

The marginal density $p(\mathbf{x}_a) = \int p(\mathbf{x}_a, \mathbf{x}_b) d\mathbf{x}_b$ can be approximated by marginalizing the unscented representation. This is achieved by keeping the sigma points and suppressing the marginalized dimensions ($\mathbf{x}_b$), while maintaining the same weights. The resulting marginal distribution $\mathbf{x}_a$ is represented by an unscented transform based on the components of the original sigma points corresponding to $\mathbf{x}_a$:

$$\mathbf{x}_a \sim \text{UT}(\mathbf{x}_a; x_a^{[i]}, \mathbf{w}_m^{[i]}, \mathbf{w}_c^{[i]}) \quad (9.8)$$

If there are more sigma points than necessary for the marginalized dimension, one can transform the representation back to parametric form (mean and covariance) and then back to an unscented representation with the appropriate number of points.

9.3.2 Unscented functions

Given a Gaussian random variable $\mathbf{x}_a$ represented by an unscented transform, and a non-linear transformation $\mathbf{x}_b = f(\mathbf{x}_a)$, an unscented approximation of the transformed variable $\mathbf{x}_b$ is obtained by projecting each sigma point through the function f :

$$\mathbf{x}_b \sim \text{UT}(\mathbf{x}_b; x_b^{[i]}, \mathbf{w}_m^{[i]}, \mathbf{w}_c^{[i]}) \quad (9.9)$$

where $x_b^{[i]} = f(x_a^{[i]})$. This means each original sigma point $\mathcal{X}_a^{[i]}$ is mapped to a new position $x_b^{[i]}$ using the function f , while retaining the original weights.

9.3.3 Unscented chain rule

Given a random variable $\mathbf{x}_a$ represented by an unscented transform and a conditional distribution $p(\mathbf{x}_b|\mathbf{x}_a) = \mathcal{N}(\mathbf{x}_b; f(\mathbf{x}_a), \Sigma_{b|a})$, we can compute the joint distribution $p(\mathbf{x}_a, \mathbf{x}_b) = \mathcal{N}(\mathbf{x}_{a,b}; \mu_{a,b}, \Sigma_{a,b})$ using the unscented transform to approximate the required statistics.

The marginal mean of $\mathbf{x}_b$ after the transformation is computed by a weighted sum of the mapped sigma points:

$$\mu_b = \sum_{i=0}^{2n} \mathbf{w}_m^{[i]} x_b^{[i]} \quad (9.10)$$

where $x_b^{[i]} = f(x_a^{[i]})$ are the projected sigma points.

The covariance matrix of the joint distribution $\Sigma_{a,b}$ is approximated using the mapped sigma points. This requires computing the covariance of the transformed points Σ_{bb} and the cross-covariance between the original and transformed points Σ_{ab} (and $\Sigma_{ba} = \Sigma_{ab}^T$) using weighted sums of the deviations from their respective means:

$$\Sigma_{ab} = \sum_{i=0}^{2n} \mathbf{w}_c^{[i]} (\mathcal{X}_a^{[i]} - \mu_a)(x_b^{[i]} - \mu_b)^T \quad \Sigma_{bb} = \sum_{i=0}^{2n} \mathbf{w}_c^{[i]} (x_b^{[i]} - \mu_b)(x_b^{[i]} - \mu_b)^T \quad (9.11)$$

With these components, the mean and covariance matrix of the joint distribution can be written as:

$$\mu_{a,b} = \begin{pmatrix} \mu_a \\ \mu_b \end{pmatrix} \quad \Sigma_{a,b} = \begin{pmatrix} \Sigma_a & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{b|a} + \Sigma_{bb} \end{pmatrix} \quad (9.12)$$

The covariance of the transformed variable includes both the projected uncertainty (Σ_{bb}) and the inherent noise in the conditional distribution ($\Sigma_{b|a}$).

In the context of the UKF update:

- $\mathbf{x}_a$ is the predicted state $\bar{\mathbf{x}}_t$ (represented by $x_t^{[i]}$),
- $\mathbf{x}_b$ is the predicted measurement $\mathbf{z}_t$ (represented by $\mathcal{Z}_t^{[i]}$),
- f is the observation function h ,
- $\Sigma_{b|a}$ is the measurement noise covariance Q_t ,
- Σ_{ab} is the cross-covariance $\Sigma_{\bar{\mathbf{x}}, \mathbf{z}_t}$, and Σ_{bb} is the covariance of the predicted measurement S_t (minus the measurement noise).

The structure is consistent with affine transformations of Gaussian variables.

9.3.4 Unscented conditioning

Unscented conditioning is not considered an easy task because the control points do not lend themselves well to being "sliced" in the way required for conditioning. Therefore, for conditioning steps, the approach is usually to go back to the usual parametric parameterization (mean and covariance) and perform conditioning in that form.

9.4 System with Gaussian noise

In the context of filtering, the system models assume that inputs and observations are affected by zero-mean Gaussian noise. The initial belief about the state is also assumed to be Gaussian.

$$\mathbf{n}_u \sim \mathcal{N}(\mathbf{n}_u | 0, \Sigma_u) \quad \mathbf{n}_z \sim \mathcal{N}(\mathbf{n}_z | 0, \Sigma_z) \quad p(\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_0 | \mu_0, \Sigma_0) \quad (9.13)$$

For compactness, the noise terms can be embedded within the state transition and observation functions.

9.5 Unscented Filtering

The Unscented Kalman Filter (UKF) is a filter that utilizes the unscented transform. It is a nonlinear extension of the Extended Kalman Filter (EKF) and it generally behaves better under nonlinear transformations compared to the linearization used in the EKF.

The UKF replaces the linearization steps of the EKF (approximating nonlinear functions with Jacobians) with the unscented transform; specifically, the unscented projection is used for the prediction step, and the unscented chain rule is used to compute the joint distribution of predicted states and measurements.

A significant advantage of the UKF is that it does not require the computation of Jacobians: this makes it a derivative-free filter, which is beneficial in domains where Jacobians are difficult to determine.

The UKF, like the EKF and standard Kalman Filter (KF), still relies on the Gaussian assumption for the belief representation and it uses pure Gaussian conditioning.

The accuracy of the unscented transform comes from the fact that it is accurate up to the first two terms of the Taylor expansion, whereas the EKF only captures the first order term. For:

- purely linear systems, the UKF produces estimates identical to those of the standard Kalman filter;
- nonlinear systems, the UKF yields results that are equal to or better than the EKF¹⁶, with the degree of improvement depending on the specific nonlinearities and the spread of the uncertainty.

The UKF is still highly efficient despite being slightly slower than the EKF in practice, as its asymptotic complexity is the same.

¹⁶In many practical applications the difference may be negligible

9.5.1 Predict

The prediction step in the UKF involves incorporating the control input to compute the Gaussian distribution of the next state. The next state $\mathbf{x}_t$ is a nonlinear function of the previous state $\mathbf{x}_{t-1}$ and the control input $\mathbf{u}_{t-1}$:

$$\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_{t-1}) \quad (9.14)$$

The previous state and control are assumed to be jointly distributed according to a Gaussian:

$$\begin{pmatrix} \mathbf{x}_{t-1|t-1} \\ \mathbf{u}_{t-1} \end{pmatrix} \sim \mathcal{N} \left(\begin{pmatrix} \mu_{t-1|t-1} \\ \mu_{u,t-1} \end{pmatrix}, \begin{pmatrix} \Sigma_{t-1|t-1} & 0 \\ 0 & \Sigma_{u,t-1} \end{pmatrix} \right) \quad (9.15)$$

where the 0 matrices indicate independence between the state and control uncertainty at this step.

Sigma points $\chi_{t-1|t-1}^{[i]}$ are computed from this joint Gaussian distribution of the previous state and control and have dimension $\dim(\mathbf{x}) + \dim(\mathbf{u})$.

Next, these sigma points are projected through the transition function f to get the predicted state sigma points:

$$\chi_{t|t-1}^{[i]} = f(\chi_{t-1|t-1}^{[i]}) \quad (9.16)$$

These predicted state sigma points $\chi_{t|t-1}^{[i]}$ have dimension $\dim(\mathbf{x})$.

The predicted mean $\bar{\mu}_t$ and covariance $\bar{\Sigma}_t$ are then computed from these predicted state sigma points using the weighted sums:

$$\bar{\mu}_t = \sum_{i=0}^{2n} \mathbf{w}_m^{[i]} \chi_{t|t-1}^{[i]} \quad \bar{\Sigma}_t = \sum_{i=0}^{2n} \mathbf{w}_c^{[i]} (\chi_{t|t-1}^{[i]} - \bar{\mu}_t)(\chi_{t|t-1}^{[i]} - \bar{\mu}_t)^T + \mathbf{R}_t \quad (9.17)$$

The prediction (additive Gaussian) noise covariance $\mathbf{R}_t$ is added to model additional prediction noise uncertainty.

9.5.2 Update: chain rule

The update step begins by considering the distribution of the predicted state $p(\mathbf{x}_t)$, which is now represented by an unscented transform based on the predicted mean $\bar{\mu}_t$ and covariance $\bar{\Sigma}_t$:

$$p(\mathbf{x}_t) = \text{UT}(\mathbf{x}_t; \chi_{t|t-1}^{[i]}) \quad (9.18)$$

A new set of sigma points $\bar{\mathcal{X}}_t$ is extracted from this predicted Gaussian:

$$\bar{\mathcal{X}}_t = (\bar{\mu}_t \quad \bar{\mu}_t + \gamma\sqrt{\bar{\Sigma}_t} \quad \bar{\mu}_t - \gamma\sqrt{\bar{\Sigma}_t}) \quad (9.19)$$

where $\gamma = \sqrt{n + \lambda}$.

The observation model is given by $p(\mathbf{z}_t|\mathbf{x}_t) = \mathcal{N}(\mathbf{z}_t; h(\mathbf{x}_t), \Sigma_{z|x})$, where h is the observation function and $\Sigma_{z|x}$ is the measurement noise covariance, denoted as $\mathbf{Q}_t$ in the algorithm table.

We want to compute the joint probability $p(\mathbf{x}_t, \mathbf{z}_t)$. To do so, we use the unscented chain rule: a predicted observation $\bar{\mathcal{Z}}_t$ is computed for each sigma point $\bar{\mathcal{X}}_t^{[i]}$ by passing it through the observation function h :

$$\bar{\mathcal{Z}}_t^{[i]} = h(\bar{\mathcal{X}}_t^{[i]}) \quad (9.20)$$

These resulting observation sigma points $\bar{\mathcal{Z}}_t$ are used to compute the predicted observation mean $\hat{\mathbf{z}}_t$ and its uncertainty $\mathbf{S}_t$:

$$\text{hatz}_t = \sum_{i=0}^{2n} \mathbf{w}_m^{[i]} \bar{\mathcal{Z}}_t^{[i]} \quad \mathbf{S}_t = \sum_{i=0}^{2n} \mathbf{w}_c^{[i]} (\bar{\mathcal{Z}}_t^{[i]} - \hat{\mathbf{z}}_t)(\bar{\mathcal{Z}}_t^{[i]} - \hat{\mathbf{z}}_t)^T + \mathbf{Q}_t \quad (9.21)$$

$\mathbf{S}_t$ represents the predicted measurement covariance, including the additive measurement noise $\mathbf{Q}_t$.

The cross-covariance between the state and observation is also computed from the predicted state sigma points $\bar{\mathcal{X}}_t$ and the predicted observation sigma points $\bar{\mathcal{Z}}_t$:

$$\Sigma_{\bar{x}, z_t} = \sum_{i=0}^{2n} \mathbf{w}_c^{[i]} (\bar{\mathcal{X}}_t^{[i]} - \bar{\mu}_t)(\bar{\mathcal{Z}}_t^{[i]} - \hat{\mathbf{z}}_t)^T \quad (9.22)$$

This cross-covariance $\Sigma_{\bar{x}, z_t}$ corresponds to the term $\bar{\Sigma}_t \mathbf{H}_t^T$ in the EKF algorithm.

9.5.3 Update: conditioning

Using the joint properties computed in the previous step:

- predicted state mean $\bar{\mu}_t$,
- predicted state covariance $\bar{\Sigma}_t$,
- predicted measurement mean $\hat{\mathbf{z}}_t$,
- predicted measurement covariance $\mathbf{S}_t$,
- predicted measurement cross-covariance $\Sigma_{\bar{x}, z_t}$

we condition on the actual measurement $\mathbf{z}_t$ to obtain the updated state mean μ_t and covariance Σ_t . First, the Kalman gain $\mathbf{K}_t$ is computed:

$$\mathbf{K}_t = \Sigma_{\bar{x}, z_t} \mathbf{S}_t^{-1} \quad (9.23)$$

The state mean and covariance are then updated using formulas equivalent to the EKF update:

$$\mu_t = \bar{\mu}_t + \mathbf{K}_t(\mathbf{z}_t - \hat{\mathbf{z}}_t) \quad \Sigma_t = \bar{\Sigma}_t - \mathbf{K}_t \mathbf{S}_t \mathbf{K}_t^T \quad (9.24)$$

9.5.4 Wrapup (UKF)

The steps for the Unscented Kalman Filter (UKF) algorithm are summarized as follows:

Predict:

1. Compute sigma points from the previous belief (mean and covariance), potentially including control input.
2. Apply the transition function to the sigma points to get predicted state sigma points. Compute the predicted state mean and covariance from these points.

Update:

1. Compute sigma points from the predicted state mean and covariance. Apply the observation function to these points to get predicted measurement sigma points. Compute the predicted measurement mean from these points.
2. Compute the cross-correlation (cross-covariance) between the predicted state sigma points and the predicted measurement sigma points, as well as the covariance of the predicted measurement.
3. Apply Gaussian conditioning (using the Kalman gain derived from the computed covariances and cross-covariance) based on the actual measurement to update the state mean and covariance.

9.6 Summary

The UKF is a powerful nonlinear extension of the EKF. It offers potentially improved accuracy over the EKF when dealing with nonlinear transition and observation functions. A key advantage is that it is derivative-free, avoiding the need to compute Jacobian matrices. The UKF achieves its performance by using a deterministic set of sigma points to propagate the mean and covariance through the nonlinearities, which is accurate to the second order of the Taylor expansion. While often slightly more computationally expensive than the EKF in practice (though with the same asymptotic complexity), it is still highly efficient. The UKF is equivalent to the linear Kalman filter for purely linear systems. However, it still relies on the assumption that the belief can be adequately represented by a Gaussian distribution. If the true distribution is highly non-Gaussian, the UKF's performance can degrade. The scaled unscented transform further enhances the UKF by allowing sigma points to be scaled to manage higher-order effects and potentially incorporate additional information.

10 Differentiation

10.1 Approaching the problem

We aim to develop a Kalman Filter (KF)-based algorithm to track the position of Orazio as it moves given:

- velocity measurements,
- landmark measurements.

Prior knowledge about the map is represented by the location of each landmark in the world.

10.2 Numerical differentiation

Using the definition of a derivative

$$f'(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x} \quad (10.1)$$

given a set of perturbation vectors

$$\epsilon_1 = \begin{pmatrix} \epsilon \\ 0 \\ \vdots \\ 0 \end{pmatrix} \quad \epsilon_2 = \begin{pmatrix} 0 \\ \epsilon \\ \vdots \\ 0 \end{pmatrix} \quad \dots \quad \epsilon_m = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ \epsilon \end{pmatrix} \quad (10.2)$$

we compute the i -th column as follows:

$$\frac{\partial f}{\partial x_i} \simeq \frac{f(x + \epsilon_i) - f(x - \epsilon_i)}{2\epsilon} \quad \frac{\partial f(x)}{\partial x} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots & \frac{\partial f_1}{\partial x_m} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \dots & \frac{\partial f_2}{\partial x_m} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \dots & \frac{\partial f_n}{\partial x_m} \end{pmatrix} \quad (10.3)$$

Each column requires evaluation f 2 times. Notice that the symmetry around linearization point in equation ?? leads to lower numerical errors.

Choosing an appropriate ϵ is crucial and not trivial:

- too small leads to machine precision errors,
- too large results in a poor derivative approximation,
- computational complexity may be reduced.

Despite this, numerical differentiation is easy to implement and often works well.

Instead of manually computing derivatives, we can use parsing trees for efficient computation.

10.3 Chain rule

To evaluate the derivative of a nested function $f(g(x))$ in a point $\hat{x}$, we need:

- the formula of the derivative of f : $f'(y) = \frac{\partial f(y)}{\partial y}$,
- the value of the argument of f : $y = g(\hat{x})$,
- the **value** of the derivative of the argument $y' = g'(\hat{x})$

The chain rule tells us that

$$\frac{\partial f(g(x))}{\partial x} \Big|_{x=\hat{x}} = f'(y)y' \quad (10.4)$$

10.4 Parsing tree example

Consider the function $\cos(5x)/\sin(\exp(x+3))$, its parsing tree enables systematic differentiation shown in figure ??.

10.4.1 Parsing tree for derivatives

A parsing tree can compute the value of a function and the value of the derivative of the function.

This is achieved by replacing basic types (float/double) with pairs (u, u') and redefining operators consistently.

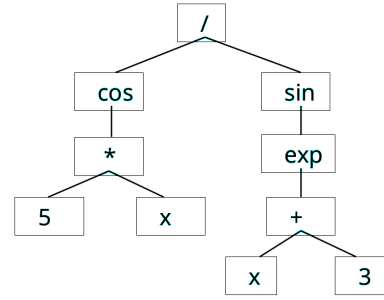


Figure 10.1: Parsing three

10.5 Atoms and unary functions

For differentiation (atoms):

- the variable becomes a pair $[x, 1]$, where x is the evaluation point and 1 is the derivative of the point I want to evaluate;
- constants become a pair $[c_i, 0]$.

Some transcendental functions are:

$$\begin{aligned}
 \sin([u, u']) &= [\sin(u), \cos(u) * u'] \\
 \cos([u, u']) &= [\cos(u), -\sin(u) * u'] \\
 \exp([u, u']) &= [\exp(u), \exp(u) + u']
 \end{aligned}
 \tag{10.5}$$

10.6 Binary operators

Sum, subtraction, multiplications and division are implemented by applying the derivative rules on their arguments:

$$\begin{aligned}
 [u, u'] + [v, v'] &= [u + v, u' + v'] \\
 [u, u'] - [v, v'] &= [u - v, u' - v'] \\
 [u, u'] \times [v, v'] &= [uv, u'v + v'u] \\
 [u, u'] / [v, v'] &= [u/v, (u'v - v'u)/v^2]
 \end{aligned}
 \tag{10.6}$$

11 Particle distributions and particle filters

11.1 Introduction

Particle filters are an alternative nonparametric implementation of the Bayes filter. The core idea is to represent the posterior $bel(x_t)$ by a set of random state samples, or particles, drawn from this posterior; this representation is approximate, but it is nonparametric, allowing it to represent a much broader space of distributions than parametric forms like Gaussians.

A **particle** is a concrete instantiation of the state at time t , effectively a hypothesis about the true world state. A set of M ¹⁷ particles $X_t := \{x_t^{[0]}, x_t^{[1]}, \dots, x_t^{[M]}\}$ represents the belief $bel(x_t)$. The intuition is that the likelihood of a state hypothesis x_t being included in the particle set X_t should ideally be proportional to its Bayes filter posterior $bel(x_t)$:

$$x_t^{[m]} \sim p(x_t | z_{1:t}, u_{1:t}) \quad (11.1)$$

Consequently, the denser a subregion of the state space is populated by samples, the more likely it is that the true state falls into this region. This property holds asymptotically for $M \uparrow \infty$ for the standard particle filter algorithm. Particle filters construct the particle set X_t recursively from the set X_{t-1} , just like other Bayes filter algorithms construct $bel(x_t)$ recursively from $bel(x_{t-1})$.

11.2 Sampling from a distribution

Sampling refers to generating a set of samples from a known distribution $p(x)$. Most random number generators typically sample from the uniform distribution $U(0, 1)$. The challenge lies in efficiently generating samples from an arbitrary distribution $p(x)$.

11.2.1 Generating samples (assuming $p(x)$ in closed form)

If the probability density function (PDF) $p(x)$ is known in closed form, samples can be generated using the inverse of the cumulative distribution function (CDF). The process is as follows:

1. compute the cumulative distribution function (CDF): $P(x) = \int_{-\infty}^x p(x') dx'$;
2. draw a sample $y^{(i)}$ from the uniform distribution $U(0, 1)$: $y^{(i)} \sim U(0, 1)$;
3. compute the inverse of the cumulative function at $y^{(i)}$ to obtain the sample $x^{(i)} = P^{-1}(y^{(i)})$.

Iterating this process generates denser samples in regions where $p(x)$ is higher, as the CDF is steeper where the PDF is higher (figure ??).

11.2.2 Discrete case

For discrete distributions, a similar approach is followed, but the CDF forms a staircase function with uneven steps (figure ??).

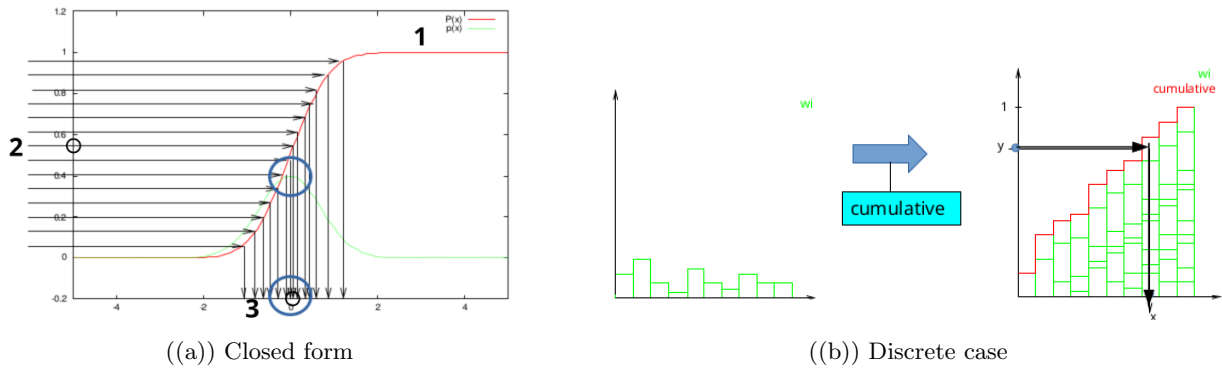


Figure 11.1: Generating samples

¹⁷ M is often a large number (e.g., $M = 1,000$)

11.2.3 Uniform sampling

When generating M random samples from a discrete distribution, calling the random number generator M times can be expensive. An alternative approach is **uniform sampling**: instead of choosing M independent random numbers, a single random number $y^{(0)}$ is sampled from $U(0, 1/M)$. The remaining $M - 1$ samples $y^{(k)}$ are then determined deterministically by adding a fixed increment $1/M$ to the previous sample, effectively sampling from a regular grid shifted by $y^{(0)}$:

$$y^{(k)} = y^{(0)} + k/M \quad (11.2)$$

for $k = 1, \dots, M - 1$. This method provides an algorithm with linear complexity $O(M)$ and a deterministic yet randomized selection of samples.

11.2.4 Importance sampling

Sometimes, the target distribution $p(x)$ is not known in closed form, making the inverse CDF method impossible. However, it might be possible to evaluate $p(x)$ point-wise. In such cases, **Importance Sampling** (IS) is used to generate weighted samples:

1. sample from a known proposal distribution $\pi(x)$, which should be reasonably close to the target distribution $p(x)$: $x^{[m]} \sim \pi(x)$;
 2. compute a weight $w^{[m]}$ for each sampled point $x^{[m]}$ by evaluating the ratio of the target distribution $p(x^{[m]})$ to the proposal distribution $\pi(x^{[m]})$ at that point: $w^{[m]} = \frac{p(x^{[m]})}{\pi(x^{[m]})}$.
- These weights, also called importance factors, account for the discrepancy between the target and proposal distributions. The weighted samples $x^{[m]}$ converge to the density $p(x)$ even though they were generated from $\pi(x)$.

For this method to be correct, if $p(x) > 0$, then $\pi(x)$ must also be greater than 0, ensuring the proposal covers all relevant portions of the target distribution. In the ideal case where $p(x) = \pi(x)$, the weights would be uniform.

11.3 Resampling

Importance Sampling approximations can suffer from increasing variance, to address this problem resampling techniques are used. The primary goal of resampling is to convert a weighted sample set $\{(x^{[m]}, w^{[m]})\}$ into an unweighted one; to do so, samples with high weights are repeated, and samples with low weights are suppressed. This is done by drawing a new set of indices from the normalized weights distribution and the probability of drawing index j is proportional to its normalized weight $\tilde{w}^{[j]}$:

$$p(\text{draw index } j) = \tilde{w}^{[j]} = \frac{w^{[j]}}{\sum_i w^{[i]}} \quad (11.3)$$

Given the weighted sample set $\{(x^{[m]}, w^{[m]})\}$ (input), the steps are:

1. sample a set of M indices $\{j^{[m]}\}$ with probability proportional to the normalized weights;
2. the new sample $\tilde{x}^{[m]}$ at position m in the resampled set is the old sample $x^{[j^{[m]}]}$ corresponding to the drawn index $j^{[m]}$: $\tilde{x}^{[m]} = x^{[j^{[m]}]}$.

Resampling allows the algorithm to not carry forward particles with low weights, focusing computational efforts on regions of high probability mass; however, it inherently introduces noise.

11.3.1 Low variance sampling

¹⁸ Resampling, while necessary, amplifies sampling variance and can lead to a loss of diversity. A strategy to reduce this variance during resampling is Low Variance Sampling. Unlike standard resampling which selects samples independently, Low Variance Sampling uses a sequential stochastic process.

The algorithm uses a single random number to draw M particles:

- it draws a random number r in the interval $[0, M^{-1}]$;

¹⁸Note: this is extra from the book and NOT IN THE SLIDES

- it iteratively selects particles. A particle i is selected if a value $U = r + (m - 1) \cdot M^{-1}$ falls within the cumulative weight range of particle i . Formally, for a number $U \in$, it points to particle i for which:

$$i = \operatorname{argmin}_j \sum_{k=1}^j w_t^{[k]} \geq U \quad (11.4)$$

. Then, The algorithm efficiently implements this in $O(M)$ time. Advantages include

- covering the sample space more systematically than independent sampling,
- preserving samples if weights are uniform,
- efficiency.

Stratified sampling is another related technique.

11.4 Particle densities

A probability density function $p(x)$ can be approximated by a set of weighted samples $\{(x^{[m]}, w^{[m]})\}$. The probability density is represented by the density of samples; regions with denser samples have higher probability. The probability that the state x falls into a region E can be approximated by summing the weights of the particles located within that region:

$$\int_E p(x) dx \simeq \sum_{x^{[m]} \in E} w^{[m]} \quad (11.5)$$

The particle representation can be formally written as a sum of Dirac delta functions centered at each particle location, weighted by the importance weights:

$$p(x) \simeq \sum_m w^{[m]} \delta(x - x^{[m]}) \quad (11.6)$$

Particle filters maintain sample sets as discrete approximations of continuous beliefs. Obtaining a continuous estimate from these samples is known as density estimation and there exists various approaches, like:

- **Gaussian approximation:** computes the mean and variance of the particles and fits a Gaussian distribution. This is simple and efficient but only suitable for unimodal densities.
- **k-means clustering:** can approximate multimodal densities by using mixtures of Gaussians.
- **Histograms:** superimposes a grid over the state space and sums the weights of particles falling into each bin. Can represent multimodal distributions and are efficient to compute, but space complexity is exponential in the number of dimensions.
- **Density tree:** a variation of histograms that can reduce space complexity but increases density lookup time.
- **Kernel density estimation:** places a kernel function (e.g., Gaussian) at each particle location and sums them to form the overall density estimate. Provides smooth density estimates but density computation at any point is linear in the number of particles.

The choice of density extraction method depends on the application's needs and computational constraints.

11.4.1 Why particles are cool

Particle filters offer several advantages:

- they can represent arbitrary distributions, unlike parametric filters that are limited to specific forms like Gaussians. This allows them to handle multimodal or non-Gaussian beliefs;
- they are easy to visualize and manipulate;
- they are effective for small state spaces;
- they can model nonlinear transformations of random variables.

11.5 Operations

11.5.1 Transformations

Given a sampled density

$$p(x_a) \simeq \sum_m w^{[m]} \delta(x_a - x_a^{[m]}) \quad (11.7)$$

and a random variable x_b that is a function of x_a , $x_b = f(x_a)$, the sampled density on x_b is obtained by transforming each sample through the function f :

$$p(x_b) \simeq \sum_m w^{[m]} \delta(x_b - f(x_a^{[m]})) \quad (11.8)$$

This is implemented by simply transforming each particle: $x_b^{[m]} = f(x_a^{[m]})$.

The weights $w^{[m]}$ remain unchanged during this transformation.

11.5.2 Marginalization

Marginalization from a joint sampled density is straightforward: given a joint density

$$p(x_a, x_b) \simeq \sum_m w^{[m]} \delta \begin{pmatrix} x_a \\ x_b \end{pmatrix} - \begin{pmatrix} x_a^{[m]} \\ x_b^{[m]} \end{pmatrix} \quad (11.9)$$

marginalizing one component (e.g., x_b) is implemented by simply deleting the coordinates of the marginalized component from the sample set:

$$p(x_a) = \int p(x_a, x_b) dx_b \simeq \sum_m w^{[m]} \delta(x_a - x_a^{[m]}) \quad (11.10)$$

11.5.3 Chain Rule

Given a sampled density on x_a

$$p(x_a) \simeq \sum_m w^{[m]} \delta(x_a - x_a^{[m]}) \quad (11.11)$$

and a conditional distribution $p(x_b|x_a)$, the joint distribution $p(x_a, x_b)$ can be represented by samples using the chain rule.

For each sample $x_a^{[m]}$ from the sampled density, a sample $x_b^{[m]}$ is generated from the conditional distribution $p(x_b|x_a^{[m]})$. The resulting joint particle is obtained by stacking these samples:

$$p(x_a, x_b) \simeq \sum_m w^{[m]} \delta \begin{pmatrix} x_a \\ x_b \end{pmatrix} - \begin{pmatrix} x_a^{[m]} \\ x_b^{[m]} \end{pmatrix} \quad (11.12)$$

11.5.4 Conditioning

Applying conditioning directly on samples is not easy; however, by considering the conditional distribution $p(x_b|x_a)$ for each sample $x_a^{[m]}$, we can analyse how conditioning on x_b affects the sample set. Starting from a mixture of conditional distributions for each sample

$$p(x_a, x_b) \simeq \sum_m w^{[m]} \delta(x_a - x_a^{[m]}) p(x_b|x_a^{[m]}) \quad (11.13)$$

and applying chain rule and marginalization, the conditional distribution $p(x_a|x_b)$ can be derived:

$$p(x_a|x_b) = \frac{p(x_a, x_b)}{p(x_b)} = \frac{p(x_a, x_b)}{\int p(x_a, x_b) dx_a} \quad (11.14)$$

Applying the mixture approximation, we find that conditioning primarily modifies the weights of the samples, leaving their positions unchanged:

$$p(x_a|x_b) \simeq \eta \sum_m w^{[m]} \delta(x_a - x_a^{[m]}) p(x_b|x_a^{[m]}) \quad \eta = \left(\sum_m w^{[m]} p(x_b|x_a^{[m]}) \right)^{-1} \quad (11.15)$$

where η is the normalizer. The updated weight $w_{a|b}^{[m]}$ for particle m given conditioning on x_b is proportional to the original weight $w^{[m]}$ multiplied by the likelihood of x_b given the particle's state $x_a^{[m]}$:

$$w_{a|b}^{[m]} \propto w^{[m]} p(x_b | x_a^{[m]}) \quad (11.16)$$

To implement conditioning on a variable x_b , each particle's weight is multiplied by the conditional probability $p(x_b | x_a^{[m]})$ evaluated at the conditioning value x_b and the particle's state $x_a^{[m]}$.

11.6 Dynamic systems and the particle filter

For dynamic systems:

- state transition is governed by a function and/or noise,
- observations are related to the state via an observation model,
- the state PDF is represented by a set of weighted samples.

The particle filter algorithm, summarized in table ??, proceeds recursively in prediction and update steps.

Predict

1. generate I noise samples

$$n_u^{(i)} \sim p(n_u)$$

2. apply f to each sample and noise pair

$$\begin{aligned} x_{t-1} &= f(x_{t-1|t-1}^{(i)}, u_{t-1} + n_u^{(i)}) \\ w_{t|t-1}^{(i)} &= w_{t-1|t-1}^{(i)} \end{aligned}$$

Update

3. multiply each weight by the conditional of the measurement evaluated at the weight
4. resample a new generation to focus computation on likely regions of the state space

$$\begin{aligned} x_{t|t}^{(i)} &= x_{t|t-1}^{(i)} \\ w_{t|t}^{(i)} &= w_{t|t-1}^{(i)} p(z_t | x_{t|t-1}^{(i)}) \end{aligned}$$

Table 11.1: Particle filter

11.6.1 Prediction step

Given the particle set X_{t-1} representing the belief $bel(x_{t-1})$

$$bel(x_{t-1}) = p(x_{t-1} | t-1) \simeq \sum_m w_{t-1|t-1}^{[m]} \delta(x_{t-1|t-1} - x_{t-1|t-1}^{[m]}) \quad (11.17)$$

the prediction step simulates the system dynamics to obtain a particle set $\bar{X}_t$ representing the belief $bel(x_t)$ before incorporating the measurement:

1. for each particle $x_{t-1}^{[m]}$ in X_{t-1} , sample a hypothetical state $x_t^{[m]}$ for time t by sampling from the state transition distribution $p(x_t | u_t, x_{t-1})$ given the control u_t and the previous state particle $x_{t-1}^{[m]}$:

$$x_t^{[m]} \sim p(x_t | u_t, x_{t-1}^{[m]}) \quad (11.18)$$

Alternatively described as applying a transition function f with additive noise n_u : $x_{t|t-1}^{[m]} = f(x_{t-1|t-1}^{[m]}, u_{t-1} + \nu_u^{(m)})$ where $n_u^{(m)} \sim p(\nu_u)$;

2. these new particles form the temporary set $\bar{X}_t$;
3. the weights typically remain unchanged in this step: $w_{t|t-1}^{[m]} = w_{t-1|t-1}^{[m]}$.

11.6.2 Update step

The update step incorporates the latest measurement z_t to refine the belief. This is done by weighting the predicted particles based on the likelihood of the measurement given the particle's state:

1. for each predicted particle $x_t^{[m]}$ in $\bar{X}_t$, calculate its importance factor $w_t^{[m]}$, which is the probability of observing z_t given the particle's state $x_t^{[m]}$:

$$w_t^{[m]} = p(z_t | x_t^{[m]}) \quad (11.19)$$

2. the predicted weight $w_{t|t-1}^{[m]}$ is multiplied by this likelihood to obtain the updated weight $w_{t|t}^{[m]}$:

$$w_{t|t}^{[m]} = w_{t|t-1}^{[m]} p(z_t | x_t^{[m]}) \quad (11.20)$$

3. the sample locations themselves do not change in this step: $x_{t|t}^{[m]} = x_{t|t-1}^{[m]}$.

The set of weighted particles $\{(x_{t|t}^{[m]}, w_{t|t}^{[m]})\}$ now approximates the Bayes filter posterior $bel(x_t)$.

11.6.3 Particle filter algorithm

The basic algorithm takes X_{t-1} , u_t , and z_t as input and returns X_t . The resampling step forces particles back to the posterior $bel(x_t)$ by selecting particles based on their weights. Particles with lower weights are less likely to be drawn. This step is crucial for focusing computation on likely regions.

11.7 Practical considerations and properties

While powerful, particle filters have properties and limitations that require practical consideration:

- particle filters use a finite number of samples, which introduces **variance inherent to random sampling**. Statistics computed from the sample set (e.g., mean, variance) will differ slightly from the true density's statistics. This is called sample variance. The sample variance decreases as the number of particles M increases. Under mild conditions, the approximation of the target density using weighted samples converges with a rate often $O(1/\sqrt{M})$;
- although resampling is vital for refocusing particles, repetitive resampling can amplify sampling variance and cause a **loss of diversity in the particle set**. This increases the variance of the particle set as an estimator of the true belief. In extreme cases, diversity can disappear, leading to M identical copies of a single particle. Controlling this variance is essential. Strategies include reducing resampling frequency (e.g., suspending resampling when the state is static or integrating multiple measurements via multiplicative weight updates $w_t^{[m]} = p(z_t | x_t^{[m]}) w_{t-1}^{[m]}$), or using low variance sampling. The variance of importance weights can indicate whether resampling is necessary;
- using a finite number of particles also introduces a **systematic bias** in the posterior estimate. For instance, with $M = 1$ particle, the resampling step deterministically selects the single particle regardless of its weight, causing the measurement z_t to be ignored. This bias stems from the implicit normalization in the resampling step. The effect of this loss of dimensionality decreases as M increases;
- the **particle deprivation problem** occurs when, despite having many particles, none are located in the vicinity of the correct state. This can happen if the number of particles is too small relative to the space of states with high likelihood, or due to an unlucky sequence of random numbers during sampling and resampling. Over time, this can theoretically lead to arbitrarily incorrect estimates. A simple, though not ideal, solution is to add a small number of randomly generated particles to the set after each resampling step, which can mitigate the problem but introduces a bias in the posterior estimate;
- the quality of the sample-based approximation improves with the number of samples M . Determining the required number of samples is problem-dependent. It **strongly depends on the dimensionality of the state space** and the uncertainty of the distributions being approximated; uniform distributions require many more samples than focused distributions. There is no perfect answer, and it often requires practical experience.

12 Finding neighbours

12.1 Neighbour search

Given:

- a collection of vectors $\mathcal{P} = \{p_n\}_{n=1:N}$, where $p_n \in \mathcal{R}^k$,
- a query vector $p_q \in \mathcal{R}^k$,
- a distance metric $d(p_n, p_q) \in \mathcal{R}^+$,

The goal is to find either

- the point in the collection that's closest to the query according to the metric: $p_i = \arg \min_{p_n \in \mathcal{P}} d(p_q, p_n)$,
- the point in the collection whose distance from the query is smaller than a certain value ϵ : $\mathcal{P}' = \{p_i \in \mathcal{P}, d(p_q, p_i) < \epsilon\}$

How can we check this distances? Two main way: collision check or solid geometric computer vision.

12.2 Distance metrics

There are different distance metrics that can be used, some of them are:

- squared norm (Squared Euclidean distance): $\|p_i - p_j\|^2 = (p_i - p_j)^T (p_i - p_j)$;
- omega norm: $\|p_i - p_j\|_\Omega^2 = (p_i - p_j)^T \Omega (p_i - p_j)$
- Hamming distance (for binary descriptors): integer valued distance between two bit strings having the same dimension, its value is the number of different bits (you count the number of bits you need to flip so that the two strings are the same. We already saw this during computer vision: ORB).

12.2.1 Brute force approach

Computing the distance for every query point and each of the points in the collection and update the minimum. It's good for testing, not for true applications since the computational complexity it's too high ($O(N * cost_distance_metric)$), which will results in unacceptable delays.

12.2.2 Distance map

If the vectors have a small dimension (e.g., < 3) and are spread in relatively a small region, we can pre-compute a grid lookup table, which can store (in each cell):

- distance to the closest point,
- identity of the closest point.

The distance map is computed using a modified Dijkstra algorithm; we initialize the grid cells which corresponds to the points in $\mathcal{P}$ as:

- $p.d = 0$, where d is the distance form the closest point;
- $p.parent = 0$, where $parent$ is the pointer to the closest point.

Then we expand each initialized grid cell p (by taking, for example, 8 neighbors); for each of the expanded cells C_i :

```
d_{p,c} = distance(p.parent, c_i)
if (c_i.isEmpty()) {
    c_i.d = d_{p,c}
    c_i.parent = p.parent
} else if (c_i.d > d_{p,c} ) {
    c_i.d = d_{p,c}
    c_i.parent = p.parent
}
```


Continue iteratively for all the expanded cells.

The complexity depends whether we need to keep the queue sorted, $O(\text{grid size} \times \log(\text{grid size}))$, or not $O(\text{grid size})$. One query on the distance map requires $O(1)$. Distance maps are particularly useful when:

- we have many low dimensional points in the collection,
- the grid is small,
- we can tolerate small association errors (one badly associated point is not source of significant errors).

We have three different types of heuristics¹⁹:

1. gating: expand up to a maximum distance;
2. best friends: construct a distance map also for the measurements and cross check;
3. lonely best friends: looking up a region of the distance map instead of a single point.

12.2.3 KD-Trees

For higher-dimensional data, we cannot use distance map since the memory (and so the time to build it) grows with the dimension of the points.

KD-trees are a search structure that partitions the database according to their spatial distribution and reduce query complexity to $O(\log N)$ for a well balanced tree (N is the number of points).

The tree is constructed recursively, splitting, at each time, the set in two parts until the number of points in a leaf is smaller than a threshold.

Consider the K -dimensional points as if they were normally distributed, to split the set

1. compute the covariance matrix of the distribution,
2. chose a splitting (hyper) plane that passes through the mean and has a normal aligned with the longest axis of the covariance matrix.

This guarantees that the policy on the leaves is closed. We can check if a point lies on the one side of the plane by evaluating

$$n_i^T(p_q - \mu_i) > 0 \quad (12.1)$$

where

- n_i is the normal of the split plane,
- μ_i is the mean of the split plane,
- p_q is the query point.

Each intermediate node of the tree contains

- the normal of the splitting plane,
- the mean,
- pointers to the children nodes.

A query requires traversing the tree from the top to the bottom and at each time going left or right. The query result is an approximation, i.e. the tree is a heuristic that might return not the real minimum, depending on how the tree was built.

12.2.4 Other tools

Other choices are:

- dimensionality reduction: reduce the dimension by projecting them, for example, in 2D and then use some easy heuristic (distance map) to perform the association in the lower dimensional space;
- Bag of Words: determine the appearance similarity of multiple points.

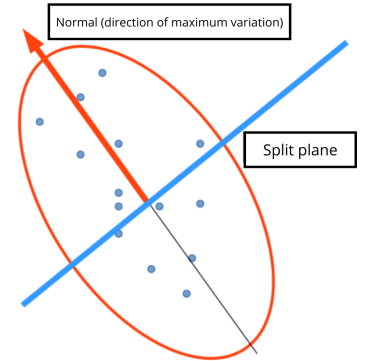


Figure 12.1: KD-Trees: simple splitting

¹⁹An heuristic is an experience-based method used to reduce the use of calculations, it provides a correct but approximate solution

12.3 Applications

These methods are used

- to speed up localization (kd-tree, distance map). In localization we are dealing with large data structures that cannot be saved in memory,
- SLAM: both distance map costs and KD-tree are expensive, since the map changes you'll have to recalculate the KD-tree for each new update.

Distance maps are efficient for static environments, while KD-trees must be recomputed for dynamic updates.

13 Least squares introduction ICP optimization

Least squares is a widely used technique for state estimation, allowing us to infer parameters or states based on observations. The core idea is to find a solution to an overdetermined system where we have more observations (equations) than unknown parameters. Since observations are typically affected by noise, a perfect solution where all errors are zero is usually not possible; instead, the least squares approach minimizes the sum of the squares of the errors.

13.1 Maximum Likelihood estimation

Drawing on Bayes' rule and assuming independence of measurements, the goal in many state estimation problems is to estimate the most likely state x given a set of observations z . This is expressed as finding the x^* that maximizes the conditional probability $p(x|z)$ ($x^* = \arg \max_x p(x|z)$). Using Bayes' rule

$$p(x|z) = \frac{p(z|x)p(x)}{p(z)} \quad (13.1)$$

and assuming $p(z)$ is constant with respect to x , maximizing $p(x|z)$ is equivalent to maximizing $p(z|x)p(x)$. If we further assume independence of individual measurements $z^{[i]}$, then

$$p(z|x) = \prod_i p(z^{[i]}|x) \quad (13.2)$$

13.2 Gaussian assumption

Assuming that the measurement noise is Gaussian simplifies the problem significantly. The probability of observing $z^{[i]}$ given the state x is

$$p(z^{[i]}|x) = \mathcal{N}(z^{[i]}; h^{[i]}(x), \Sigma^{[i]}) \propto \exp\left(-(h^{[i]}(x) - z^{[i]})^T \Sigma^{[i]-1} (h^{[i]}(x) - z^{[i]})\right) \quad (13.3)$$

We can define the maximization problem as

$$x^* = \arg \max_x \prod_i p(z^{[i]}|x) = \arg \max_x \prod_i \exp(-e^{[i]}(x)^T \Omega^{[i]} e^{[i]}(x)) \quad (13.4)$$

Where:

- $e^{[i]}(x) = h^{[i]}(x) - z^{[i]}$ is the error function and follows a Gaussian distribution,
- $h^{[i]}(x)$ is the predicted observation for measurement i given state x ,
- $z^{[i]}$ is the actual observation,
- $\Sigma^{[i]}$ is the covariance matrix,
- $\Omega^{[i]} = \Sigma^{[i]-1}$ is the information matrix.

Maximizing the product of exponentials is equivalent to minimizing the sum of the terms in the exponent, thus, the problem becomes minimizing the sum of squared errors weighted by the information matrices:

$$x^* = \arg \min_x \sum_i e^{[i]T}(x) \Omega^{[i]} e^{[i]}(x) \quad (13.5)$$

So, the goal is to minimize the cost function

$$F(x) = \sum_i e^{[i]}(x)^T \Omega^{[i]} e^{[i]}(x) \quad (13.6)$$

13.3 Gauss-Newton method

In the basic least squares formulation, a linear dependency between parameters and predicted observations is assumed; however, in most real-world problems, this relationship is non-linear. To solve these non-linear least squares problems, iterative numerical approaches are used, such as the **Gauss-Newton method**.

Gauss-Newton performs iterative minimization of the cost function $F(x)$. Each iteration refines the current state estimate x^* by computing a perturbation Δx and updating the estimate $x \leftarrow x^* + \Delta x$. This perturbation is obtained by minimizing a quadratic approximation of the problem around the current estimate x^* .

13.3.1 Linearization

The quadratic approximation is obtained by linearizing the error functions $e^{[i]}(x)$ around the current estimate x^* using a first-order Taylor expansion:

$$e^{[i]}(x^* + \Delta x) \simeq e^{[i]}(x^*) + \left. \frac{\partial e^{[i]}(x)}{\partial x} \right|_{x=x^*} \Delta x \quad (13.7)$$

Let

- $e^{[i]} = e^{[i]}(x^*)$ be the error vector evaluated at the current estimate,
- $J^{[i]} = \left. \frac{\partial e^{[i]}(x)}{\partial x} \right|_{x=x^*}$ be the Jacobian matrix of the error function with respect to the state, evaluated at x^* ,

the linearized error vector is then approximately:

$$e^{[i]}(x^* + \Delta x) \simeq e^{[i]} + J^{[i]} \Delta x \quad (13.8)$$

Substituting this into the scalar squared error term in equation ?? and expanding the product, we obtain a quadratic approximation for the contribution of the i -th measurement to the cost function around x^*

$$e^{[i]}(x^* + \Delta x)^T \Omega^{[i]} e^{[i]}(x^* + \Delta x) \simeq (e^{[i]} + J^{[i]} \Delta x)^T \Omega^{[i]} (e^{[i]} + J^{[i]} \Delta x) \quad (13.9)$$

Expanding this expression yields terms constant with respect to Δx , linear in Δx , and quadratic in Δx :

$$\simeq \Delta x^T J^{[i]T} \Omega^{[i]} J^{[i]} \Delta x + 2(J^{[i]T} \Omega^{[i]} e^{[i]})^T \Delta x + e^{[i]T} \Omega^{[i]} e^{[i]} \quad (13.10)$$

Grouping the terms for each measurement i by defining:

$$H^{[i]} = J^{[i]T} \Omega^{[i]} J^{[i]} \quad b^{[i]} = J^{[i]T} \Omega^{[i]} e^{[i]} \quad c^{[i]} = e^{[i]T} \Omega^{[i]} e^{[i]} \quad (13.11)$$

Summing over all measurements, the quadratic approximation of the global cost function is:

$$F(x^* + \Delta x) \simeq \Delta x^T H \Delta x + 2b^T \Delta x + c \quad (13.12)$$

With the aggregate matrices and vectors defined as:

$$H = \sum_i H^{[i]} \quad b = \sum_i b^{[i]} \quad c = \sum_i c^{[i]} \quad (13.13)$$

This function resembles a paraboloid.

13.3.2 Minimizing the quadratic approximation

To find the perturbation Δx that minimizes this quadratic approximation, we compute its derivative with respect to Δx and set it to zero²⁰. Assuming H is symmetric:

$$\frac{\partial}{\partial \Delta x} \Delta x^T H \Delta x + 2b^T \Delta x + c = 0 \implies 2H \Delta x + 2b = 0 \quad (13.14)$$

Solving this linear system for Δx provides the optimal perturbation in this iteration. This system can be solved by

$$H \Delta x = -b \implies \Delta x = H^{-1}(-b) \quad (13.15)$$

although in practice, matrix inversion is avoided for computational efficiency. A better approach to find the solution is least squares.

13.4 Least squares algorithm (one iteration)

A single iteration of the Gauss-Newton algorithm to compute the perturbation Δx is as follows:

1. initialize H and b to zero matrices/vectors of appropriate size²¹;
2. for each measurement i :

²⁰Classic solution for a minimization problem

²¹For H is dimension of the state $\times$ dimension of the state, for b is dimension of the state $\times 1$

- compute the error vector $e^{[i]} \leftarrow h^{[i]}(x^*) - z^{[i]}$ at the current state estimate x^* ;
 - compute the Jacobian $J^{[i]} \leftarrow \frac{\partial e^{[i]}(x)}{\partial x} \Big|_{x=x^*}$ evaluated at x^* ;
 - update $H \leftarrow H + J^{[i]T} \Omega^{[i]} J^{[i]}$;
 - update $b \leftarrow b + J^{[i]T} \Omega^{[i]} e^{[i]}$;
3. solve the linear system $H\Delta x = -b$ for Δx . H is typically symmetric and positive definite;
 4. update the state estimate $x^* \leftarrow x^* + \Delta x$.

This process is iterated until Δx is close to zero or a maximum number of iterations is reached. Convergence is not always guaranteed, especially with poor initial guesses or highly non-linear problems; however, if the error function is linear, the Jacobian is constant with respect to x , and the method converges in a single iteration.

13.4.1 Methodology

To apply the least squares approach to a problem, a clear methodology is needed:

1. identify the state space x : define the parameters or unknowns to be estimated. Choose a suitable parametrization, ideally a locally Euclidean one (so Euclidean but also manifold, *wink wink*);
2. identify the measurement space(s) z : define the observations obtained from sensors. Qualify their domain and potentially find locally Euclidean parametrizations;
3. identify the prediction functions $h(x)$: define the functions that map the state x to predicted measurements $h(x)$ that can be compared to the actual measurements z .

13.5 Gauss-Newton in SLAM

Gauss-Newton is applicable to various problems, particularly in robotics and state estimation, including:

- calibration;
- registration (Cloud to Cloud like **ICP**, Image to Cloud like Posit);
- global optimization (e.g., Pose-SLAM, Bundle Adjustment).

A key requirement for the basic Gauss-Newton method is that data associations are assumed to be known. Gauss-Newton alone is not sufficient to solve a full problem like SLAM; it requires a strategy to compute or estimate these data associations.

13.6 ICP optimization in 2D

Iterative Closest Point (ICP) is a standard algorithm for aligning point clouds: given a set of points in a world frame and a set of measurements in a robot (or sensor) frame, and roughly known correspondences between them, ICP aims to find a transformation that minimizes the distance between corresponding points. This transformation typically represents the pose of one frame relative to the other.

The ICP algorithm is iterative and the basic version consists of two main steps:

1. data association: for each point in one point cloud, find the closest corresponding point in the other point cloud (the nearest neighbour criterion using Euclidean distance is a common approach). Finding good data associations is crucial for successful alignment;
2. transformation estimation: given the estimated correspondences, compute the rigid body transformation (rotation and translation) that best aligns the corresponding points by minimizing the sum of squared distances between them. This step can be formulated as a least squares problem²².

These two steps are repeated iteratively: estimate correspondences based on the current alignment, then estimate the transformation based on the new correspondences, until convergence.

²²Using least squares for the transformation estimation step allows for a more general formulation compared to SVD-based methods used in simpler cases.

13.6.1 State and measurements

In a 2D ICP problem, the state x represents the rigid body transformation:

$$x = \begin{pmatrix} t_x \\ t_y \\ \theta \end{pmatrix} = \begin{pmatrix} t \\ \theta \end{pmatrix} \quad (13.16)$$

where $t = (t_x, t_y)^T$ is the 2D translation vector and θ is the rotation angle.

This state represents the pose of the world frame with respect to the robot frame (or vice versa, depending on the chosen formulation).

The measurements $z^{[i]}$ are the coordinates of corresponding points. Let $p^{[i]}$ be a point in the robot frame and $z^{[i]}$ its corresponding point in the world frame, the prediction function $h^{[i]}(x)$ maps the point $p^{[i]}$ from the robot frame to the world frame using the transformation parameters x :

$$h^{[i]}(x) = R(\theta)p^{[i]} + t \quad (13.17)$$

where $R(\theta)$ is the 2D rotation matrix for angle θ :

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \quad (13.18)$$

The 2-dimensional error vector $e^{[i]}(x)$ for the i -th correspondence is the difference between the transformed point $p^{[i]}$ and its corresponding point $z^{[i]}$ in the world frame:

$$e^{[i]}(x) = h^{[i]}(x) - z^{[i]} = R(\theta)p^{[i]} + t - z^{[i]} \quad (13.19)$$

13.6.2 Jacobian

To apply Gauss-Newton, we need to compute the Jacobian of the measurement function $h^{[i]}(x)$ with respect to the state. The Jacobian

$$J^{[i]} = \frac{\partial h^{[i]}(x)}{\partial x} = \begin{pmatrix} \frac{\partial h^{[i]}(x)}{\partial t_x} & \frac{\partial h^{[i]}(x)}{\partial t_y} & \frac{\partial h^{[i]}(x)}{\partial \theta} \end{pmatrix} \quad (13.20)$$

will have dimensions (dimension of measurement) $\times$ (dimension of state); in this 2D case, it is a 2×3 matrix. The partial derivative with respect to translation $t = (t_x, t_y)^T$ is the identity matrix:

$$\frac{\partial h^{[i]}(x)}{\partial t} = I \quad (13.21)$$

The partial derivative with respect to rotation θ involves the derivative of the rotation matrix $R(\theta)$ with respect to θ :

$$\frac{\partial h^{[i]}(x)}{\partial \theta} = \frac{\partial (R(\theta)p^{[i]} + t - z^{[i]})}{\partial \theta} = \frac{\partial R(\theta)}{\partial \theta} p^{[i]} \quad (13.22)$$

Let $R'(\theta) = \frac{\partial R(\theta)}{\partial \theta}$ be the derivative of the 2D rotation matrix: $R'(\theta) = \begin{pmatrix} -\sin \theta & -\cos \theta \\ \cos \theta & -\sin \theta \end{pmatrix}$. So, the Jacobian matrix is:

$$J^{[i]} = (I \quad R'(\theta)p^{[i]}) = \begin{pmatrix} 1 & 0 & (-p_x^{[i]} \sin \theta - p_y^{[i]} \cos \theta) \\ 0 & 1 & (p_x^{[i]} \cos \theta - p_y^{[i]} \sin \theta) \end{pmatrix} \quad (13.23)$$

where $p^{[i]} = (p_x^{[i]}, p_y^{[i]})^T$ are the coordinates of the point $p^{[i]}$ in the robot frame.

13.6.3 Variants and practical remarks

While the basic point-to-point metric is simple, normal-based distance metrics typically perform better. The point-to-plane metric minimizes the distance projected onto the normal of the corresponding point's surface. A symmetric point-to-plane metric improves upon this by using an average of the normals from both corresponding points. When building a registration technique, starting with the symmetric metric is recommended.

Dealing with outliers, which are measurements that are grossly wrong, is crucial for robust registration: robust least squares methods, often using robust kernels (M-estimators), can down-weight the impact of outliers on the error minimization. Informed outlier rejection, which uses background knowledge about the problem (e.g., static parts of the environment), can also improve results.

Several practical remarks are useful when implementing registration algorithms:

- always exploit an initial guess if available, as a good initial guess significantly improves the likelihood and speed of convergence for non-linear methods;
- normal-based distance metrics (point-to-plane, symmetric) generally outperform point-to-point alignment;
- combine informed outlier rejection with general methods like robust kernels. Adaptive kernels are powerful for adapting to the current outlier distribution;
- consider combining multiple registration techniques or using information from multiple sensors for increased robustness and accuracy;
- exploit dynamic constraints (e.g., vehicle velocity limits) to identify potential failure cases.

Incremental registration aligns consecutive scans and is useful for local estimation, but for long-term drift-free operation and global consistency, integration into a SLAM framework with global optimization is necessary. While ICP-based methods provide incremental corrections, the uncertainties estimated from these techniques may not be entirely trustworthy in practice.

More advanced applications of least squares registration include non-rigid registration, which deals with deformable objects like plants or humans. This requires more complex cost functions and sophisticated data association strategies, sometimes incorporating additional information like skeletal structures.

13.6.4 Solving the linear system

The most computationally demanding task in the Gauss-Newton iteration is solving the linear system $H\Delta x = -b$. While theoretically $\Delta x = -H^{-1}b$, explicitly computing the inverse H^{-1} is generally inefficient and numerically unstable for large systems. Instead, decomposition techniques like Cholesky factorization are preferred for symmetric positive-definite matrices like H . Cholesky factorization decomposes H into a product of a lower triangular matrix L and its transpose L^T , i.e., $H = LL^T$. The system $LL^T\Delta x = -b$ can then be solved efficiently by performing two back-substitutions: first solve $Ly = -b$ for y , and then solve $L^T\Delta x = y$ for Δx .

The structure of the Jacobian $J^{[i]}$ influences the structure of the matrix H . In SLAM, observations often relate only a small number of variables, leading to sparse Jacobians and consequently a sparse H matrix, which can be exploited for more efficient solving.

13.6.5 Testing: good and bad initial guess

Since ICP relies on least squares which depends on the initial guess of H and b , this guess will influence the outcome of the algorithm:

- when started from a good guess (close to the true location), the system converges nicely;
- if the guess is poor (for example the origin or away from the true location), the system might take long to converge or not reach the minimum.

The "basin of convergence," or the maximum initial misalignment from which the algorithm can still find the precise optimal parameters, is an important performance aspect of ICP variants. Standard ICP requires the initial point clouds (or correspondences) to be "close enough" to converge reliably.

14 Least squares and uncertainty

A significant advantage of the least squares formulation under the Gaussian assumption is the ability to derive an estimate of the uncertainty of the solution around the found optimum. This is done by relating the Gaussian distribution of the measurements to the Gaussian distribution of the state estimate using concepts like **chain rule** and **conditioning**.

Around the optimum x^* , the measurement function $h^{[i]}(x)$ can be approximated by its first-order Taylor expansion, as seen in the previous section.

If the measurements $z^{[i]}$ are affected by Gaussian noise $\mathcal{N}(0, \Sigma^{[i]})$, then the conditional probability of observing $z^{[i]}$ given the state $x = \Delta x + x^*$ is approximately Gaussian:

$$p(z^{[i]}|\Delta x + x^*) \sim \mathcal{N}(J^{[i]}\Delta x + z^{[i]*}, \Omega^{[i]-1}) \quad (14.1)$$

14.1 Consequences of Taylor

Assuming independence, the **conditional probability over all measurements** $z = (z^T, \dots, z^{[K]T})^T$ given the state **is a multivariate Gaussian**:

$$p(z|\Delta x + x^*) \sim \mathcal{N}(\mu_z, \Omega_z^{-1}) \quad (14.2)$$

The mean μ_z is the stacked vector of predicted measurements given the perturbation Δx :

$$\mu_z = \begin{pmatrix} J^{[1]}\Delta x + z^{[1]*} \\ J^{[2]}\Delta x + z^{[2]*} \\ \vdots \\ J^{[K]}\Delta x + z^{[K]*} \end{pmatrix} = \begin{pmatrix} J^{[1]} \\ J^{[2]} \\ \vdots \\ J^{[K]} \end{pmatrix} \Delta x + \begin{pmatrix} z^{[1]*} \\ z^{[2]*} \\ \vdots \\ z^{[K]*} \end{pmatrix} = J\Delta x + z^* \quad (14.3)$$

where J is the stacked Jacobian matrix and z^* is the stacked vector of predicted measurements at the optimum. The information matrix Ω_z for all measurements is a block diagonal matrix containing the individual measurement information matrices:

$$\Omega_z = \begin{pmatrix} \Omega^{[1]} & & \\ & \ddots & \\ & & \Omega^{[K]} \end{pmatrix} \quad (14.4)$$

14.2 Chain rule

Using the chain rule, we can combine the prior distribution over the state $p(\Delta x)$ and the conditional distribution $p(z|\Delta x + x^*)$ to find the joint distribution $p(\Delta x, z)$.

If we assume no prior knowledge about the state before incorporating measurements, the prior information matrix Ω_x is zero (corresponding to infinite covariance). At the optimum x^* , we are working with $\Delta x = 0$.

The joint distribution $p(\Delta x, z)$ is also Gaussian. The joint information matrix $\Omega_{x,z}$ of measurements and states has a specific structure involving J and Ω_z :

$$\Omega_{x,z} = \begin{pmatrix} J^T \Omega_z J & -J^T \Omega_z \\ -\Omega_z J & \Omega_z \end{pmatrix} \quad (14.5)$$

By marginalizing over z from the joint distribution, we can obtain the distribution over Δx . The block corresponding to Δx in the joint information matrix is

$$\Omega_{xx} = J^T \Omega_z J = \sum_i J^{[i]T} \Omega^{[i]} J^{[i]} \quad (14.6)$$

This sum is precisely the definition of the matrix H from the Gauss-Newton formulation:

$$\Omega_{xx} = H \quad (14.7)$$

14.3 Conditioning

Conditioning the joint Gaussian distribution $p(\Delta x, z)$ on the observed measurements z gives the posterior distribution $p(\Delta x|z)$, which is also Gaussian. The information matrix of this conditional distribution is

$$\Omega_{\Delta x|z} = \Omega_{xx} = H \quad (14.8)$$

The covariance matrix of the state estimate is the inverse of this information matrix:

$$\Sigma_{\Delta x|z} = \Omega_{\Delta x|z}^{-1} = H^{-1} \quad (14.9)$$

Thus, the inverse of the H matrix obtained from the final iteration of the Gauss-Newton algorithm provides an estimate of the covariance of the state estimate x^* , assuming the Gaussian noise model is accurate and the linearization holds well around the optimum. The more measurements incorporated (and the higher their information), the larger the resulting H matrix, leading to a smaller covariance and a higher degree of certainty in the estimate.

14.4 Conclusions

In summary, the Gauss-Newton method finds the mean of the Gaussian approximation of the solution x^* , and the inverse of the H matrix provides its covariance estimate. Minimizing the sum of squared errors under the assumption of independent Gaussian noise is equivalent to maximizing the log-likelihood of the observations given the state. This connection highlights how the least squares approach is fundamentally linked to probabilistic state estimation.

15 3D Point Registration using ICP

15.1 Example ICP optimization in 3D

Point cloud registration is a central task in perception and mapping applications: the goal is to find a local transformation, typically a rigid body transformation consisting of a translation vector and a rotation matrix, to align point clouds or scans recorded at different locations into the same reference frame. This is a key ingredient used in techniques like scan matching to build maps of the environment.

Consider an example ICP Optimization problem in 3D, given:

- a set of points in the world frame p_i ;
- a set of 3D measurements in the robot frame z_i
- a roughly known correspondences between points p_i and z_i (the problem of point cloud registration with **known** data association assumes that for each point in one set, its corresponding partner in the other set is known),

we want to find a transform that minimizes the distance between corresponding points, more formally, the task is defined as finding the translation vector t and rotation matrix R that minimize the sum of squared errors between corresponding points. Such a transform will provide the pose of the world with respect to the robot.

We could also estimate the robot pose with respect to the world, but this leads to longer calculations.

15.1.1 State and measurements

The state we want to estimate in this ICP optimization example is the pose of the robot (or the transformation between the two point clouds) and the measurements are the corresponding points in the robot frame and the

State

Measurements

$$x \in SE(3)$$

$$z \in \mathcal{R}^3$$

world frame

$$x = (x \quad y \quad z \quad \alpha_x \quad \alpha_y \quad \alpha_z)^T = (t \quad \alpha)^T$$

$$h^{[i]}(x) = R(\alpha)p^{[i]} + t$$

Pose of the robot

We have as many predictions as the number of points we have

As said before, we want to find a rotation matrix R and a translation vector t that transforms p_i to be as close as possible to z_i . The transformed point is given by $Rp_i + t$ and the error for a single correspondence i is defined as the difference between the transformed point and its corresponding point in the target frame. The goal is to minimize the sum of the squared errors over all corresponding points:

$$\sum_{i=1}^N \|(Rp_i + t) - z_i\|^2 \quad (15.1)$$

where N is the number of corresponding points and weights p_n can optionally be included in the sum for a weighted minimization.

15.1.2 On rotation matrices

A rotation in 3D is obtained by composing the rotations along $x - y - z$:

$$R(\alpha) = R_x(\alpha_x)R_y(\alpha_y)R_z(\alpha_z) \quad (15.2)$$

$$R_x = \begin{pmatrix} 1 & 0 & 0 \\ 0 & c & -s \\ 0 & s & c \end{pmatrix}$$

$$R_y = \begin{pmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{pmatrix}$$

$$R_z = \begin{pmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$R'_x = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -s & -c \\ 0 & c & -s \end{pmatrix}$$

$$R'_y = \begin{pmatrix} -s & 0 & c \\ 0 & 0 & 0 \\ -c & 0 & -s \end{pmatrix}$$

$$R'_z = \begin{pmatrix} -s & -c & 0 \\ c & -s & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

15.1.3 Jacobian

The Jacobian J_i is the derivative of the measurement vector $h^{[i]}(x)$ with respect to the state vector x :

$$J_i = \frac{\partial h^{[i]}(x)}{\partial x} = \begin{pmatrix} \frac{\partial h^{[i]}(x)}{\partial t} & \frac{\partial h^{[i]}(x)}{\partial \alpha_x} & \frac{\partial h^{[i]}(x)}{\partial \alpha_y} & \frac{\partial h^{[i]}(x)}{\partial \alpha_z} \end{pmatrix} \quad (15.3)$$

The Jacobian J is a 3x6 matrix:

- the first three columns correspond to the derivatives with respect to the translation vector $[t_x, t_y, t_z]$, and these form a 3x3 identity matrix:

$$J(1:3, 1:3) = \frac{\partial h^{[i]}(x)}{\partial t} = I \quad (15.4)$$

- the last three columns correspond to the derivatives with respect to the rotation angles and involves the derivatives of the rotation matrices:

$$J(1:3, 4) = \frac{\partial h^{[i]}(x)}{\partial \alpha_x} = R'_x R_y R_z p^{[i]} \quad J(1:3, 5) = \frac{\partial h^{[i]}(x)}{\partial \alpha_y} = R_x R'_y R_z p^{[i]} \quad J(1:3, 6) = \frac{\partial h^{[i]}(x)}{\partial \alpha_z} = R_x R_y R'_z p^{[i]} \quad (15.5)$$

15.1.4 Sum up

The ICP algorithm proceeds iteratively:

1. initialize the state estimate with a guess;
2. for a specified number of iterations initialize the Hessian matrix approximation H /²³ (a 6x6 matrix), the gradient vector b (a 6x1 vector) to zero and the total squared error chi to zero;
3. iterate through each point correspondence i between point sets p and z ;
4. for each correspondence, compute the error vector e_i and the Jacobian matrix J_i using the current state estimate x and the corresponding points $p_i \in p$ and $z_i \in z$;
5. accumulate H and b : $H+ = J_i^T J_i$ and $b+ = J_i^T e_i$;
6. accumulate the total squared error: $chi+ = e_i^T e_i$;
7. after iterating through all correspondences, solve the linear system $H\Delta x = -b$ for the state update Δx ;
8. update the state estimate: $x+ = \Delta x$;
9. repeat for the next iteration.

This iterative process aims to find the transformation parameters that minimize the sum of squared errors, effectively aligning the point clouds.

15.2 ICP linear relaxation

A question arises whether the registration problem can be approached without iteration, providing a direct solution. In the standard non-linear least squares formulation described earlier, the presence of rotation matrices in the prediction function means the Jacobian depends on the current state estimate (initial guess), necessitating an iterative solution.

One approach to attempt a non-iterative solution is **linear relaxation**: we relax the constraint that the transformation matrix must be a pure rotation matrix (part of a rigid body transform) and can estimate the 12²³ parameters of a general affine transformation that minimizes the error: $x^T = (r_1^T \ r_2^T \ r_3^T \ t^T)$:

$$h^{[i]}(x) = R p^{[i]} + t = \begin{pmatrix} r_1^T \\ r_2^T \\ r_3^T \end{pmatrix} p^{[i]} + t = M^{[i]} \begin{pmatrix} r_1 \\ r_2 \\ r_3 \end{pmatrix} + t \quad M^{[i]} = \begin{pmatrix} p^{[i]T} & & \\ & p^{[i]T} & \\ & & p^{[i]T} \end{pmatrix} \quad (15.6)$$

²³Since we have 9 dofs from the "rotation" matrix and 3 from the translation

If the initial transformation is assumed to be the identity and the weight matrix Ω is the identity, the least squares problem for this relaxed affine transformation can be reduced to a linear system. The Jacobian

$$\frac{\partial h^{[i]}(x)}{\partial x} = (M^{[i]}|I) \quad (15.7)$$

in this case becomes a 3×12 matrix that does not depend on the state, allowing for a solution potentially found in one iteration of least squares on the following system

$$H = \begin{pmatrix} \sum_i M^{[i]T} M^{[i]} & \sum_i M^{[i]T} \\ \sum_i M^{[i]} & \sum_i I \end{pmatrix} \quad b = \sum_i (M^{[i]}|I)^T (h^{[i]}(x) - z^{[i]}) \quad (15.8)$$

Applying one iteration of least squares with the relaxed affine model allows finding 12 parameters that minimize the error in this linearized sense; these 12 parameters define a linear transformation part (a 3×3 matrix, let's call it A) and a translation vector. The columns of the matrix A , denoted as r_1, r_2, r_3 , correspond to the transformed basis vectors, but they are generally not orthonormal, meaning A is not a valid rotation matrix.

To obtain a valid rigid body transformation, we need to find the rotation matrix that is "closer" to the estimated linear part A from the least squares solution. This can be achieved using **Singular Value Decomposition** (SVD).

Given the estimated linear part A (the 3×3 matrix from the 12 parameters), we perform SVD on A :

$$A = \begin{pmatrix} r_1^T \\ r_2^T \\ r_3^T \end{pmatrix} = U D V^T \quad (15.9)$$

where U and V are orthonormal matrices (rotation matrices if their determinant is +1) and D is a diagonal matrix containing the singular values. This decomposition is a standard mathematical procedure.

The rotation matrix R that is closest to A in a least squares sense is then given by:

$$R = U V^T \quad (15.10)$$

This step of computing the rotation matrix from the SVD of a cross-covariance matrix is also the core of the direct solution for point cloud registration with **KNOWN** data association.

Linear relaxation provides a reasonable solution under certain conditions:

- there are few wrong correspondences;
- the initial error is small.

If these assumptions are violated, the resulting affine transformation will not be an isometry (it will not preserve distances and angles), and it will use the additional degrees of freedom (beyond rigid body transformation) to reduce the error in a way that doesn't correspond to a valid physical alignment. One can check this condition by inspecting the values of the diagonal matrix D obtained from the SVD: if D is close to the identity matrix, A was close to a rotation matrix.

Generally, even when using linear relaxation, a few further iterations of non-linear least squares are usually beneficial to refine the solution and converge to the optimal rigid body transformation.

16 Robust estimators

16.1 Gauss-Newton optimization

Gauss-Newton (GN) is an iterative method to find the minimum of an objective function:

$$F(x) = \sum_i e^{[i]}(x) \quad (16.1)$$

where

$$e^{[i]}(x) = e^{[i]}(x)^T \Omega^{[i]} e^{[i]}(x) = \|e^{[i]}(x)\|_{\Omega^{[i]}}^2 \quad (16.2)$$

is called Omega L2 norm and it represents the squared norm of the a vector, modulated by a weight matrix Ω : $\|v\|_{\Omega}^2 = v^T \Omega v$.

16.1.1 L2 norm

The standard GN algorithm minimizes the sum of the squared L2 Omega norms of the errors. The L2 norm assigns to each term a cost that is quadratic in the magnitude of the error vector. The following linear transformation of a vector maps an omega norm as a plain squared norm:

$$\Omega = LL^T \quad \|v\|_{\Omega}^2 = v^T \Omega v = v^T LL^T v = v^T L v' = \|v'\|^2 \quad (16.3)$$

where $\|v'\|^2$ is the squared of the Euclidean norm

16.1.2 L1 norm

The L1 norm, defined as the magnitude of the vector and can be obtained by taking the square root of the L2 norm:

$$\|v\|_{\Omega} = \sqrt{v^T \Omega v} \quad (16.4)$$

It scales linearly with error magnitude, reducing the influence of outliers but violating Gaussian assumptions.

16.2 Outliers

In general, registration problems are characterized by heuristics to compute the association, which results to some associations that will not be correct and we cannot know a priori which ones are wrong. To deal with this issue there are two strategies:

- consensus-based methods (like RANSAC): select a subset of measurements that simultaneously agree with a solution;
- M-Estimators: assume the initial guess is good enough and consider “less” the large error terms.

16.2.1 M estimators

The goal is to minimize the objective function

$$F(x) = \sum_i \rho(\|e_i(x)\|_{\Omega}^2) = \sum_i \rho(u^{[i]}(x)) \quad (16.5)$$

Where $\rho(u)$ is a scalar monotonically increasing function that maps the L1 Omega norm of the error to a scalar. If we choose it as $\rho(u) = u^2$ we go back to GN.

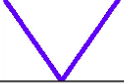
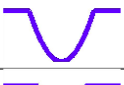
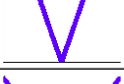
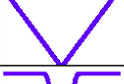
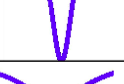
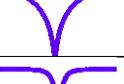
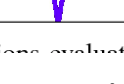
	$f(r)$	$f(r)$ graphical	$\omega(r)$
L^2	$\frac{1}{2}r^2$		1
L^1	$ r $		$\frac{1}{ r }$
L^2_{trunc}	$\begin{cases} \frac{1}{2}r^2 & \text{if } r \leq k \\ \frac{k^2}{2} & \text{otherwise} \end{cases}$		$\begin{cases} 1 & \text{if } r \leq k \\ 0 & \text{otherwise} \end{cases}$
L^1_{trunc}	$\begin{cases} r & \text{if } r \leq k \\ k & \text{otherwise} \end{cases}$		$\begin{cases} \frac{1}{ r } & \text{if } r \leq k \\ 0 & \text{otherwise} \end{cases}$
<i>Huber</i>	$\begin{cases} \frac{r^2}{2} & \text{if } x \leq k \\ k(r - k/2) & \text{otherwise} \end{cases}$		$\begin{cases} \frac{1}{ r } & \text{if } x \leq k \\ \frac{k}{ r } & \text{otherwise} \end{cases}$
<i>Tukey</i>	$\begin{cases} \frac{k^2}{6} \left(1 - \left(1 - \left(\frac{r}{k}\right)^2\right)^3\right) & \text{if } x \leq k \\ \frac{k^2}{6} & \text{otherwise} \end{cases}$		$\begin{cases} \left(1 - \left(\frac{r}{k}\right)^2\right)^2 & \text{if } r \leq k \\ 0 & \text{otherwise} \end{cases}$
<i>Cauchy</i>	$\frac{k^2}{2} \log\left(1 + \left(\frac{r}{k}\right)^2\right)$		$\frac{1}{1 + \left(\frac{r}{k}\right)^2}$
<i>Geman-McClure</i>	$\frac{r^2/2}{1 + r^2}$		$\frac{1}{(1 + r^2)^2}$

TABLE II: Summary of the robust functions evaluated in this paper

Figure 16.1: Common cost functions, $\rho = f$ and $u = r$

16.3 Robustifiers

The overall goal is minimization, which requires to set the derivatives of the function to 0. So let's do some derivatives (omitting the indices):

Derivative of the L2 norm (used to incorporate arbitrary robust cost function in GN)

$$\frac{\partial \|e(x)\|_{\Omega}^2}{\partial x} = 2e^T(x)\Omega \frac{\partial e(x)}{\partial x}$$

Derivative of the L1 norm (used to incorporate arbitrary robust estimators in GN)

$$\frac{\partial \sqrt{\|e(x)\|_{\Omega}^2}}{\partial x} = \frac{1}{2\|e(x)\|_{\Omega}} 2e^T(x)\Omega \frac{\partial e(x)}{\partial x} = \frac{1}{u(x)} \frac{\partial \|e(x)\|_{\Omega}^2}{\partial x}$$

We can use chain rule to express the robustified error, so the derivative of the error weighted by the cost function:

$$\begin{aligned} \frac{\partial \rho(u(x))}{\partial x} &= \frac{\partial \rho(u)}{\partial u} \Big|_{u=u(x)} \frac{1}{u(x)} \frac{\partial \|e(x)\|_{\Omega}^2}{\partial x} \\ &= \gamma(x) e^T(x) \Omega \frac{\partial e(x)}{\partial x} \end{aligned} \tag{16.6}$$

Notice how the derivatives of the

L2 norm

$$\frac{\partial \rho(u(x))}{\partial x} = \gamma(x) e^T(x) \Omega \frac{\partial e(x)}{\partial x}$$

robustified norm

$$\frac{\partial \|e(x)\|_{\Omega}^2}{\partial x} = 2e^T(x)\Omega \frac{\partial e(x)}{\partial x}$$

differ just by the scalar $\gamma(x)$, which can be absorbed as a scaling factor in the matrix Ω , which will change on each iteration depending on u .

16.3.1 Robust GN algorithm (one iteration)

The robust GN Algorithm is a modification of the GN algorithm to incorporate arbitrary robust cost functions (one of those in figure 18.1):

1. clear H and b

$$H \leftarrow 0$$

$$b \leftarrow 0$$

2. for each measurement:

(a) compute the error and it's Jacobian

$$e^{[i]} \leftarrow h^{[i]}(x^*) - z^{[i]}$$

$$J^{[i]} \leftarrow \frac{\partial e^{[i]}(x)}{\partial x} \Big|_{x=x^*}$$

(b) update the scaling based on the robustifier

$$u^{[i]} \leftarrow \sqrt{e^{[i]T} \Omega^{[i]} e^{[i]}}$$

$$\gamma^{[i]} \leftarrow \frac{1}{u^{[i]}} \frac{\partial p(u)}{\partial u} \Big|_{u=u^{[i]}}$$

(c) update the quadratic formula

$$H \leftarrow H + J^{[i]T} \gamma^{[i]} \Omega^{[i]} J^{[i]}$$

$$b \leftarrow b + J^{[i]T} \gamma^{[i]} \Omega^{[i]} e^{[i]}$$

(d) solve the linear system

$$\Delta x \leftarrow \text{solve}(H \Delta x = -b)$$

(e) apply the increments

$$x^* \leftarrow x^* + \Delta x$$

17 Least squares on manifolds

Point cloud registration is a fundamental problem in robotics and computer vision, involving the alignment of two or more point clouds or range scans, typically recorded at different poses or locations. This is a key ingredient in tasks like scan matching for map building and simultaneous localization and mapping (SLAM). The primary goal is to find a transformation, often a rigid body transformation (rotation and translation), that optimally aligns the point clouds so that corresponding points in the physical world overlap, iterative closest point (ICP) is a widely used standard method for this task. While the core ICP problem can be solved efficiently if data associations (correspondences) are known²⁴, the real challenge in practice lies in estimating these correspondences when they are unknown, which typically leads to an iterative solution. Furthermore, robotics problems often involve optimization in non-Euclidean spaces, necessitating specialized techniques like least squares on **manifolds**. Operations like simple vector subtraction or addition are not directly valid or meaningful in these spaces and to handle this, we often operate on a locally Euclidean parameterization, meaning that a manifold can be approximated as a Euclidean space in the neighborhood of a point.

17.1 Differences on a manifold

To compute a "difference" between two points on a manifold, say a starting point X_0 and an ending point X_1 , and represent this difference Δx in a locally Euclidean chart, we follow a specific procedure:

1. compute a chart around X_0 , essentially building a Euclidean mapping in the vicinity of X_0 ;
2. compute the location of X_1 within this chart;
3. measure the difference between the points in the chart using standard Euclidean subtraction;
4. in the chart, if X_0 and X_1 represent the same location, the difference Δx is zero;
5. this operation of computing the difference on the chart is formalized by the **boxminus operator** ($\boxminus$): $X_1 \boxminus X_0 = \Delta x$. This operator is crucial for applying standard Euclidean optimization techniques in the tangent space of the manifold. It's important to define the chart carefully to ensure the direction of the distance is consistent.

Conversely, if we have a starting point X_0 on the manifold and a difference Δx on a chart around X_0 , and we want to find the resulting endpoint X_1 on the manifold, we use the **boxplus operator** ($\boxplus$):

1. compute a chart around X_0 ;
2. move by Δx within this chart;
3. map the resulting point back onto the manifold to find X_1 ;
4. this operation is encapsulated as $X_1 = X_0 \boxplus \Delta x$. This means we apply the perturbation Δx to X_0 in its local Euclidean space and then map the result back to the manifold.

17.2 Least squares and Gauss in the manifold

Applying standard optimization methods like Gauss-Newton and Least Squares to problems formulated on manifolds requires adapting the standard linear algebra operations using the boxplus and boxminus operators.

17.2.1 Gauss in non-Euclidean spaces

In the context of non-Euclidean spaces, the error function $e^{[i]}(x)$ for a measurement i is defined as the difference between the prediction $h^{[i]}(x)$ and the measurement $z^{[i]}$, computed using the boxminus operator:

$$e^{[i]}(x) = h^{[i]}(x) \boxminus z^{[i]} \quad (17.1)$$

For optimization, we linearize the error function using a Taylor expansion around the current state estimate X^* and a small perturbation Δx on the chart:

$$e^{[i]}(X^* \boxplus \Delta x) \approx e^{[i]}(X^*) + \left. \frac{\partial e^{[i]}(X^* \boxplus \Delta x)}{\partial \Delta x} \right|_{\Delta x=0} \Delta x \quad (17.2)$$

Where

$$\left. \frac{\partial e^{[i]}(X^* \boxplus \Delta x)}{\partial \Delta x} \right|_{\Delta x=0} = J^{[i]} \quad (17.3)$$

²⁴The data association problem can be easily solved using SVD for known correspondence

is the Jacobian of the error function with respect to the perturbation, evaluated at the origin of the chart (i.e., around the current estimate X^*). The state estimate is updated by applying the computed perturbation using the boxplus operator: $X^* \leftarrow X^* \boxplus \Delta x$.

17.2.2 Least square

The iterative process for solving a least squares problem on a manifold is similar to the Euclidean case:

1. initialize the Hessian matrix H and the bias vector b to zero:

$$H \leftarrow 0 \quad b \leftarrow 0 \quad (17.4)$$

2. for each measurement i , compute its contribution to H and b , compute

$$\begin{array}{ll} \text{the error vector} & \text{the Jacobian} \\ e^{[i]} \leftarrow h^{[i]}(X^*) \boxminus Z^{[i]} & J^{[i]} \leftarrow \frac{\partial e^{[i]}(X^* \boxplus \Delta x)}{\partial \Delta x} \Big|_{\Delta x=0} \end{array} \quad (17.5)$$

and update H and b using the Jacobian and the information matrix $\Omega^{[i]}$ for the measurement:

$$H \leftarrow H + J^{[i]T} \Omega^{[i]} J^{[i]} \quad b \leftarrow b + J^{[i]T} \Omega^{[i]} e^{[i]} \quad (17.6)$$

3. compute the perturbation Δx by solving the linear system and apply it to the current state estimate X^* :

$$\Delta x \leftarrow \text{solve}(H \Delta x = -b) \quad X^* \leftarrow X^* \boxplus \Delta x \quad (17.7)$$

This iterative process is the core of manifold least squares optimization.

17.2.3 Methodology

To successfully apply manifold least squares to a problem, several key components must be identified and defined:

1. state space X :
 - define the domain of the state (e.g., SE(2) or SE(3) for poses);
 - define an Euclidean parameterization for the perturbation Δx on the chart. The Euclidean parameterization can be chosen based on the specific manifold. For rigid transformations, it typically involves components for translation and rotation perturbation;
 - define the boxplus operator ($\boxplus$) for this space.
2. measurement space(s) Z :
 - define the domain of the measurements (e.g., $\mathbb{R}^2$ or $\mathbb{R}^3$ for point coordinates);
 - define an Euclidean parameterization for perturbations in the measurement space;
 - define the boxminus operator ($\boxminus$) for this space. For Euclidean measurement spaces like $\mathbb{R}^n$, the boxminus is standard subtraction
3. prediction functions $h(X)$: define how to predict a measurement $z^{[i]}$ given the current state X and known data (e.g., a point in the world frame);
4. error functions $e(X)$: define how to compute the error between the prediction and the actual measurement using the defined boxminus operator.

17.3 Example: ICP optimization in 2D

Given two sets of 2D points with known or estimated correspondences (one set in a "world" frame, one in a "robot" frame), the goal is to find the 2D rigid body transformation (pose of the world relative to the robot) that minimizes the distances between corresponding points applying the manifold least squares methodology:

1. state space: the state X is a 2D rigid body transformation, living in $SE(2)$ and can be represented by a 3×3 homogeneous matrix:

$$X \in SE(2), \quad X = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \quad (17.8)$$

where $R \in SO(2)$ is the 2×2 rotation matrix and $t \in \mathbb{R}^2$ is the translation vector. A common Euclidean parameterization for the perturbation Δx on the chart is a 3-dimensional vector

$$\Delta x = (\Delta x \quad \Delta y \quad \Delta \theta)^T \quad (17.9)$$

representing small translations and rotation. A function $v2t(\Delta x)$ converts this perturbation vector to a transformation matrix representation

$$\Delta X = v2t(\Delta x) = \begin{pmatrix} \Delta R & \Delta t \\ 0 & 1 \end{pmatrix} \quad (17.10)$$

and the boxplus is

$$X \boxplus \Delta x = v2t(\Delta x)X = \Delta X \cdot X \quad (17.11)$$

2. measurement space: measurements are 2D points $z^{[i]} \in \mathbb{R}^2$. The prediction function for a point $p^{[i]}$ in the world frame, given the state X , is

$$h^{[i]}(X) = Rp^{[i]} + t = Xp^{[i]} \quad (17.12)$$

Since the measurement space is $\mathbb{R}^2$, the boxminus is standard subtraction:

$$e^{[i]} = h^{[i]}(X^*) - z^{[i]} \quad (17.13)$$

The prediction with a perturbed state is

$$h^{[i]}(X \boxplus \Delta x) = (X \boxplus \Delta x)p^{[i]} = v2t(\Delta x)Xp^{[i]} \quad (17.14)$$

Linearizing around $\Delta x = 0$ is approximately

$$h^{[i]}(X \boxplus \Delta x) = R(\Delta \theta)p'^{[i]} + \Delta t \quad (17.15)$$

where $p'^{[i]} = Xp^{[i]} = Rp^{[i]} + t$ is the point transformed by the current state X .

3. Jacobian computation: the Jacobian of the perturbed measurement is

$$h^{[i]}(X \boxplus \Delta x) = R(\Delta \theta)p'^{[i]} + \Delta t \quad \Delta x = (\Delta t_x \quad \Delta t_y \quad \Delta \theta)^T \quad (17.16)$$

evaluated at $\Delta x = 0$ is:

$$\left. \frac{\partial h^{[i]}(X \boxplus \Delta x)}{\partial \Delta x} \right|_{\Delta x=0} = \left(\left. \frac{\partial h^{[i]}(\cdot)}{\partial \Delta t} \right|_{\Delta x=0} \quad \left. \frac{\partial h^{[i]}(\cdot)}{\partial \Delta \theta} \right|_{\Delta x=0} \right) \quad (17.17)$$

$$\left. \frac{\partial h^{[i]}(\cdot)}{\partial \Delta t} \right|_{\Delta x=0} = I \quad \left. \frac{\partial h^{[i]}(\cdot)}{\partial \Delta \theta} \right|_{\Delta x=0} = I \quad (17.18)$$

The derivatives with respect to Δt_x and Δt_y give the identity matrix $I_{2 \times 2}$. The derivative with respect to $\Delta \theta$ involves the derivative of the 2D rotation matrix evaluated at $\Delta \theta = 0$, multiplied by $p'^{[i]}$. The resulting Jacobian is a 2×3 matrix (as tall as the output as wide as the input).

4. iterative update: the transformation estimate X^* is iteratively updated using the accumulated

$$H = \sum^{[i]} J^{[i]T} \Omega^{[i]} J^{[i]} \quad b = \sum^{[i]} J^{[i]T} \Omega^{[i]} e^{[i]} \quad (17.19)$$

solving $H\Delta x = -b$, and applying $X^* \leftarrow X^* \boxplus \Delta x$. $\Omega^{[i]}$ is the information matrix for the i -th correspondence.

17.4 Uncertainty of the solution

When performing optimization on a manifold using least squares, the resulting Hessian matrix H is computed based on the linearization in the local chart. The inverse of this Hessian, H^{-1} , provides a **covariance estimate for the solution** within this chart, specifically around the origin of the chart (which corresponds to the optimal state estimate X^*). This implies that the perturbation Δx around the optimum can be modeled as a Gaussian distribution:

$$\Delta x \sim \mathcal{N}(\Delta x; 0, H^{-1}) \quad (17.20)$$

To represent the uncertainty of the state X itself on the manifold, which is related to the perturbation Δx via the non-linear boxplus operation ($X = X^* \boxplus \Delta x$), two common methods are used to approximate the resulting non-Gaussian distribution

17.4.1 Linearization

In linearization, a first-order linearization of the boxplus function around $\Delta x = 0$ gives a Jacobian

$$J_X = \left. \frac{\partial X^* \boxplus \Delta x}{\partial \Delta x} \right|_{\Delta x=0} \quad (17.21)$$

The covariance of X can then be approximated by propagating the covariance from the chart: $\text{Cov}(X) \approx J_X H^{-1} J_X^T$.

The distribution of X is approximated as a Gaussian on the manifold centered at X^* with this covariance in the tangent space.

17.4.2 Unscented transform

This technique involves sampling a set of "sigma points" $\Delta x^{(i)}$ from the Gaussian distribution $\mathcal{N}(0, H^{-1})$ in the chart. These points are then propagated through the exact boxplus function

$$X^{(i)} = X^* \boxplus \Delta x^{(i)} \quad (17.22)$$

The resulting points $X^{(i)}$ are used to estimate the mean and covariance of the distribution of X on the manifold, providing a potentially more accurate approximation than linearization for highly non-linear systems.

17.4.3 Measurement uncertainty

Measurement uncertainty can also be modelled on a chart centered on the measurement mean $\mu_Z^{(i)}$. A perturbation $\Delta z^{(i)}$ on this chart is typically assumed Gaussian

$$\Delta z^{(i)} \sim \mathcal{N}(0, \Sigma_{\Delta z}^{(i)}) \quad (17.23)$$

The actual measurement $Z^{[i]}$ is related to its mean and the perturbation by

$$Z^{[i]} = \mu_Z^{(i)} \boxplus \Delta z^{(i)} \quad (17.24)$$

The error used in the least squares formulation is the difference between the prediction and the measurement mean, computed on the chart:

$$h^{[i]}(X) \boxminus \mu_Z^{(i)} \quad (17.25)$$

The distribution of this error is

$$e^{[i]}(X) \sim \mathcal{N}(h^{[i]}(X) \boxminus \mu_Z^{(i)}, \Sigma_{\Delta z}^{(i)}) \quad (17.26)$$

and the information matrix

$$\Omega^{[i]} = (\Sigma_{\Delta z}^{(i)})^{-1} \quad (17.27)$$

17.5 Manifold least squares (Omega on chart)

The general algorithm for manifold least squares, incorporating the information matrix $\Omega^{[i]}$ on the chart, is as follows:

1. initialize $H \leftarrow 0$ and $b \leftarrow 0$.

2. for each measurement i , compute

$$\begin{array}{ll}
 \text{the error vector} & \text{the Jacobian matrix} \\
 e^{[i]} \leftarrow h^{[i]}(X^*) \boxminus \mu_Z^{(i)} & J^{[i]} \leftarrow \left. \frac{\partial e^{[i]}(X^* \boxplus \Delta x)}{\partial \Delta x} \right|_{\Delta x=0}
 \end{array} \tag{17.28}$$

and update

$$\begin{array}{ll}
 H \leftarrow H + J^{[i]T} \Omega^{[i]} J^{[i]} & b \leftarrow b + J^{[i]T} \Omega^{[i]} e^{[i]}
 \end{array} \tag{17.29}$$

3. compute $\Delta x \leftarrow \text{solve}(H \Delta x = -b)$

4. apply the perturbation $X^* \leftarrow X^* \boxplus \Delta x$.

17.6 Conclusion

In conclusion, using least squares on smooth manifolds provides a more robust formulation for non-linear optimization problems, particularly when dealing with non-Euclidean spaces in robotics. The core principle is to linearize the problem within locally Euclidean charts using perturbations around the current optimum. The use of the boxplus and boxminus operators is essential for correctly handling operations on the manifold. This framework also provides methods for handling uncertainties through covariance propagation or the unscented transform. Compared to standard Euclidean approaches on non-Euclidean problems, manifold optimization often leads to improved convergence properties. It is emphasized that following the systematic methodology for defining state space, measurement space, prediction, and error functions is crucial.

18 ICP Optimization on a manifold

18.1 ICP 3D algorithm (one iteration)

One iteration of 3D ICP using the manifold least squares approach proceeds as follows:

1. initialize $H \leftarrow 0$ and $b \leftarrow 0$;
2. for each correspondence i between a point $p^{[i]}$ and a measurement $z^{[i]}$, compute

$$\begin{array}{ll} \text{the error vector} & \text{the Jacobian matrix} \\ e^{[i]} \leftarrow h^{[i]}(X^*) \boxminus Z^{[i]} & J^{[i]} \leftarrow \frac{\partial e^{[i]}(X^* \boxplus \Delta x)}{\partial \Delta x} \Big|_{\Delta x=0} \end{array} \quad (18.1)$$

Since 3D points are in $\mathbb{R}^3$, the measurement space is Euclidean, and the boxminus is standard subtraction: $e^{[i]} = h^{[i]}(X^*) - Z^{[i]}$; Update

$$H \leftarrow H + J^{[i]T} \Omega^{[i]} J^{[i]} \quad b \leftarrow b + J^{[i]T} \Omega^{[i]} e^{[i]} \quad (18.2)$$

where $\Omega^{[i]}$ is the information matrix for the correspondence;

3. compute and apply the perturbation Δx

$$\Delta x \leftarrow \text{solve}(H \Delta x = -b) \quad X^* \leftarrow X^* \boxplus \Delta x \quad (18.3)$$

18.2 Methodology

Applying the manifold least squares methodology to 3D ICP involves defining the following:

1. state space X : the state is the rigid body transformation, which lives in $\text{SE}(3)$. It's commonly represented by a 4×4 homogeneous matrix

$$X = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \quad (18.4)$$

where $R \in \text{SO}(3)$ and $t \in \mathbb{R}^3$. A standard Euclidean parameterization for the perturbation Δx is a 6D vector representing infinitesimal transformations (translations and rotations):

$$\Delta x = (\Delta t_x \quad \Delta t_y \quad \Delta t_z \quad \Delta \alpha_x \quad \Delta \alpha_y \quad \Delta \alpha_z)^T = \begin{pmatrix} \Delta t \\ \Delta \alpha \end{pmatrix} \quad (18.5)$$

The boxplus operator $\boxplus$ for $\text{SE}(3)$ is defined as

$$X \boxplus \Delta x = v2t(\Delta x)X = [R(\Delta \alpha)R]R(\Delta \alpha)t + \Delta t \quad (18.6)$$

2. measurement space(s) Z : measurements are 3D points in $\mathbb{R}^3$;
3. prediction functions $h(X)$: for a point $p^{[i]}$ in the world frame, the predicted location in the current sensor frame is

$$h^{[i]}(X \boxplus \Delta x) = (X \boxplus \Delta x)p^{[i]} = R(\Delta \alpha) [Rp^{[i]} + t] + \Delta t = R(\Delta \alpha)p'^{[i]} + \Delta t \quad (18.7)$$

4. error: the measurement space is Euclidean, so the boxminus is standard subtraction

$$e^{[i]}(X \boxplus \Delta x) = h^{[i]}(X \boxplus \Delta x) - z^{[i]} = R(\Delta \alpha)p'^{[i]} + \Delta t - z^{[i]} \quad (18.8)$$

5. Jacobian computation: the Jacobian of the error $e^{[i]}$ with respect to the perturbation Δx is needed for the least squares step. Linearizing around $\Delta x = 0$:

$$J^{[i]} = \frac{\partial e^{[i]}(X \boxplus \Delta x)}{\partial \Delta x} \Big|_{\Delta x=0} = \left[\frac{\partial e^{[i]}(\cdot)}{\partial \Delta t} \quad \frac{\partial e^{[i]}(\cdot)}{\partial \Delta \alpha} \right] \Big|_{\Delta x=0} \quad (18.9)$$

The derivative with respect to the translation perturbation Δt is the 3×3 identity matrix I_3 .

The derivative with respect to the rotation perturbation $\Delta \alpha$ involves the derivatives of the composed rotation matrix evaluated at $\Delta \alpha = 0$, multiplied by $p'^{[i]}$. This term evaluates to $-[p'^{[i]}]_{\times}$, where $[p'^{[i]}]_{\times}$ is the 3×3 skew-symmetric matrix representing the cross product with $p'^{[i]}$. The full Jacobian matrix is a 3×6 matrix:

$$\begin{aligned} J^{[i]} &= \left(\frac{\partial \Delta t}{\partial \Delta t} \frac{\partial R(\Delta \alpha)p'^{[i]}}{\partial \Delta \alpha} \right) \Big|_{\Delta x=0} = \left(\frac{\partial \Delta t}{\partial \Delta t} \frac{\partial R(\Delta \alpha_x)p'^{[i]}}{\partial \Delta \alpha_x} \frac{\partial R(\Delta \alpha_y)p'^{[i]}}{\partial \Delta \alpha_y} \frac{\partial R(\Delta \alpha_z)p'^{[i]}}{\partial \Delta \alpha_z} \right) \Big|_{\Delta x=0} \\ &= \left(I - [p'^{[i]}]_{\times} \right) \end{aligned} \quad (18.10)$$

The estimated transformation X^* is updated iteratively by solving the normal equations using the accumulated

$$H = \sum^{[i]} J^{[i]T} \Omega^{[i]} J^{[i]} \quad b = \sum^{[i]} J^{[i]T} \Omega^{[i]} e^{[i]} \quad (18.11)$$

and applying the perturbation Δx using the SE(3) boxplus.

18.3 Testing

Testing MICP typically involves creating synthetic data:

1. generate a set of random 3D points;
2. define a known "ground truth" robot pose;
3. compute synthetic measurements by transforming the generated points by the ground truth pose and adding realistic noise;
4. provide an initial guess for the robot pose, usually the identity transformation (origin) or a small perturbation from the ground truth;
5. run the MICP algorithm and monitor its convergence by plotting the evolution of the total error or the number of estimated inliers over iterations.

Comparisons indicate that MICP often converges faster than standard ICP methods that do not use manifold optimization or potentially point-to-point metrics.

18.4 Outliers

A significant challenge in ICP, especially with unknown data association, is the presence of **outliers**: false correspondences that arise, for example, from measurement noise, dynamic objects, or lack of overlap between scans. If a large percentage of correspondences are outliers, the standard least squares solution can be heavily biased, potentially leading to divergence or a poor alignment. Points with correct associations are called **inliers** and the goal is to reduce the impact of outliers on the optimization.

One common technique to mitigate the effect of outliers is using **robust kernels** in the least squares formulation. Robust kernels (or M-estimators) modify the quadratic cost function of standard least squares to reduce the influence of measurements with large errors, which are likely outliers; instead of minimizing the squared error, they minimize a different function of the error. This effectively down-weights the contribution of outlier measurements to the computation of H and b . Examples include the Huber kernel or adaptive kernels that dynamically adjust their shape based on the observed error distribution.

Outlier rejection, possibly combined with robust kernels, is often necessary for robust ICP in real-world scenarios. Simple outlier rejection can be based on distance thresholds, while more advanced methods use compatibility checks, trimmed ICP, or machine learning for segmentation.

18.4.1 Behavior with outliers

When dealing with outliers, monitoring the total sum of squared errors might not be the best indicator of the algorithm's progress or success. Instead, tracking the **number of estimated inliers** as the algorithm iterates provides a better insight into how well the data association and pose estimation are converging: as the transformation estimate improves, the distances between true corresponding points should decrease, allowing the data association step to identify more inliers and fewer outliers. A system's robustness can be evaluated by how closely the estimated fraction of inliers matches the true fraction under varying levels of injected outliers.

18.5 Conclusions

Utilizing manifolds in ICP optimization offers advantages, primarily leading to improved convergence characteristics compared to methods that ignore the non-Euclidean nature of the pose space; while the derivation might seem complex, following the systematic methodology helps. Handling outliers is paramount for robustness in practical applications: robust kernels can effectively down-weight outliers, though they might slightly reduce convergence speed or the basin of attraction. However, the benefit of robustness often outweighs these minor drawbacks in real-world scenarios.

Modern ICP variants, particularly for 3D registration, often incorporate several advanced techniques beyond the basic iterative closest point approach:

- using a least squares framework on a manifold formulation;

- employing normal-based metrics like the point-to-plane metric, which tend to provide better convergence properties than point-to-point metrics, especially for structured surfaces. The symmetric point-to-plane metric is a popular choice;
- implementing sophisticated data association strategies, often leveraging initial guesses or additional information like normals, colour, or features to find compatible points. Exploiting any available initial guess is always strongly recommended;
- incorporating robust outlier rejection mechanisms, ranging from simple distance checks to robust kernels and techniques for identifying and discarding dynamic objects. Adaptive robust kernels that tune themselves to the current outlier distribution are particularly powerful.

These elements, combined within an iterative optimization framework, form the basis of robust point cloud registration algorithms used in modern SLAM systems and other applications.

19 Registration on a manifold

The input for registration problems often consists of:

- a set of prediction, typically in the world frame $p^{[i]}$;
- a set of measurements obtained from a sensor, typically expressed in the coordinate frame of the robot or platform carrying the sensor $z^{[i]}$;
- a roughly known correspondence between the two sets.

As the robot moves through the environment, it records these measurements from different locations.

The primary task of registration is then to find the transformation required to align these individual sets of points, recorded at different times or locations, into a single, consistent reference frame. This common reference frame could be a global map coordinate system or the coordinate system of a preceding sensor reading.

19.1 Similarity registration in 3D

Beyond simple rigid body transformations ($SE(3)$), registration can involve finding a similarity transformation **Sim(3)**, which includes a **uniform scale factor** s in addition to a rotation matrix R and a translation vector t . This is a more general class of transformations where the scale is not necessarily fixed to one. The problem of finding a similarity transformation between two 3D point sets is often referred to as the absolute orientation problem, and rigid body transformation is a special case of it. **Similarity registration** is relevant when the relationship between the coordinate systems of the two point clouds **involves a scaling factor**, such as in certain camera-based 3D reconstruction scenarios.

The objective remains to find the transformation parameters that best align corresponding points from two sets; with known correspondences and optional weights p_n , the goal is to minimize the sum of squared errors:

$$\min_{R,t,s} \sum_n p_n \|y_n - (sRx_n + t)\|^2 \quad (19.1)$$

where y_n and x_n are corresponding points, and s is the scalar scale factor. This optimization problem is more complex than that for rigid body registration due to the inclusion of the scale parameter.

19.2 Similarity iterative cloud point (SICP)

19.2.1 State and boxplus

For Similarity ICP (SICP), the state is represented by a $Sim(3)$ matrix:

$$X = \begin{pmatrix} R & t \\ 0 & s \end{pmatrix} \in Sim3 \quad (19.2)$$

The state update uses a 7-dimensional perturbation vector Δx applied via the boxplus operator. The perturbation vector can be written as

$$\Delta x = (\Delta x \quad \Delta y \quad \Delta z \quad \Delta \alpha_x \quad \Delta \alpha_y \quad \Delta \alpha_z; \Delta s)^T = (\Delta t \quad \Delta \alpha; \log s)^T \quad (19.3)$$

where

- Δt is the translational increment (3D),
- $\Delta \alpha$ is the rotation increment (represented as a 3D vector),
- Δs is the scalar scale increment in log space.

The mapping from the perturbation vector to a $Sim(3)$ matrix is given by $v2s(\Delta x)$:

$$v2s(\Delta x) = \begin{pmatrix} R(\Delta \alpha) & \Delta t \\ 0 & \exp(\Delta s) \end{pmatrix} \quad (19.4)$$

Some notes:

- the rotation matrix $R(\Delta \alpha)$ is the exponential map of the skew-symmetric matrix of the rotation vector $\Delta \alpha$;
- the scale is represented as $\exp(\Delta s)$ because the exponential function is always positive, which is a useful representation for a parameter that must be positive.

The boxplus update is the composition of the perturbation transformation with the current state:

$$X \boxplus \Delta x = v2s(\Delta x)X \quad (19.5)$$

19.2.2 Operators

The operators for Sim(3) define how transformations are applied to points and how they are composed.

Applying a similarity transformation X (??) to a 3D point p :

$$X \cdot p = sRp + t \quad (19.6)$$

This operation involves first rotating the point by R , then scaling the result by s , and finally translating by t .

Composing two similarity transformations

$$\begin{aligned} X_1 &= \begin{pmatrix} R_1 & t_1 \\ 0 & s_1 \end{pmatrix} & X_2 &= \begin{pmatrix} R_2 & t_2 \\ 0 & s_2 \end{pmatrix} \\ X_1 X_2 &= \begin{pmatrix} R_1 R_2 & R_1 t_2 + s_2 t_1 \\ 0 & s_1 s_2 \end{pmatrix} \end{aligned} \quad (19.7)$$

The composition of rotations is $R_1 R_2$, and the composition of scales is the product $s_1 s_2$. The translation composition $R_1 t_2 + s_2 t_1$ is described as the rotation of the second translation by the first rotation, plus the scaled translation of the first transform.

19.2.3 Measurements

In SICP, the measurements $z^{[i]}$ are typically 3D points in $\mathcal{R}^3$. Given a prediction point $p^{[i]}$ from one point cloud and the current state (similarity transformation) ??, the predicted measurement $h^{[i]}(X)$ is the point $p^{[i]}$ transformed by X and is calculated as:

$$h^{[i]}(X) = s(Rp^{[i]} + t) \quad (19.8)$$

When considering the state updated by a perturbation Δx , the predicted measurement $h^{[i]}(X \boxplus \Delta x)$ involves transforming the original point $p^{[i]}$ by the updated state $X \boxplus \Delta x$:

$$h^{[i]}(X \boxplus \Delta x) = (X \boxplus \Delta x) = \exp(\Delta s)(R(\Delta \alpha)sRp^{[i]} + t) + \Delta t = \exp(\Delta s)(R(\Delta \alpha)p'^{[i]}) + \Delta t \quad (19.9)$$

This formula indicates that the point $p^{[i]}$ is first transformed by $sRp^{[i]} + t$, then rotated by $R(\Delta \alpha)$, scaled by $\exp(\Delta s)$, and finally translated by Δt .

19.2.4 Error and Jacobian

The error function for the i -th correspondence in SICP is the difference between the predicted measurement $h^{[i]}(X \boxplus \Delta x)$ and the actual measurement $z^{[i]}$:

$$e^{[i]}(X \boxplus \Delta x) = h^{[i]}(X \boxplus \Delta x) - z^{[i]} \quad (19.10)$$

To perform iterative minimization (e.g., using Gauss-Newton) of the sum of squared errors $\sum_i \|e^{[i]}(X \boxplus \Delta x)\|^2$, the error function needs to be linearized around the current state, which requires computing the Jacobian of the error function with respect to the perturbation vector Δx , evaluated at $\Delta x = 0$. The Jacobian for the i -th correspondence is denoted

$$\begin{aligned} J^{[i]} &= \left. \frac{\partial e^{[i]}(X \boxplus \Delta x)}{\partial \Delta x} \right|_{\Delta x=0} = \left(\frac{\partial e^{[i]}(\cdot)}{\partial \Delta t} \frac{\partial e^{[i]}(\cdot)}{\partial \Delta \alpha} \frac{\partial e^{[i]}(\cdot)}{\partial \Delta s} \right) \Big|_{\Delta x=0} \\ &= \left(I - [p'^{[i]}]_{\times} p'^{[i]} \right) \end{aligned} \quad (19.11)$$

The Jacobian with respect to the 7-dimensional perturbation vector Δx evaluated at the current state X is a 3×7 matrix.

Here

- I is the 3x3 identity matrix,
- $[\cdot]_{\times}$ is the skew-symmetric matrix operator,
- s, R, t are the parameters of the current state X used to transform point $p^{[i]}$.

The dimensions reflect the 3D nature of the error vector and the 7 parameters ($\Delta t \in \mathcal{R}$, $\Delta \alpha \in \mathcal{R}^3$, $\Delta s \in \mathcal{R}$) in the perturbation vector for Sim(3).

19.3 Exercise

Model: N points in the 3D space. Measurements: direction vectors of these points expressed in the reference frame of the sensor (the sensor only measure the direction and not the dimension of the vector)

- $Z \in S^2$ (all points lie on a sphere),
- $h^{[i]}(X) = \frac{Rp^{[i]} + t}{\|Rp^{[i]} + t\|}$ (Normalization of the vector which guarantees us that the measurement are of unitary norm).

Hint: map S^2 in $\mathcal{R}^3$ (overparametrize). The easier choice is the unit vector, so $z = (x \ y \ z)^T$ such that $\|z\| = 1$. Solution of the problem: Step 1: define the state

$$X = \begin{bmatrix} R & | & t \end{bmatrix} \in SE(3) \quad \begin{matrix} \Delta x = \\ (\Delta x \ \Delta y \ \Delta z \ \Delta \alpha_x \ \Delta \alpha_y \ \Delta \alpha_z)^T = \\ (\Delta t \ \Delta \alpha) \end{matrix}$$

$$X \boxplus \Delta x = v2t(\Delta x)X = \begin{bmatrix} R(\Delta \alpha)R & | & R(\Delta \alpha)t + \Delta t \end{bmatrix} \quad (19.12)$$

Step 2: define the measurement space (we are on a manifold)

$$Z = (x \ y \ z)^T \in S^2 \quad Z_1 \boxminus Z_2 = \Delta z = (\theta, \phi)^{25}$$

Calculating distances on a sphere is complex since we'd need to perform a long, non-intuitive rotation (since the distance on the manifold is shorter), possible but why do it when there is a simpler solution? We can simplify the problem (loosing the benefits of operating on a geodesic distance), by treating Z as if it was Euclidean: this replaces the distance along the geodesic (distance on the manifold) with the chordal distance (Euclidean).

Step 3: define the error (keep in mind that we are considering a simplified approach where we are not anymore on the manifold but considering a Euclidean distance)

$$e^{[i]}(X) = h^{[i]}(X) - Z^{[i]} \quad h^{[i]}(X) = \text{normalize}(Rp^{[i]} + t) = \text{normalize}(p'^{[i]})^{26}$$

$$h^{[i]}(X \boxplus \Delta x) = \text{normalize}(\Delta Rp'^{[i]} + \Delta t) = \text{normalize}(h_{icp})^{27}$$

$$\text{normalize}(v) = \frac{v}{\sqrt{v^T v}} \quad \frac{\partial \text{normalize}(v)}{\partial v} = \frac{I\|v\| - vv^T/\|v\|}{\|v\|^2} = \frac{1}{\|v\|}I - \frac{1}{\|v\|^3}vv^T$$

Step 4: JACOBIANS

$$\frac{\partial e^{[i]}(X \boxplus \Delta x)}{\partial \Delta x} \Big|_{\Delta x=0} = \frac{\partial \text{normalize}(p)}{\partial p} \Big|_{p=p'^{[i]}} \frac{\partial h_{icp}^{[i]}(X \boxplus \Delta x)}{\partial \Delta x} \Big|_{\Delta x=0} = \frac{1}{\|p'^{[i]}\|^3} \left(I\|p'^{[i]}\|^2 - p'^{[i]}p'^{[i]T} \right) \left(I[-p'^{[i]}]_{\times} \right)$$

²⁷The dimension is still 3×3 , so we are coherent

20 Mini recap of epipolar geometry on unit sphere

Epipolar geometry describes the intrinsic projective geometry between two views. It captures the geometric relationship between multiple cameras, a 3D point, and its projections in each image plane. This geometry is independent of scene structure and relies only on camera internal parameters and relative pose. Having multiple views is crucial because recovering the entire 3D structure from a single image is generally not possible due to the ambiguity of the 3D to 2D mapping.

The standard setup involves two cameras observing the same 3D point P , with projections p and p' in their respective image planes and centers at C and C' (or O_1 and O_2); the centers and the line connecting them is the baseline. The plane defined by the two camera centers and the 3D point P is called the epipolar plane.

20.1 Lines

A line in 3D space can be described as

$$l_0 = p_0 + d_0 s_0 \quad (20.1)$$

where

- p_0 is an offset point,
- d_0 is the direction vector,
- s_0 is a scalar parameter that spans points along the line.

When a line is transformed by a rigid motion $T = [R \ t]$ (rotation R and translation t), the transformed line l_1 is given by

$$l_1 = Tl_0 = Rp_0 + t + Rd_0 s_0 = p_1 + d_1 s_0 \quad (20.2)$$

20.1.1 Lines, closest point

Lines in space do not always intersect; to find the closest point between two lines,

$$l_0 : p_0, d_0 \quad l_1 : p_1, d_1 \quad (20.3)$$

we seek the values of s_0 and s_1 for which the distance between points on the lines is minimized. The difference vector is

$$\Delta(s_0, s_1) = p_0 + d_0 s_0 - p_1 - d_1 s_1 = (p_0 - p_1) + (d_0 \mid -d_1) \begin{pmatrix} s_0 \\ s_1 \end{pmatrix} = \Delta p + \Delta D s \quad (20.4)$$

The optimal s is found by minimizing the squared norm:

$$s^* = \operatorname{argmin}_s \|\Delta p + \Delta D s\|^2 = -(\Delta D^T \Delta D)^{-1} \Delta D^T \Delta p \quad (20.5)$$

ΔD has dimension 3×2

20.1.2 Line intersection

Two lines intersect if the projection of the difference vector between any two points on the lines (e.g., p_0 and p_1) onto the vector orthogonal to both lines ($d_0 \times d_1$) is zero. The intersection condition is

$$0 = (p_0 - p_1) \cdot (d_0 \times d_1) \quad (20.6)$$

20.1.3 Line intersection (camera-specific)

Consider two lines (rays) originating from camera centers. Let

- l_0 be a ray from camera C_0 (at origin $p_0 = 0$) with direction d_0 ,
- l_1 be a ray from camera C_1 (at origin $p_1 = 0$) with direction d_1 in C_1 's reference frame.

We can express l_1 in C_0 's frame using transformation $T = [R, t]$ as

$$l'_1 : t, Rd_1 \quad (20.7)$$

The condition for these two rays to intersect (i.e., meet at a 3D point P) is the coplanarity of the two camera centers C_0, C_1 and the point P . This coplanarity is captured by the **epipolar constraint**. Using the intersection condition

$$0 = (p_0 - (t + Rd_1 s_1)) \cdot (d_0 \times Rd_1) \quad (20.8)$$

evaluated at $s_1 = 0$ for point t , and considering the rays pass through the origins ($p_0 = 0, p_1 = 0$ in their frames), the vectors involved are d_0 , Rd_1 , and t ; these three vectors must be coplanar for the rays to intersect at a 3D point. The coplanarity condition

$$(t) \cdot (Rd_1 \times d_0) = 0 \quad (20.9)$$

Using vector identities, the Epipolar constraint is determined as:

- intersection condition: $0 = t \cdot (d_0 \times (Rd_1))$,
- cross product identity: $0 = d_0 \cdot (t \times (Rd_1))$,
- cross product to skew-symmetric matrix multiplication: $0 = d_0[t]_{\times} Rd_1$,
- transpose and negate: $0 = d_1^T (R^T[t]_{\times}) d_0$,
- epipolar constraint: $0 = d_1 E d_0$.

The matrix $E = [t]_{\times} R$ is the **essential Matrix**. The epipolar constraint in terms of normalized coordinates (camera frame coordinates) x, x' is

$$x'^T E x = 0 \quad (20.10)$$

20.2 Essential matrix

The Essential matrix E is a 3×3 matrix encoding the relative pose (rotation R and translation t) between two cameras with known internal calibrations. It allows expression of the epipolar constraint in a compact form

$$l' = E x \quad x'^T E x = 0 \quad (20.11)$$

Properties of the essential matrix $E = [t]_{\times} R$:

- satisfies the epipolar constraint for corresponding normalized points x, x' ;
- maps a normalized point x in the first image to its corresponding epipolar line l' in the second image: $l' = E x$. Similarly, $l = E^T x'$ is the epipolar line in the first image for point x' ;
- the epipole e' in the second image is the left null-vector of E , satisfying $e'^T E = 0$,
- the epipole e in the first image is the right null-vector of E , satisfying $E e = 0$,
- the epipoles are the images of the camera centers in the other view,
- E has 5 degrees of freedom (DOF): rotation R has 3 DOF, translation t has 3 DOF, but there is an overall scale ambiguity as it's a homogeneous matrix, removing one DOF;
- E has rank 2. This is because $[t]_{\times}$ has rank 2 and R has rank 3 (invertible);
- E has two equal non-zero singular values, and the third singular value is zero. This is a defining property.

The Essential Matrix operates on points expressed in camera coordinate systems (normalized coordinates).

20.3 Fundamental matrix

The fundamental matrix F is a 3×3 matrix that plays a similar role to the essential matrix but operates on image points in pixel coordinates and encodes camera intrinsic parameters as well. It is defined by the relationship

$$F = K'^{-T} E K^{-1} \quad F = K'^{-T} [t]_{\times} R K^{-1} \quad (20.12)$$

where K and K' are the intrinsic calibration matrices of the two cameras and E is the essential matrix.

Properties of the Fundamental Matrix F :

- it is the algebraic representation of epipolar geometry for uncalibrated cameras;
- satisfies the epipolar constraint $x'^T F x = 0$ for any pair of corresponding points $x \leftrightarrow x'$ in the two images (pixel coordinates). This condition allows computation of F from corresponding points alone, without camera parameters;
- maps a point x in the first image to its corresponding epipolar line l' in the second image: $l' = Fx$. Similarly, $l = F^T x'$ gives the epipolar line in the first image for point x' ;
- the epipole e' in the second image is the left null-vector of F , satisfying $e'^T F = 0$. The epipole e in the first image is the right null-vector of F , satisfying $F e = 0$;
- F has 7 degrees of freedom (DOF). It's a 3×3 homogeneous matrix (8 DOF) with the constraint $\det(F) = 0$, reducing DOF by one;
- F has rank 2. This property is crucial.
- unlike the essential matrix, F generally has two different non-zero singular values, and the third is zero.



Figure 20.1: Fundamental matrix song

The fundamental matrix operates on points in the pixel coordinate system.

20.4 Essential matrix vs vs fundamental matrix vs homography

Both Essential and Fundamental matrices are 3×3 matrices.

Essential	Fundamental	Homography
maps a point (normalized coordinates) to a line	maps a point (pixel coordinates) to a line	maps a point to a point
$l' = Ex$	$l' = Fx$	$x' = Hx$
$\text{rank}(E) = 2$	$\text{rank}(F) = 2$	$\text{rank}(H) = 3$
5 dof	7 dof	8 dof

Homography is a special case when the scene is planar.

20.5 Extracting R and t from E

Since E contains both R and t , we can extract them by decomposing E in the product of a rotation and a skew symmetric matrix:

1. compute SVD: $E = R^T [t]_{\times} = U S V^T$ s.t. $S = \begin{pmatrix} \sigma_1 & & \\ & \sigma_2 & \\ & & \sigma_3 \end{pmatrix}$ with $\sigma_1 \geq \sigma_2 \geq \sigma_3$, the rank 2 constraint is enforced by setting $\sigma_3 = 0$,
2. define a rotation matrix that's mapping the x on the y , the y on $-x$ and z on itself: $W = \begin{pmatrix} & & 1 \\ -1 & & \\ & & 1 \end{pmatrix}$,
3. use the re-arranging matrix: $WS = \begin{pmatrix} & \sigma_2 & \\ -\sigma_1 & & \\ & & 0 \end{pmatrix} \implies WSW^T = S \implies S = UWSW^TV^T$,
4. extract R and t by identification: $R = UWV^T$ and $[t]_{\times} = VSW^TV^T$,
5. out of the four solutions (for $t, -t, W$ and W^T), choose one by triangulating the points and pick the one with most points in front of camera

The equal singular value constraint for E is handled by replacing σ_1, σ_2 with their average $(\sigma_1 + \sigma_2)/2$ (or typically setting to 1 after normalizing E such that $\sigma_1^2 + \sigma_2^2 + \sigma_3^2 = 2$, resulting in $\Sigma = \text{diag}(1, 1, 0)$ for the ideal case).

20.6 Estimating E (8 point)

Given a set (at least 8 pairs) of point correspondences between two images, we can estimate the essential matrix (if cameras are calibrated) or the fundamental matrix (if cameras are uncalibrated or weakly calibrated) using the eight-point algorithm.

Each correspondence $d_i = (x_i, y_i, z_i)^T$ and $d'_i = (x'_i, y'_i, z'_i)^T$ satisfies the epipolar constraint $d_i'^T E d_i = 0$.

$$E = \begin{pmatrix} e_{11} & e_{12} & e_{13} \\ e_{21} & e_{22} & e_{23} \\ e_{31} & e_{32} & e_{33} \end{pmatrix} \quad e = \begin{pmatrix} e_{11} \\ e_{12} \\ \vdots \\ e_{33} \end{pmatrix} \quad (20.13)$$

From the epipolar constraint, we know that $d'^T E d = 0$, to avoid the trivial solution $E = 0$ we enforce $\|e\| = 1$ and, given $N \geq 8$ correspondences, we form a linear system $Ae = 0$, where A is an $N \times 9$ matrix constructed from the point coordinates, and e is the 9×1 vector of e 's elements (vectorized matrix).

$$d'^T E d = \begin{pmatrix} x' & x' & x' & y' & y' & y' & z' & z' & z' \end{pmatrix} \begin{pmatrix} e_{11} \\ e_{12} \\ \vdots \\ e_{33} \end{pmatrix} = Ae \quad (20.14)$$

we want to solve

$$e^* = \arg \min_e \sum_i e^T A^{[i]T} A^{[i]} e = \arg \min_e e^T H e \quad (20.15)$$

such that $e^T e = 1$.

This is a constrained minimization problem solvable using **Lagrange multipliers**:

$$L(e, \lambda) = e^T H e - \lambda(e^T e - 1) \quad (20.16)$$

Setting the derivative with respect to e to zero gives

$$\frac{\partial \mathcal{L}(e, \lambda)}{\partial e} = 2He - 2\lambda e = 0 \implies \frac{\partial \mathcal{L}(e, \lambda)}{\partial \lambda} = e^T e - 1 = 0 \quad (20.17)$$

This is an eigenvalue problem and the solution e is an eigenvector of H , and λ is the corresponding eigenvalue. The cost is proportional to the eigenvalue (since $e^T e = 1$)

$$e^T H e = e^T (\lambda e) = \lambda(e^T e) = \lambda \quad (20.18)$$

To minimize the cost, we choose the eigenvector corresponding to the smallest eigenvalue of $H = A^T A$.

After obtaining the initial estimate $\hat{E}$, it might not satisfy the rank 2 constraint ($\det(\hat{E}) \neq 0$). We need to find the closest rank 2 matrix E to $\hat{E}$. This is done by computing the SVD of $\hat{E} = U \Sigma V^T$ and setting the smallest singular value in Σ to zero to get Σ' . The rank 2 approximation is $E = U \Sigma' V^T$.

20.6.1 Considerations

While the 8-point algorithm requires 8 correspondences, the essential matrix has only 5 DOF; this suggests that fewer points might theoretically be sufficient. Indeed, algorithms exist that can compute E or F from the minimum number of points (5 for E , 7 for F); however, the 8-point algorithm is commonly used, especially for F . Moreover, using more than 8 points ($N > 8$) is often done in practice to reduce the impact of noisy measurements.

20.6.2 The normalized 8-point algorithm (not in the course slides)

The standard 8-point algorithm can suffer from poor numerical stability, especially with large pixel values from modern cameras. The resulting matrix A can be ill-conditioned for SVD. The normalized eight-point algorithm improves precision by normalizing image points before constructing the matrix A .

Normalization involves applying a translation and scaling transformation to the image coordinates; for each image, a transformation matrix T (or T') is computed such that:

1. the centroid of the image points is moved to the origin;

2. the mean squared distance of the transformed points from the origin is a fixed value (e.g., $\sqrt{2}$ pixels, so mean square distance is 2). The scaling factor is

$$\left(\frac{2}{\frac{1}{N} \sum_{i=1}^N \|x_i - \bar{x}\|^2} \right)^{1/2} \quad (20.19)$$

The normalization process is represented by matrices T and T' for the first and second images, respectively. Normalized points are $q_i = Tp_i$ and $q'_i = T'p'_i$. The 8-point algorithm is then applied using the normalized points q_i, q'_i to compute a normalized fundamental matrix F_q : $q_i'^T F_q q_i = 0$. Once F_q is computed (and its rank enforced), it must be denormalized to be used with original pixel coordinates. The denormalized fundamental matrix F is given by: $F = T'^T F_q T$. This normalized approach yields better results in real-world applications.

20.7 How to get T?

To estimate the camera pose (relative rotation R and translation t) between two views, given corresponding points:

1. if using pixel coordinates, first compute the Fundamental Matrix F (e.g., using the Normalized 8-point algorithm);
2. if camera intrinsic matrices K, K' are known, recover the essential matrix E from F using $E = K'^T F K$. If starting with normalized coordinates (camera frame), E can be estimated directly (e.g., with a normalized 8-point or 5-point algorithm on direction vectors). Direction vectors from camera centers can be obtained from image points by undoing the camera matrix effects, $d = K^{-1}x$ (or something proportional);
3. decompose the essential matrix E to extract R and t (section ??)

Decomposing E into R and $[t]_\times$ yields multiple solutions: given $E = U \text{diag}(1, 1, 0) V^T$, there are two possible skew-symmetric matrices $S = U Z U^T$ or $S = U W Z W^T U^T$, where

$$Z = \begin{pmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \quad W = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad Z = \begin{pmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \quad (20.20)$$

are related to singular values, and two possible rotation matrices $R = U W V^T$ or $R = U W^T V^T$. This results in four possible solutions for the (R, t) pair:

$$[U W V^T | +u_3] \quad [U W V^T | -u_3] \quad [U W^T V^T | +u_3] \quad [U W^T V^T | -u_3] \quad (20.21)$$

where u_3 is the third column of U (related to t).

The four solutions correspond to different camera configurations (baseline direction reversal, 180-degree rotation about baseline). **To determine the correct solution, one can triangulate** a 3D point X from a corresponding 2D point pair x, x' using each of the four (R, t) solutions. The correct solution is the one where the reconstructed 3D point lies in front of both cameras (i.e., has positive depth in both camera frames). In practice, **triangulating multiple points and choosing the solution with the most points in front of both cameras (or within the camera frustum) is more robust**.

20.8 Concusions

Epipolar geometry provides fundamental constraints between multiple views, encapsulated by the essential and fundamental matrices. These matrices can be estimated from corresponding points alone:

- the essential matrix, applicable to calibrated cameras, allows extraction of camera pose (R, t) up to a four-fold ambiguity, which can be resolved by triangulation,
- the fundamental matrix, for uncalibrated cameras, captures the projective relationship and can be estimated using algorithms like the normalized eight-point algorithm.

Estimating E or F is a key step in multi-view geometry problems such as stereo correspondence, camera motion estimation, and structure from motion.

Improvements to the basic algorithms include:

- using algorithms requiring fewer correspondences (e.g., 5-point algorithm for E) for potentially better efficiency or robustness in certain situations;

- normalizing image points to improve the numerical stability of the estimation process;
- refining the initial estimates of E , F , R , t , or 3D points using non-linear optimization techniques like bundle adjustment.

21 RANSAC

21.1 3D point registration

In order to find a correspondence between two images in 3D space, we need at least three points; for 3D point registration with unknown correspondences and an initial guess, the goal is to find a transform that minimizes the distance between corresponding points.

If we knew a minimal set of correct correspondences, we could:

1. find an initial guess of our system by using previously seen tools;
2. with this initial guess, find more "good" correspondences (inliers);
3. determine the solution by considering all "good" correspondences (the inliers), and dropping the bad ones (the outliers).

21.2 RANSAC: random sample consensus

RANSAC is a principle rather than a strict algorithm: it is a schema used to seek for a good solution, not a "closed" algorithm.

It is used to find a robust model by iteratively selecting minimal sets of correspondences and filtering out outliers and only has guarantees if there are less than 50% outliers. All the inliers will agree with each other on the translation vector, while the outliers will disagree.

The general version of the RANSAC algorithm operates for N iterations:

1. sample (randomly) a minimal set of correspondences among the candidate ones, the minimal sample size s is chosen such that it allows fitting a model. Use the minimum number of sample points k possible for any given trial;
2. compute an initial alignment: hypothesize a model based on these samples, for example, fit a line to those samples. The required number of sample points depends on the motion model, and each sample is a correspondence (a pair of matching points);
3. use this alignment or model to determine the number of good/bad correspondences: compute an error function and determine inliers/outliers based on this alignment. Select points consistent with the model;
4. compute the "consensus" of the guess as a function of number of inliers and error: count the number of inliers that approximately fit the model.
5. repeat the above steps N times, and at each time keep the "best" solution (choose the model that has the largest set of inliers).

When done, use only inliers to improve the final solution. Re-compute the least-squares estimate on all of the inliers.

For estimating homography, a loop version of RANSAC is used:

1. select four feature pairs at random;
2. compute homography H (exact);
3. compute inliers where $\text{dist}(p', Hp) < \epsilon$. The inlier threshold is related to the amount of noise expected in inliers, often modeled as Gaussian noise with some standard deviation (e.g., 3 pixels);
4. keep the largest set of inliers;
5. re-compute least-squares H estimate on all of the inliers.

This process is an example of a "voting"-based fitting scheme, where each hypothesis gets voted on by each data point, and the best hypothesis wins.

21.3 RANSAC: What do we need

To implement the RANSAC schema, we need procedures for the following components:

- seek for correspondences;
- smartly select a set of pseudo-random correspondences between candidate correspondences (in the worst case, it is uniform sampling);
- compute a solution, immune to poor initial guesses;
- count the inliers.

21.4 Correspondences

The correspondence search is usually done exploiting the appearance of features, often by feature matching, this usually leads to few "good" correspondences. The correspondence search is characterized by an inlier ratio, denoted by w , defined as the number of good matches divided by the total number of matches:

$$w = \frac{\text{good matches}}{\text{total matches}} \quad (21.1)$$

A reasonable inlier ratio might be 0.2.

In a worst-case scenario, such as assuming "all" points match with all other points, with N points, we might have $N \times N$ correspondences and the inlier ratio could be as low as $1/N$.

21.5 Determining a solution

Assuming a poor initial guess, linear relaxation is preferred to compute the initial solution as it is immune from the initial guess; for example, linear least squares (section ??) can be used to compute transformations like translation, affine transformations, or homographies, provided there are enough matching points.

21.6 Pruning correspondences

In the absence of a reasonable appearance-based correspondence selection, some invariants like distance in space can be exploited. This is not part of RANSAC itself, but a simple optimization to help prune wrong correspondences; for example, one can select a triplet of points in the world that are reasonably distant from each other and select a triplet of points in the measurement that have more or less the same distances.

21.7 Selecting inliers

Given correspondences, we can "count" the inliers:

1. compute the error of each corresponding point under the proposed solution;
2. correspondences that are "good" are counted;
3. the more inliers there are, the better the solution (more good points will correspond to a better solution).

The inlier threshold determines which points are considered consistent with the model.

21.8 How many rounds?

RANSAC is a random procedure, so a correct solution is never guaranteed; however, given a sufficient number of trials k , we can determine the probability of obtaining a correct solution.

To determine the required number of trials, we need to know:

- the inlier ratio w of the correspondences;
- the minimal number of model points n required to compute the initial solution;
- the desired probability of success p .

The probability of:

- fetching n inliers in one trial is w^n ;

- NOT fetching n inliers in one trial is $1 - w^n$;
- not having n inliers in k rounds (trials) is $(1 - w^n)^k$;
- failure is $1 - p = (1 - w^n)^k$

The required minimum number of trials k to achieve a desired probability of success p can be derived from this equation:

$$k = \frac{\log(1 - p)}{\log(1 - w^n)} \quad (21.2)$$

The number of trials grows quickly with the number of sample points used: the smaller the minimal number of model points n , the less trials are needed.

For example, suppose there are 20% outliers ($e = 0.2$, so $w = 0.8$), and we want to find the correct answer with 99% probability ($p = 0.99$). If the minimal sample size $n = 2$, the number of rounds

$$k \geq \frac{\log(1 - 0.99)}{\log(1 - (0.8)^2)} = \frac{\log(0.01)}{\log(1 - 0.64)} = \frac{\log(0.01)}{\log(0.36)} \approx \frac{-4.605}{-1.022} \approx 4.5 \quad (21.3)$$

since the number of rounds must be an integer, we would need at least 5 rounds. If $n = 5$,

$$k \geq \frac{\log(0.01)}{\log(1 - (0.8)^5)} = \frac{\log(0.01)}{\log(1 - 0.32768)} = \frac{\log(0.01)}{\log(0.67232)} \approx \frac{-4.605}{-0.396} \approx 11.6 \quad (21.4)$$

we would need at least 12 rounds. This illustrates that the number of trials grows quickly with n .

21.9 Pros and cons

RANSAC has several advantages and disadvantages:

Pros	Cons
Simple and general.	Parameters to tune.
Applicable to many different problems.	Sometimes too many iterations are required.
Often works well in practice.	Can fail for extremely low inlier ratios.
Provides a robust framework for estimating transformations in the presence of outliers.	We can often do better than brute-force sampling.

Optimizing correspondence selection and using minimal model points reduces the required iterations.

22 Multi-Point Registration

The environment for the multi-point registration problem consists of unknown and identifiable landmarks, observed by a 3D sensor from multiple positions, whose positions are unknown. The overall goal is to determine both the landmark locations and the robot positions from which these observations were made. This problem is a form of SLAM.

22.1 State

The state of the system is represented as

$$X : X = \{X_r^{[1]}, \dots, X_r^{[N]}, X_l^{[1]}, \dots, X_l^{[M]}\} \quad (22.1)$$

so a collection of:

- robot poses: $X_r^{[n]} \in SE(3)$ for robot pose n , represented as a transformation matrix $\begin{pmatrix} R^{[n]} & t^{[n]} \end{pmatrix}$ where $R^{[n]} \in SO(3)$ is the rotation matrix and $t^{[n]} \in \mathbb{R}^3$ is the translation vector;
- landmark positions: $X_l^{[m]} \in \mathbb{R}^3$ for landmark m , represented as a 3D vector $\begin{pmatrix} x^{[m]} & y^{[m]} & z^{[m]} \end{pmatrix}^T$.

To represent increments or updates to this state, a large vector containing the minimal perturbation for each state variable is used. The full state perturbation vector $\Delta x \in \mathbb{R}^{6N+3M}$ concatenates the perturbations for all N robot poses and M landmarks:

$$\Delta x = \left(\Delta x_r^{[1]T} \quad \dots \quad \Delta x_r^{[N]T} \quad \Delta x_l^{[1]T} \quad \dots \quad \Delta x_l^{[M]T} \right)^T \quad (22.2)$$

- robot state perturbation: $\Delta x_r^{[n]} \in \mathbb{R}^6$ for pose n , represented as a 6D vector $\begin{pmatrix} \Delta t^{[n]} & \Delta \alpha^{[n]} \end{pmatrix}^T$, where

$$\begin{aligned} \Delta t^{[n]} &= \begin{pmatrix} \Delta x^{[n]} & \Delta y^{[n]} & \Delta z^{[n]} \end{pmatrix} \in \mathbb{R}^3 & \Delta \alpha^{[n]} &= \begin{pmatrix} \Delta \alpha_x^{[n]} & \Delta \alpha_y^{[n]} & \Delta \alpha_z^{[n]} \end{pmatrix} \in \mathbb{R}^3 \\ \text{translational perturbation} & & \text{rotational perturbation (e.g., exponential map coordinates)} \end{aligned} \quad (22.3)$$

- landmark state perturbation: $\Delta x_l^{[m]} \in \mathbb{R}^3$ for landmark m , represented as a 3D vector $\Delta t^{[m]}$ (defined in the same way as the translational perturbation in equation ??).

22.1.1 State (implementation)

In an implementation, such as in Octave, the state can be represented using arrays and matrices:

- robot poses: an array of 4×4 transformation matrices, one for each pose;
- landmarks: a matrix of $3 \times \text{num_landmarks}$ elements, where each column represents a landmark's 3D position vector.

To manage the mapping between pose/landmark indices and their corresponding blocks in the large perturbation vector Δx , functions can be used. Let `pose_dim` be the dimension of the robot pose perturbation and `landmark_dim` be the dimension of the landmark perturbation:

- for a pose with index `pose_index` (1-based), its perturbation block starts at index:

$$\text{v_idx} = 1 + (\text{pose_index} - 1) * \text{pose_dim}$$

using `pose_dim=6`;

- for a landmark with index `landmark_index` (1-based), its perturbation block starts at index:

$$\text{v_idx} = 1 + (\text{num_poses}) * \text{pose_dim} + (\text{landmark_index} - 1) * \text{landmark_dim}$$

using `pose_dim=6` and `landmark_dim=3`.

22.2 Boxplus

The boxplus operator is used to update the state variables with their computed perturbations. The general update rule is $X' = X \boxplus \Delta x$. For a:

- robot pose $X_r^{[n]}$, the update with perturbation $\Delta x_r^{[n]}$ is given by applying the transformation corresponding to the perturbation to the current pose:

$$X_r^{[n]'} = v2t(\Delta x_r^{[n]})X_r^{[n]} \quad (22.4)$$

$v2t(\cdot)$ is a function that maps a 6-dimensional perturbation vector to an $SE(3)$ transformation matrix.

- landmark position $X_l^{[m]}$, the update with perturbation $\Delta x_l^{[m]}$ is a simple vector addition:

$$X_l^{[m]'} = X_l^{[m]} + \Delta x_l^{[m]} \quad (22.5)$$

22.3 Measurements and predictions

A measurement in this problem involves a robot pose observing a landmark; a measurement of landmark m performed by robot pose n is denoted as

$$z^{[n,m]} \in \mathbb{R}^3 : z^{[n,m]} = \begin{pmatrix} x^{[n,m]} & y^{[n,m]} & z^{[n,m]} \end{pmatrix} \quad (22.6)$$

representing the observed 3D position of the landmark relative to the robot's sensor frame.

The prediction $h^{[n,m]}$ (note: sometimes the prediction is indicated with $\hat{z}^{[n,m]}$) for such a measurement is calculated based on the current estimates of the robot pose $X_r^{[n]}$ and the landmark position $X_l^{[m]}$. Assuming the landmark's position is given in the world frame, the predicted measurement is the landmark's position transformed into the robot's frame:

$$h^{[n,m]} = X_r^{[n]} X_l^{[m]} \quad (22.7)$$

The error $e^{[m,n]}(X)$ for this measurement is the difference between the predicted measurement and the actual measurement:

$$e^{[m,n]}(X) = h^{[n,m]} - z^{[n,m]} = X_r^{[n]} X_l^{[m]} - z^{[n,m]} \quad (22.8)$$

If we consider a perturbation $(X \boxplus x)$, then the error is

$$e^{[m,n]}(X \boxplus x) = v2t(\Delta x_r^{[n]}) X_r^{[n]} (X_l^{[m]} + \Delta x_l^{[m]}) - z^{[n,m]} \quad (22.9)$$

22.4 Jacobians

The prediction $h^{[n,m]}$ depends only on the robot pose $X_r^{[n]}$ and the landmark position $X_l^{[m]}$. Consequently, the Jacobian of the error with respect to the full state perturbation vector Δx will be mostly zero and non-zero elements exist only for the blocks corresponding to robot pose n and landmark m : the Jacobian matrix is sparse, with non-zero blocks only at the columns associated with pose n and landmark m .

$$\begin{aligned} J^{[n,m]} &= \frac{\partial e^{[m,n]}(X \boxplus \Delta x)}{\partial \Delta x} = \begin{pmatrix} 0_{3 \times 6} & \dots & \frac{\partial e^{[m,n]}(X \boxplus \Delta x)}{\partial \Delta x_r^{[n]}} & 0_{3 \times 6} & 0_{3 \times 3} & \dots & \frac{\partial e^{[m,n]}(X \boxplus \Delta x)}{\partial \Delta x_l^{[m]}} & \dots & 0_{3 \times 3} \end{pmatrix} \\ &= \begin{pmatrix} 0_{3 \times 6} & \dots & J_r^{[n,m]} & 0_{3 \times 6} & 0_{3 \times 3} & \dots & J_l^{[n,m]} & \dots & 0_{3 \times 3} \end{pmatrix} \end{aligned} \quad (22.10)$$

The specific non-zero Jacobian blocks with respect to the pose and landmark perturbations, evaluated at zero perturbation ($\Delta x = 0$), are:

- Jacobian with respect to the robot pose perturbation $\Delta x_r^{[n]}$:

$$J_r^{[n,m]} = \left. \frac{\partial e^{[m,n]}(X \boxplus \Delta x)}{\partial \Delta x_r^{[n]}} \right|_{\Delta x=0} = \begin{pmatrix} I_{3 \times 3} & [-h^{[n,m]}]_{\times} \end{pmatrix} \quad (22.11)$$

where $I_{3 \times 3}$ is the 3x3 identity matrix and $[-h^{[n,m]}]_{\times}$ is the skew-symmetric matrix of the negative of the predicted measurement.

- Jacobian with respect to the landmark position perturbation $\Delta x_l^{[m]}$:

$$J_l^{[n,m]} = \left. \frac{\partial e^{[m,n]}(X \boxplus \Delta x)}{\partial \Delta x_l^{[m]}} \right|_{\Delta x=0} = R^{[n]} \quad (22.12)$$

where $R^{[n]}$ is the rotation part of the robot pose $X_r^{[n]}$.

22.5 H Matrix and B vector

In the context of least squares optimization, the problem is linearized and solved iteratively. Each measurement contributes to building a global Hessian matrix H and coefficient vector b for a linear system $H\Delta x = -b$. For a single measurement of landmark m by robot pose n , the contribution to H and b is given by:

$$H^{[m,n]} = J^{[m,n]T} \Omega^{[m,n]} J^{[m,n]} \quad b^{[m,n]} = J^{[m,n]T} \Omega^{[m,n]} e^{[m,n]} \quad (22.13)$$

where

- $J^{[m,n]}$ is the Jacobian of the error for this measurement with respect to the full state perturbation Δx ,
- $\Omega^{[m,n]}$ is the information matrix for the measurement (inverse of its covariance),
- $e^{[m,n]}$ is the error vector.

Since $J^{[m,n]}$ is sparse with non-zero blocks only for pose n and landmark m , the contribution matrix $H^{[m,n]}$ will have only 4 non-zero blocks:

$$H^{[m,n]} = \begin{pmatrix} \dots & J_r^{[m,n]T} \Omega^{[m,n]} J_r^{[m,n]} & \dots & J_r^{[m,n]T} \Omega^{[m,n]} J_l^{[m,n]} & \dots \\ \dots & \dots & \dots & \dots & \dots \\ \dots & J_l^{[m,n]T} \Omega^{[m,n]} J_r^{[m,n]} & \dots & J_l^{[m,n]T} \Omega^{[m,n]} J_l^{[m,n]} & \dots \end{pmatrix} \quad (22.14)$$

These blocks are located at the intersubsections of the rows and columns corresponding to pose n and landmark m . The contribution vector $b^{[m,n]}$ has two non-zero blocks:

$$b^{[m,n]} = \begin{pmatrix} \vdots \\ J_r^{[m,n]T} \Omega^{[m,n]} e^{[m,n]} \\ \vdots \\ J_l^{[m,n]T} \Omega^{[m,n]} e^{[m,n]} \end{pmatrix} \quad (22.15)$$

The overall H and b matrices for the entire system are formed by summing the contributions from all measurements. Due to the sparsity of the individual measurement contributions, the resulting global H matrix is also sparse and its structure reflects the connections in the graph of poses and landmarks.

22.6 Degrees of Freedom

The multi-point registration problem, formulated as a least squares problem on the full state, has inherent ambiguities. Applying any rigid transformation (translation and rotation) to all robot poses and landmark positions simultaneously results in a new state configuration that has the same error value: the system has 6 degrees of freedom (3 for translation and 3 for rotation) which are unconstrained by the relative measurements alone. This makes the resulting linear system $H\Delta x = -b$ under-determined and the matrix H will not have full rank. To obtain a unique solution, we need to either

- remove the redundant degrees of freedom;
- add a prior.

22.6.1 Eliminating variables

One way to address the under-determined nature of the system is by eliminating redundant variables. This can be achieved by fixing the pose of one of the robot poses. For example, the first robot pose can be fixed to the origin of the coordinate system.

To implement this, the linear system $H\Delta x = -b$ is reduced by suppressing the rows and columns corresponding to the variables that are fixed, we can do this because the perturbation Δx for a variable that is fixed (not moving on the manifold) is zero. The reduced system is then solved for the remaining, non-fixed variables.

The computed perturbation vector from the reduced system is then used to update the state, including propagating zero perturbation to the variables that were eliminated (fixed).

22.6.2 Adding a prior

An alternative to eliminating variables is adding a prior constraint, which effectively fixes the global coordinate frame, to the least squares problem. The simplest prior is a flexible constraint that the first robot pose should be close to a specific value, typically the origin 0. This constraint can be expressed as

$$\Delta x_r^{[1]} = 0 \quad (22.16)$$

This is added to the least squares formulation by introducing an additional error term, whose Jacobian with respect to $\Delta x_r^{[1]}$ is simply the identity matrix:

$$e_{prior} = \Delta x_r^{[1]} \quad J_{prior} = I \quad (22.17)$$

This prior constraint contributes to the H and b matrices by adding a large information matrix Ω_{prior} (e.g., $I_{6 \times 6}$ scaled by a large value) to the block of the H matrix corresponding to the first robot pose variables. The larger the values in Ω_{prior} , the stiffer the constraint, forcing the first pose perturbation closer to zero. The corresponding addition to the b vector is evaluated at the current estimate

$$H_{prior} = \Omega_{prior} \quad b_{prior} = \Omega_{\Delta x} \quad (22.18)$$

As the optimization converges, Δx should approach zero, making b_{prior} also approach zero:

$$\Delta x \rightarrow 0 \implies b_{prior} \rightarrow 0 \quad (22.19)$$

22.7 Structure

The structure of the Hessian matrix H refers to the pattern of its non-zero elements and is determined solely by the connections between variables introduced by the measurements. Since a measurement relates one robot pose and one landmark, it introduces non-zero elements in H only at the blocks connecting that pose and that landmark.

The independence of measurements ensures that the overall H matrix is sparse. The structure features **full diagonal blocks** at the positions corresponding to each variable and also **full blocks at positions $[m, n]$ if robot m observes landmark n** . This sparse structure effectively represents the adjacency matrix of the underlying graph formed by robot poses and landmarks as nodes and measurements as edges. An important property is that the structure of H **does not change** during the iterations of the optimization process.

22.8 Conclusions

Multi-point registration of landmarks observed from multiple robot poses is a complex problem, but by formulating it as a least squares problem, the state estimation can be tackled effectively.

- The problem formulation naturally leads to a sparse structure for the Hessian matrix H due to the independence of measurements, where each measurement constrains only the involved robot pose and landmark.
- The structure of the H matrix remains constant throughout the optimization iterations.
- This sparsity is a key advantage, enabling the linear system $H\Delta x = -b$ to be solved efficiently using sparse methods. Examples of such methods include sparse Cholesky decomposition and conjugate gradients.

23 Multi-pose registration and graph-SLAM

Graph-SLAM is an alternative SLAM algorithm to EKF SLAM that addresses the **full SLAM problem**. It calculates a solution for the **offline problem** defined over all poses and all features in the map.

EKF SLAM

incremental algorithm that only maintains a posterior over the momentary pose

proactive: it resolves new information immediately

lower accuracy

Graph-SLAM

calculates posteriors over robot paths

lazy: accumulates information in its graph without immediate resolution²⁸

higher accuracy since it can use the full dataset for linearization and data association, potentially revisiting past decisions and iterating these steps.

23.1 Graph-based SLAM in a nutshell

Graph-based SLAM formulates the simultaneous localization and mapping problem as an optimization problem over a graph. In the context of pose graph SLAM, this graph consists of **nodes representing robot poses** at different points in time and **edges defining spatial constraints** between these nodes; these constraints are derived from robot motion (odometry) or loop closure detections (observing previously seen areas). The constraints are inherently uncertain, considered as soft constraints.

The core idea of Graph-based SLAM is to build this graph and then find a configuration of the nodes (robot poses) that minimizes the error introduced by these constraints. This is typically formulated as a **least squares optimization problem** aiming to minimize the sum of squared errors associated with the constraints.

23.2 What does the graph look like?

The graph in graph-based SLAM consists of nodes and edges:

- each node, denoted as $x^{[i]}$, represents the pose of the robot at a specific time t_i . In 2D, a pose includes position and orientation (e.g., x, y, θ). In 3D, it involves position (x, y, z) and rotational angles (e.g., yaw, pitch, roll). Nodes can also represent laser measurements in some formulations;
- an edge $e^{[i,j]}$ (a constraint) exists between two nodes $x^{[i]}$ and $x^{[j]}$ if a spatial relationship between them is observed:
 - motion edges: link consecutive robot poses, e.g., $x^{[i]}$ and x_{i+1} . These represent motion commands or odometry measurements, indicating the relative transformation between the poses based on the robot's movement;
 - measurement edges: link poses to features sensed at that pose. In landmark-based SLAM, these edges connect a robot pose node to a landmark node. In pose graph SLAM, where landmarks might be marginalized out, edges can exist between non-consecutive poses ($x^{[i]}$ and $x^{[j]}$, where $j \neq i + 1$) if the robot observed the same part of the environment from both poses. By comparing or aligning sensor data (like laser scans) from these different poses (e.g., using ICP), a "virtual measurement" or relative transformation between $x^{[i]}$ and $x^{[j]}$ is computed. These edges represent the position of $x^{[j]}$ as seen from $x^{[i]}$ based on the observations.

The structure of the graph naturally forms a sparse graph, where the number of edges is linear in the number of nodes, and each node is connected to a finite number of other nodes. Motion constraints link consecutive poses, and measurement constraints link poses to features or introduce links between poses that share observations, leading to a sparse structure.

23.3 Graph Optimization

Graph-based SLAM algorithms generally interleave graph construction and graph optimization. We already saw the construction step, the optimization step, often called the back-end, focuses on finding the configuration of the nodes that is maximally consistent with the observations encoded in the edges. A consistent map, derived from the current pose estimates, can aid the front-end in determining new constraints by reducing the search space for correspondences.

The optimization problem is to find the assignment of poses to the nodes of the graph which minimizes the negative log likelihood of the observations, which corresponds to minimizing the sum of quadratic constraints,

as derived from the assumption of Gaussian noise in motion and measurement models. The objective function is $J_{\text{Graph-SLAM}}$, a sum of nonlinear quadratic constraints.

23.4 The Edge Information Matrices

Each edge in the graph represents a constraint, and these constraints are uncertain due to sensor noise or imperfections in data association. To encode this uncertainty, an information matrix $\Omega^{[i,j]}$ is associated with each edge $e^{[i,j]}$. The information matrix is the inverse of the covariance matrix associated with the measurement or motion model uncertainty (R_t^{-1} for motion, Q_t^{-1} for measurement noise).

The "bigger" (in the matrix sense) the information matrix $\Omega^{[i,j]}$ is, the more that edge "matters" in the optimization procedure: a large value in $\Omega^{[i,j]}$ signifies low uncertainty and thus a strong constraint. For example, scan matching using a laser range scanner typically provides more accurate relative pose information than odometry, so the information matrices for scan-matching edges would generally be "bigger" than those for odometry edges. In a long, featureless corridor, scan matching provides strong constraints perpendicular to the corridor walls and on orientation, but weak constraints along the corridor axis, leading to a covariance matrix that is narrow perpendicular to the axis and stretched along it. The corresponding information matrix would be large perpendicular to the axis and small along it.

23.5 Pose graph

The input for the optimization procedure is the constructed graph, annotated with nodes (poses) and edges (constraints), where each edge has an associated observation (virtual measurement or odometry) and an information matrix representing its uncertainty. The goal is to find the configuration of poses that minimizes the sum of weighted squared errors defined by these annotated edges:

$$\hat{X} = \arg \min_x \sum_{i,j} e^{[i,j]T} \Omega^{[i,j]} e^{[i,j]} \quad (23.1)$$

where

- X is the collection of all robot poses,
- $e^{[i,j]}(X)$ is the error function for the constraint between poses $x^{[i]}$ and $x^{[j]}$,
- $\Omega^{[i,j]}$ is the information matrix for that constraint.

23.5.1 State

The state variable in pose graph SLAM is the collection of all robot poses throughout the trajectory. If there are N poses, $x_1, \dots, x_N$, the state can be represented as a large vector

$$X : \quad X = \{X_r^{[1]}, \dots, X_r^{[N]}\} \quad (23.2)$$

Each pose $x^{[i]}$ is typically represented as a transformation matrix in $SE(3)$ (for 3D) or $SE(2)$ (for 2D), including both rotation and translation

$$X_r^{[n]} \in SE(3) : \quad X^{[n]} = \begin{pmatrix} R^{[n]} & t^{[n]} \end{pmatrix} \quad (23.3)$$

In the context of landmarks, they are sometimes considered absorbed by the relative pose constraints.

For optimization using linearization methods like Gauss-Newton, updates to the state are represented by a vector of minimal perturbations Δx . For $SE(3)$, each pose perturbation $\Delta x^{[i]}$ is a 6D vector (3 for translation, 3 for rotation). The full perturbation vector Δx concatenates the perturbations for all N poses:

$$\begin{aligned} \Delta x \in R^{6N} : \quad \Delta x &= \left(\Delta x_r^{[1]T} \quad \dots \quad \Delta x_r^{[N]T} \right)^T \\ \Delta x_r^{[n]T} \in R^6 : \quad \Delta x_r^{[n]T} &= \left(\Delta x^{[n]} \quad \Delta y^{[n]} \quad \Delta z^{[n]} \quad \Delta \alpha_x^{[n]} \quad \Delta \alpha_y^{[n]} \quad \Delta \alpha_z^{[n]} \right)^T = \left(\Delta t^{[n]} \quad \Delta \alpha^{[n]} \right) \end{aligned} \quad (23.4)$$

23.5.2 Measurements and predictions

A measurement or constraint between two robot poses $x^{[i]}$ and $x^{[j]}$ provides information about their relative configuration. Let $Z^{[i,j]}$ denote this measurement, represented as a transformation in $SE(3)$ (or $SE(2)$).

$$Z^{[i,j]} \in SE(3) : \quad Z^{[i,j]} = \begin{pmatrix} R^{[i,j]} & t^{[i,j]} \end{pmatrix} \quad (23.5)$$

The prediction, denoted $h^{[i,j]}(X)$, is the relative transformation between $x^{[i]}$ and $x^{[j]}$ according to the current state estimate X . This is calculated as the inverse of the transformation of pose i multiplied by the transformation of pose j :

$$\hat{Z}^{[i,j]} = h^{[i,j]}(X) = X_r^{[i]-1} X_r^{[j]} \quad (23.6)$$

The error function for this constraint measures the discrepancy between the predicted relative pose and the actual measurement $Z^{[i,j]}$. Since poses and measurements live on a non-Euclidean space (the manifold of rigid body transformations), a direct subtraction is not appropriate, instead, the error is computed using the boxminus operation

$$e^{[i,j]}(X) = X_r^{[i]-1} X_r^{[j]} \boxminus Z^{[i,j]} \quad (23.7)$$

The error in case of a perturbation can be formulated by transforming the relative pose from the graph by the inverse of the measured relative pose:

$$\begin{aligned} e^{[i,j]}(X \boxplus \Delta x) &= t2v \left(Z^{[i,j]-1} \left(v2t(\Delta x_r^{[i]}) X_r^{[i]} \right)^{-1} \left(v2t(\Delta x_r^{[j]}) X_r^{[j]} \right) \right) \\ &= t2v \left(Z^{[i,j]-1} X_r^{[i]-1} v2t(\Delta x_r^{[i]})^{-1} v2t(\Delta x_r^{[j]}) X_r^{[j]} \right) \end{aligned} \quad (23.8)$$

where $t2v$ (transformation to vector) maps the resulting transformation matrix back into a vector representation (e.g., 6D vector for SE(3)). In the ideal case, if the prediction exactly matches the measurement, this discrepancy should be the identity transformation, and the error vector should be zero.

23.5.3 Boxplus

When performing optimization, the state estimate X is updated iteratively using the calculated perturbation Δx using the boxplus operation (in a manifold). For a single pose $X_r^{[n]}$ and its perturbation $\Delta x_r^{[n]}$, the updated pose $X_r^{[n]}'$ is calculated as:

$$X_r^{[n]}' = X_r^{[n]} \boxplus \Delta x_r^{[n]} = v2t(\Delta x_r^{[n]}) X_r^{[n]} \quad (23.9)$$

where $v2t$ (vector to transformation) maps the perturbation vector $\Delta x_r^{[n]}$ into a transformation matrix representation. This is applied to each pose in the state vector $X' = X \boxplus \Delta x$, where Δx is the concatenation of individual pose perturbations.

23.5.4 Jacobians

In the least squares optimization framework, linearizing the error function is crucial for methods like Gauss-Newton. The Jacobian matrix J contains the partial derivatives of the error function $e(X)$ with respect to the state variables (the pose parameters).

For a single constraint $e^{[i,j]}(X)$, the error depends only on the two poses involved in the constraint, $x^{[i]}$ and $x^{[j]}$. This means the Jacobian matrix for this constraint, when considered with respect to the entire state vector X , will be sparse and non-zero elements exist only in the columns corresponding to $x^{[i]}$ (the "observing" pose) and $x^{[j]}$ (the "observed" pose):

$$J^{[i,j]} = \frac{\partial e^{[i,j]}(X \boxplus \Delta x)}{\partial \Delta x} = \begin{pmatrix} \dots & 0_{6 \times 6} & \dots & J_i^{[i,j]} & 0_{6 \times 6} & J_j^{[i,j]} & \dots & 0_{6 \times 6} & \dots \end{pmatrix} \quad (23.10)$$

where

$$J_i^{[i,j]} = \frac{\partial e^{[i,j]}(X \boxplus \Delta x)}{\partial \Delta x_r^{[i]}} \Big|_{\Delta x_r^{[i]}=0} \quad J_j^{[i,j]} = \frac{\partial e^{[i,j]}(X \boxplus \Delta x)}{\partial \Delta x_r^{[j]}} \Big|_{\Delta x_r^{[j]}=0} \quad (23.11)$$

are the partial derivatives of the error function with respect to the parameters of pose i and pose j , respectively. These non-zero blocks are 6x6 matrices for SE(3) poses when the error is represented as a 6D vector. Computing these Jacobians can be complex, especially with the boxminus formulation on manifolds. The Jacobian is tall 6 and as wide as the state.

23.6 Information Matrix

In the information form of Gaussian distributions, the precision matrix (inverse covariance) represents the information. The information matrix for the overall SLAM problem is the sum of contributions from each individual constraint's information matrix, weighted by the squared Jacobian.

For a single measurement $Z^{[i,j]}$ on a non-Euclidean space, the adapted information matrix is used:

$$\tilde{\Omega}^{[i,j]} = \left(J_e^{[i,j]} \Omega^{[i,j]} (J_e^{[i,j]})^T \right)^{-1} \quad (23.12)$$

where:

- $\Omega^{[i,j]}$ is the original measurement noise covariance;
- $J_e^{[i,j]} = \frac{\hat{z}^{[i,j]} \boxminus z}{\partial z} \Big|_{z=z^{[i,j]}}$ is the derivative of the error with respect to the measurement itself;
- $\hat{z}^{[i,j]} = x_r^{[i]-1} x_r^{[j]}$ is the prediction.

This $\tilde{\Omega}^{[i,j]}$ then weights the contribution of the constraint's error and Jacobians to the overall system.

23.7 H matrix and b vector

In the Gauss-Newton approach, the optimization problem $\min \sum e^{[i,j]}(X)^T \Omega^{[i,j]} e^{[i,j]}(X)$ is linearized and solved iteratively by solving a linear system $H \Delta x = -b$.

The matrix H (often called the Hessian or information matrix of the linearized system) and the vector b (often called the gradient or information vector) are computed by summing the contributions from each individual constraint $e^{[i,j]}(X)$:

$$H = \sum_{i,j} J^{[i,j]T} \tilde{\Omega}^{[i,j]} J^{[i,j]} \quad b = \sum_{i,j} J^{[i,j]T} \tilde{\Omega}^{[i,j]} e^{[i,j]} \quad (23.13)$$

where

- $J^{[i,j]}$ is the Jacobian of the error function $e^{[i,j]}$ with respect to the state X ,
- $\tilde{\Omega}^{[i,j]}$ is the adapted information matrix for the constraint (or simply $\Omega^{[i,j]}$ in formulations where the error is in a Euclidean space after mapping from the manifold).

Because the Jacobian $J^{[i,j]}$ is sparse, having non-zero blocks only for poses $x^{[i]}$ and $x^{[j]}$, the contribution of each constraint to H and b is also sparse. For a single constraint, the contribution to the:

- vector b has non-zero elements only at the blocks corresponding to $x^{[i]}$ and $x^{[j]}$;
- matrix H has non-zero blocks only at the intersubsections of the rows and columns corresponding to $x^{[i]}$ and $x^{[j]}$.

When summing the contributions from all constraints, the resulting matrix H inherits this sparsity. It is generally a **sparse matrix**, reflecting the connectivity of the graph; this sparsity is key to efficiently solving the linear system $H \Delta x = -b$, even for very large graphs, as sparse solvers (like sparse Cholesky decomposition or conjugate gradient) can be used.

$$H^{[i,j]} = J^{[i,j]T} \tilde{\Omega}^{[i,j]} J^{[i,j]} = \begin{pmatrix} \dots & J_i^{[i,j]T} \tilde{\Omega}^{[i,j]} J_i^{[i,j]} & \dots & J_i^{[i,j]T} \tilde{\Omega}^{[i,j]} J_j^{[i,j]} & \dots \\ & \vdots & & \vdots & \\ \dots & J_j^{[i,j]T} \tilde{\Omega}^{[i,j]} J_i^{[i,j]} & \dots & J_j^{[i,j]T} \tilde{\Omega}^{[i,j]} J_j^{[i,j]} & \dots \end{pmatrix} \quad b^{[i,j]} = J^{[i,j]T} \tilde{\Omega}^{[i,j]} e^{[i,j]} = \begin{pmatrix} \vdots \\ J_i^{[i,j]T} \tilde{\Omega}^{[i,j]} e^{[i,j]} \\ \vdots \\ J_j^{[i,j]T} \tilde{\Omega}^{[i,j]} e^{[i,j]} \\ \vdots \end{pmatrix} \quad (23.14)$$

23.8 Chordal Distance

Computing derivatives for error functions defined using the boxminus operation on manifolds can be difficult. As an alternative, especially for pose-pose constraints, the **chordal distance** can be used to define the error in a Euclidean space: given two transformation matrices, the chordal distance is the standard Euclidean difference between them after vectorizing their components. A "flatten" function can be introduced to turn a transformation matrix (e.g., a 4x4 matrix in homogeneous coordinates for 3D poses) into a vector containing its components. For a 3D pose in SE(3) represented by a matrix

$$X = \begin{pmatrix} R & t \\ 0 & 1 \end{pmatrix} \quad (23.15)$$

where

- $R = \begin{pmatrix} r_1 & r_2 & r_3 \end{pmatrix}$ is a 3x3 rotation matrix,
- t is a 3x1 translation vector,

the flatten operation might yield a 12x1 vector by concatenating the columns of the rotation matrix and the translation vector:

$$\text{flatten}(X) = \begin{pmatrix} r_1 \\ r_2 \\ r_3 \\ t \end{pmatrix} \quad (23.16)$$

This allows using standard vector subtraction (regular minus) to compute the error.

23.8.1 Prediction and error

Using the flatten function and standard vector subtraction, the prediction and error for a pose-pose constraint between $x^{[i]}$ and $x^{[j]}$ can be reformulated.

The prediction (now indicated as g and not any more as $\hat{z}$ or h , h here is the chordal prediction) is still the relative transformation $x^{[i]-1}x^{[j]}$:

$$g^{[i,j]}(X) = X^{[i]-1}X^{[j]} \quad (23.17)$$

The chordal prediction $h^{[i,j]}(X)$ is the flattened version of $g^{[i,j]}(X)$:

$$h^{[i,j]}(X) = \text{flatten}(g^{[i,j]}(X)) = \text{flatten}(X^{[i]-1}X^{[j]}) \quad (23.18)$$

The error $e^{[i,j]}(X)$ is then the simple vector difference between the chordal prediction and the flattened measurement $Z^{[i,j]}$ and has a dimension corresponding to the flattened matrix size (e.g., 12 for SE(3)).

$$e^{[i,j]}(X) = \text{flatten}(X^{[i]-1}X^{[j]}) - \text{flatten}(Z^{[i,j]}) \quad (23.19)$$

This formulation makes computing derivatives easier.

23.8.2 Jacobian

Using the chordal distance formulation simplifies the computation of the Jacobian of the error function. The error $e^{[i,j]}(X)$ depends on $x^{[i]}$ and $x^{[j]}$. When considering the error with respect to perturbations $\Delta x^{[i]}$ and $\Delta x^{[j]}$ applied to $x^{[i]}$ and $x^{[j]}$, i.e., $e^{[i,j]}(x^{[i]} \boxplus \Delta x^{[i]}, x^{[j]} \boxplus \Delta x^{[j]})$, the Jacobians are taken with respect to these perturbations evaluated at $\Delta x^{[i]} = 0, \Delta x^{[j]} = 0$.

$$\frac{\partial e^{[i,j]}(X \boxplus \Delta x)}{\partial \Delta x} = \begin{pmatrix} \dots & 0_{12 \times 6} & \dots & \frac{\partial e^{[i,j]}(X \boxplus \Delta x)}{\partial \Delta x_r^i} & 0_{12 \times 6} & \frac{\partial e^{[i,j]}(X \boxplus \Delta x)}{\partial \Delta x_r^j} & \dots & 0_{12 \times 6} & \dots \end{pmatrix} \quad (23.20)$$

For small perturbations, the derivative of the prediction $g^{[i,j]}(x^{[i]} \boxplus \Delta x^{[i]}, x^{[j]} \boxplus \Delta x^{[j]})$ with respect to $\Delta x^{[i]}$ evaluated at $\Delta x = 0$ is the negative of the derivative with respect to $\Delta x^{[j]}$ evaluated at $\Delta x = 0$:

$$\begin{aligned} g(X_i \boxplus \Delta x_i, X_j \boxplus \Delta x_j) &= X_i^{[-1]} v2t(\Delta x)^{-1} v2t(\Delta x_j) X_j = \\ &= \begin{pmatrix} R_i^T & -R_i^T t_i \\ & 1 \end{pmatrix} \begin{pmatrix} R(-\Delta \alpha_i) & -\Delta t_i \\ & 1 \end{pmatrix} \begin{pmatrix} R(\Delta \alpha_j) & \Delta t_j \\ & 1 \end{pmatrix} \begin{pmatrix} R_j & t_j \\ & 1 \end{pmatrix} \end{aligned} \quad (23.21)$$

$$v2t(\Delta x)^{-1} \simeq v2t(-\Delta x)$$

$$\left. \frac{\partial g^{[i,j]}(x^{[i]} \boxplus \Delta x^{[i]}, x^{[j]} \boxplus \Delta x^{[j]})}{\partial \Delta x^{[i]}} \right|_{\Delta x=0} = - \left. \frac{\partial g^{[i,j]}(x^{[i]} \boxplus \Delta x^{[i]}, x^{[j]} \boxplus \Delta x^{[j]})}{\partial \Delta x^{[j]}} \right|_{\Delta x=0} \quad (23.22)$$

This allows focusing on computing only one of these derivatives. Focusing on the derivative with respect to $\Delta x^{[j]}$,

$$\begin{aligned} g(X_i, X_j \boxplus \Delta x_j) &= [R_i^T \quad -R_i^T t_i] [R(\Delta \alpha_j) \quad \Delta t_j] [R_j \quad t_j] \\ &= [R_i^T R(\Delta \alpha_j) R_j \quad R_i^T (R(\Delta \alpha_j) t_j + \Delta t_j - t_j)] \end{aligned} \quad (23.23)$$

The derivation w.r.t each component of x_j gives us a 4×4 matrix, of which only the first three rows are relevant. Remembering that

$$\begin{aligned} \frac{\partial g}{\partial \Delta \alpha_x} &= (R_i^T R'_{x0} R_j \quad R_i^T R'_{x0} t_j) & \frac{\partial g}{\partial \Delta t_x} &= (0_{3 \times 3} \quad R_i^T (1 \quad 0 \quad 0)^T) \\ \frac{\partial g}{\partial \Delta \alpha_y} &= (R_i^T R'_{y0} R_j \quad R_i^T R'_{y0} t_j) & \frac{\partial g}{\partial \Delta t_y} &= (0_{3 \times 3} \quad R_i^T (0 \quad 1 \quad 0)^T) \\ \frac{\partial g}{\partial \Delta \alpha_z} &= (R_i^T R'_{z0} R_j \quad R_i^T R'_{z0} t_j) & \frac{\partial g}{\partial \Delta t_z} &= (0_{3 \times 3} \quad R_i^T (0 \quad 0 \quad 1)^T) \end{aligned} \quad (23.24)$$

The final Jacobian²⁹ $J^{[i,j]}(X)$ is assembled by flattening these blocks and zeros for other pose variables:

$$\frac{\partial h(X_i, X_j \boxplus \Delta x_j)}{\partial \Delta x_j} \big|_{\Delta x=0} = (dh_x \quad dh_y \quad dh_z \quad dh_{\alpha_x} \quad dh_{\alpha_y} \quad dh_{\alpha_z}) \quad (23.25)$$

where:

$$\begin{aligned} dh_x &= \text{flatten} \left(\frac{\partial g}{\partial \Delta t_x} \right) & dh_y &= \text{flatten} \left(\frac{\partial g}{\partial \Delta t_y} \right) & dh_z &= \text{flatten} \left(\frac{\partial g}{\partial \Delta t_z} \right) \\ dh_{\alpha_x} &= \text{flatten} \left(\frac{\partial g}{\partial \Delta \alpha_x} \right) & dh_{\alpha_y} &= \text{flatten} \left(\frac{\partial g}{\partial \Delta \alpha_y} \right) & dh_{\alpha_z} &= \text{flatten} \left(\frac{\partial g}{\partial \Delta \alpha_z} \right) \end{aligned} \quad (23.26)$$

The partial derivatives of $g^{[i,j]}(x^{[i]}, x^{[j]} \boxplus \Delta x^{[j]})$ with respect to the components of $\Delta x^{[j]}$ (translation Δt_j and rotation $\Delta \alpha_j$) can be computed component-wise and then flattened to form the columns of $J_j^{[i,j]}$.

To sum up, the **final Jacobian** is:

$$J_j^{[i,j]}(X) = \begin{bmatrix} 0_{9 \times 3} & [r'_x & | & r'_y & | & r'_z] \\ R_i^T & -R_i^T [t_j]_{\times} \end{bmatrix} \quad (23.27)$$

where:

$$r'_x = \text{flatten}(R_i^T R'_{x0} R_j) \quad r'_y = \text{flatten}(R_i^T R'_{y0} R_j) \quad r'_z = \text{flatten}(R_i^T R'_{z0} R_j) \quad (23.28)$$

23.9 Conclusions

Graph-SLAM provides a **robust way to optimize multi-pose registration problems**. By formulating the problem as minimizing a sum of constraints in a graph, and recognizing the **sparsity of the resulting information matrix** H , efficient optimization techniques, particularly those designed for sparse systems like Gauss-Newton with sparse solvers, can be leveraged for computational efficiency. While the basic formulation requires careful linearization and handling of manifold geometry (e.g., using boxplus/boxminus), approximations like the chordal distance can simplify the mathematical derivations and implementation. The considerations on sparsity and rank deficiency observed in pose-landmark problems also hold for pose-pose graph SLAM.

Dealing with outliers and data association ambiguities is a critical challenge in real-world SLAM. Standard least squares, assuming Gaussian noise, is sensitive to outliers. Techniques like Max-Mixtures, Dynamic Covariance Scaling (DCS), and Robust M-Estimators (using robust kernels like Huber or Geman-McClure) are employed to mitigate the impact of outliers. Adaptive robust kernels, particularly when estimated using EM-based approaches, offer flexibility by adapting the loss function shape to the current outlier distribution. However, the core optimization structure based on sparse linear systems remains fundamental to solving large-scale SLAM problems efficiently.

²⁹ 12×6 matrix

24 Factor graphs

Factor graphs provide an efficient representation of estimation problems in robotics. Many relevant estimation problems, including those in SLAM, require estimating a large number of variables based on a large number of measurements. Despite the high number of variables, a single measurement is typically only affected by a subset of the state variables; factor graphs model these dependencies between variables and measurements.

24.1 Large problems

Large problems in robotics estimation involve estimating a large number of variables using a very large number of measurements, the larger the variables the more complex the solution is. Examples of such problems include pose-graphs, pose-landmark SLAM and calibration.

24.2 Large and sparse problems

Despite the large number of variables in problems like SLAM, a single measurement typically influences only a subset of the state variables. For instance:

- landmark measurement is determined by the position of the observer robot pose $X_r^{[n]}$ and the position of the landmark $X_l^{[m]}$;
- pose-pose measurement is determined by the position of the observer robot at time i , $X_r^{[i]}$, and the position of the robot at time j , $X_r^{[j]}$.

This characteristic dependency means that the overall estimation problem, when formulated as minimizing a sum of error functions, leads to a sparse structure.

Formally, the dependency of a measurement $z^{[k]}$ on a subset of state variables $x^{[k]}$ can be highlighted; let:

- the total state variables be $x^T = (x^{[1]T} \dots x^{[n]T})$,
- the error function for a measurement $z^{[k]}$ be $e^{[k]}(x^{[k]})$,

the subset of variables affecting $z^{[k]}$ is

$$x^{[k]T} = (x^{[k_1]T} \dots x^{[k_q]T}) \subset (x^{[1]T} \dots x^{[n]T}) \quad (24.1)$$

The log likelihood of the overall system, assuming Gaussian noise models for measurements and motion, can be expressed as a sum of quadratic error terms:

$$F(x) = \sum_{k \in C} e^{[k]}(x^{[k]})^T \Omega^{[k]} e^{[k]}(x^{[k]}) = \sum_{k \in C} F^{[k]} \quad (24.2)$$

where $\Omega^{[k]}$ is the information matrix for the k -th constraint. The optimum state x^* is found by minimizing this function:

$$x^* = \operatorname{argmin}_x F(x) \quad (24.3)$$

This minimization is a nonlinear least squares problem that can be represented with a factor graph.

24.3 Graphical representation

Factor graphs are bipartite graphs:

- state variables (such as robot poses and landmarks) are represented as nodes;
- measurements are represented as nodes;
- an edge connects a state variable node to a measurement node if the measurement is dependent on that variable. Two variable nodes are connected only through a measurement node. Factors (measurement nodes) can be seen as hyper-edges connecting multiple variables.

Examples include:

- pose-landmark SLAM: nodes represent robot poses and landmarks; edges connect poses to landmarks observed from that pose;
- landmark + odometry: nodes represent robot poses and landmarks; edges connect consecutive poses (odometry) and poses to observed landmarks.

24.4 Structure

The error function (equation ??) is minimized using techniques like Gauss-Newton, so the first step is to linearize the error functions around an initial guess and solving a system of linear equations. The core linear system to solve is $H\Delta x = -b$, where H is the Hessian matrix of $F(x)$ and b is the gradient vector.

The contribution of a single measurement $z^{[k]}$ to the Hessian matrix H and the gradient vector b affects only the state components in $x^{[k]}$. Let $J^{[k]}$ be the Jacobian matrix of the error function $e^{[k]}(x^{[k]})$ with respect to the full state vector x , since $e^{[k]}$ only depends on $x^{[k]}$, $J^{[k]}$ will have zero columns for variables not in $x^{[k]}$. We can write $J^{[k]}$ in block form as:

$$J^{[k]} = (\dots \quad 0 \quad \dots \quad J^{[k_i]} \quad \dots \quad J^{[k_i]} \quad \dots \quad J^{[k_q]} \quad 0 \quad \dots) \quad (24.4)$$

The contribution of $z^{[k]}$ to the Hessian matrix H and to the gradient vector b is

$$J^{[k]T} \Omega^{[k]} J^{[k]} \quad \quad \quad b^{[k]} = J^{[k]T} \Omega^{[k]} e^{[k]} \quad (24.5)$$

The contribution of both the matrix and the vector is sparse, with non-zero blocks only between the variables in $x^{[k]}$ (for the Hessian) and non-zero elements only at the indices corresponding to the variables in $x^{[k]}$ (for the vector):

$$H^{[k]} = J^{[k]T} \Omega^{[k]} J^{[k]} = \begin{pmatrix} \dots & J^{[k_1]T} \Omega^{[k]} J^{[k_1]} & \dots & J^{[k_1]T} \Omega^{[k]} J^{[k_i]} & \dots & J^{[k_1]T} \Omega^{[k]} J^{[k_q]} & \dots \\ \dots & J^{[k_i]T} \Omega^{[k]} J^{[k_1]} & \dots & J^{[k_i]T} \Omega^{[k]} J^{[k_i]} & \dots & J^{[k_i]T} \Omega^{[k]} J^{[k_q]} & \dots \\ \dots & J^{[k_q]T} \Omega^{[k]} J^{[k_1]} & \dots & J^{[k_q]T} \Omega^{[k]} J^{[k_i]} & \dots & J^{[k_q]T} \Omega^{[k]} J^{[k_q]} & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \end{pmatrix} \quad (24.6)$$

$$b^{[k]T} = \begin{pmatrix} J^{[k_1]T} \Omega^{[k]} e^{[k]} \\ J^{[k_i]T} \Omega^{[k]} e^{[k]} \\ J^{[k_q]T} \Omega^{[k]} e^{[k]} \\ \dots \end{pmatrix}$$

The overall Hessian matrix H is the sum of these contributions from all measurements: $H = \sum_{k \in C} H^{[k]}$. The structure of H , i.e., the location of its non-zero elements, depends only on the structure of the measurements (which variables a measurement connects) and does not change during the optimization iterations. This sparse structure reflects the adjacency matrix of the factor graph.

24.5 Structure and efficiency

The number of non-zero blocks in the Hessian matrix H depends on factors like the number of poses, sensor perception range, and landmark density. In typical SLAM problems, the number of measurements and constraints grows linearly with the length of the trajectory, this results in H having a linear number of non-zero blocks, meaning it is mostly empty. The sparsity is crucial for computational efficiency and special techniques designed for sparse linear systems are needed to solve $H\Delta x = -b$ efficiently.

24.6 Solving Sparse Systems

A common approach to solve a symmetric positive definite system of linear equations like $H\Delta x = -b$ is through Cholesky factorization:

1. the matrix H is decomposed into a lower triangular matrix L such that $H = LL^T$;
2. the system then becomes $LL^T \Delta x = -b$;
3. solve for an intermediate vector y using forward substitution: $Ly = -b$;
4. solve for the state update Δx using backward substitution: $L^T \Delta x = y$.

However, a major issue is that if H is sparse, its Cholesky factor L may not be sparse; this is known as "fill-in" and would negate the benefits of sparsity.

24.6.1 Permutations and Cholesky

To address the fill-in problem in Cholesky factorization of sparse matrices, reordering techniques that use a permutation matrix P , which encodes a reordering of the variables, are applied. A permutation matrix has elements that are either 0 or 1, with exactly one non-zero element in each row and column and its inverse is its transpose ($P^{-1} = P^T$).

If a matrix is sparse, there exists a reordering of variables (represented by P) that makes the Cholesky factor L' of the permuted matrix $H' = PHP^T$ as sparse as possible, minimizing fill-in. Finding the optimal ordering is NP-complete and efficient heuristic algorithms exist.

To solve the system $H\Delta x = -b$ using permutation, we need to:

1. apply a permutation: $PHP^T P\Delta x = -Pb$. This can be written as $H'\Delta x' = b'$ where
 - $H' = PHP^T$,
 - $\Delta x' = P\Delta x$,
 - $b' = -Pb$;
2. perform Cholesky decomposition on the permuted sparse matrix H' : $H' = L'L'^T$;
3. solve the system $H'\Delta x' = b'$ using forward and backward substitution: $L'y = b'$ for y , then $L'^T \Delta x' = y$ for $\Delta x'$;
4. recover the original state update Δx by applying the inverse permutation: $\Delta x = P^T \Delta x'$.

24.6.2 Comments

The structure of the Hessian matrix H in factor graph SLAM problems is sparse due to the independence of measurements on large subsets of variables. This sparse structure allows for efficient solution of the linear system $H\Delta x = -b$ using sparse methods; sparse methods are effective because they exploit the matrix structure by finding variable orderings that maintain sparsity in the factorization.

Cholesky decomposition with permutations is one common method, but other techniques like QR factorization or conjugate gradient can also be used.

The initial pose of the robot must be fixed or anchored to define the global coordinate system, otherwise, the system is under-determined, and the Hessian matrix H is singular (not full rank). This is typically handled by adding a prior constraint, often fixing the first pose to the origin (0 0 0) with infinite information. This is represented by adding a term like $x_0^T \Omega_0 x_0$ to the objective function, where Ω_0 is a matrix with infinite values on the diagonal for the initial pose variables.

Alternatively, a subset of variables can be explicitly fixed by constructing the full system and then suppressing the rows and columns corresponding to the variables to be held constant.

The information matrix H provides information about the uncertainty of the estimate given the linearization point. The covariance matrix, which quantifies this uncertainty, can be obtained by inverting H ($\Sigma = H^{-1}$). However, the full covariance matrix is generally dense, even if H is sparse, and its computation can be computationally costly, especially for large systems. Often, only parts of the covariance matrix, such as the diagonal blocks representing the uncertainties of individual variables, are needed. Relative uncertainties between variables can also be computed from the inverse of H .

24.7 Total least squares

The optimization problem in Graph SLAM aims to find the configuration of robot poses and landmark locations that minimize the sum of squared errors from motion and measurement constraints (a total least squares problem). It can involve different types of constraints depending on the representation of the state and the nature of the measurements.

24.7.1 Pose-Landmark Constraint

In a pose-landmark SLAM problem, the state can be represented as the poses of the robot X_r and the locations of landmarks X_l **in the world coordinate frame**. A measurement $z_{\text{icp}}^{[i,j]}$ (e.g., from ICP or a range-bearing sensor converted to relative Cartesian) provides information about the location of landmark $X_l^{[j]}$ relative to the robot pose $X_r^{[i]}$ at time i . The expected observation or prediction function $h_{\text{icp}}^{[i,j]}(X)$ can be formulated based on the geometric relationship. If $X_r^{[i]}$ is represented as a transformation (rotation matrix $R_r^{[i]}$ and translation vector $t_r^{[i]}$) from the robot frame to the world frame, the inverse transformation $X_r^{[i]-1}$ maps from the world

frame to the robot frame. The prediction function and the relative error for observing landmark $X_l^{[j]}$ from pose $X_r^{[i]}$ can be given by:

$$\begin{aligned} h_{\text{icp}}^{[i,j]}(X) &= X_r^{[i]-1} X_l^{[j]} = R_r^{[i]T} (X_l^{[j]} - t_r^{[i]}) \\ e_{\text{icp}}^{[i,j]}(X_r^{[i]} \boxplus \Delta x_r^{[i]}, X_l^{[j]} \boxplus \Delta x_l^{[j]}) &= h_{\text{icp}}^{[i,j]}(X_r^{[i]} \boxplus \Delta x_r^{[i]}, X_l^{[j]} \boxplus \Delta x_l^{[j]}) - z_{\text{icp}}^{[i,j]} \end{aligned} \quad (24.7)$$

The error function considers small perturbations $\Delta x_r^{[i]}$ and $\Delta x_l^{[j]}$ and the $\boxplus$ notation is used to represent adding a perturbation in a way appropriate for the variable's manifold (e.g., adding a twist vector to a pose).

24.7.2 Pose-proj constraint

For sensors like cameras, the measurement might be a projection of a landmark onto an image plane. If the state includes robot poses $X_r^{[i]}$ and landmark locations $X_l^{[j]}$ in the world frame, the prediction function $h_{\text{proj}}^{[i,j]}(X)$ projects the landmark location into the image plane of the camera at pose $X_r^{[i]}$. Assuming a camera intrinsic matrix K and a projection function π (which normalizes coordinates and applies perspective projection):

$$\begin{aligned} h_{\text{proj}}^{[i,j]}(X) &= \pi \left(K(X_r^{[i]-1} X_l^{[j]}) \right) = \pi \left(K R_r^{[i]T} (X_l^{[j]} - t_r^{[i]}) \right) \\ e_{\text{proj}}^{[i,j]}(X_r^{[i]} \boxplus \Delta x_r^{[i]}, X_l^{[j]} \boxplus \Delta x_l^{[j]}) &= h_{\text{proj}}^{[i,j]}(X_r^{[i]} \boxplus \Delta x_r^{[i]}, X_l^{[j]} \boxplus \Delta x_l^{[j]}) - z_{\text{proj}}^{[i,j]} \end{aligned} \quad (24.8)$$

The error function $e_{\text{proj}}^{[i,j]}$ is the difference between this predicted measurement and the actual measured image coordinates $z_{\text{proj}}^{[i,j]}$.

24.7.3 Pose-pose constraint

Pose-pose constraints relate two robot poses $X_r^{[i]}$ and $X_r^{[j]}$. These constraints can arise from odometry measurements between consecutive poses or from "virtual" observations like scan matching between non-consecutive poses (loop closures).

If the state variables are the poses in the world frame, the expected relative transformation between pose $X_r^{[i]}$ and $X_r^{[j]}$ can be calculated as $X_r^{[i]-1} X_r^{[j]}$. This relative transformation can be represented as a vector by a 'flatten' or 't2v' function that maps the transformation matrix to a vector (e.g., (x, y, θ) for 2D poses). The prediction and error functions for a pose-pose constraint are:

$$h_{\text{p2p}}^{[i,j]}(X) = \text{flatten}(X_r^{[i]-1} X_r^{[j]}) \quad e_{\text{p2p}}^{[i,j]}(X) = h_{\text{p2p}}^{[i,j]}(X) - \text{flatten}(Z_{\text{p2p}}^{[i,j]}) \quad (24.9)$$

Considering state perturbations, the error function is:

$$e_{\text{p2p}}^{[i,j]}(X_r^{[i]} \boxplus \Delta x_r^{[i]}, X_r^{[j]} \boxplus \Delta x_r^{[j]}) = h_{\text{p2p}}^{[i,j]}(X_r^{[i]} \boxplus \Delta x_r^{[i]}, X_r^{[j]} \boxplus \Delta x_r^{[j]}) - \text{flatten}(Z_{\text{p2p}}^{[i,j]}) \quad (24.10)$$

24.8 Conclusions

The overall GraphSLAM problem involves minimizing the sum of squared errors for all these constraints. The sparse structure of the resulting Hessian matrix and the use of efficient sparse solvers are key to making GraphSLAM computationally feasible for large environments.